# Python for Everybody

Python para Todos es una introducción a los conceptos básicos de la programación en Python 3 con un énfasis en el uso práctico. Está pensado como una base para los estudiantes que buscan aplicar Python en el contexto otras materias académicas, así como para la preparación para el estudio serio de informática.


- Introducción
- 1 Comenzando el viaje
- 2 Variables
- 3 Condicionales
- 4 Funciones
- 5 Bucles
- 6 Cadenas
- 7 Archivos
- 8 Listas
- 9 Diccionarios
- 10 Tuplas
- 11 Regex
- 12 Programas en red
- 13 Python y servicios web
- 14 Objetos
- 15 Python y Bases de datos
- Créditos


Copyright 2026

# Introducción

## Por Charles Severance

Python para Todos es una introducción a los conceptos básicos de la programación en Python 3 con un énfasis en el uso práctico. Está pensado como una base para los estudiantes que buscan aplicar Python en el contexto otras materias académicas, así como para la preparación para el estudio serio de informática.

> Este curso ha sido revisado por CATEDU. Se ofrece por su calidad y porque el autor lo ha licenciado bajo Creative Commons, haciendo un con ello un enorme regalo a la comunidad. En agradecimiento y por respeto a su material original, añadimos los enlaces de compra de la versión comercial de sus libros justo debajo de estas líneas.
> No obstante, no es necesario comprarlos ni se consigue ningún material adicional. También tenéis enlaces a la descarga de este curso y a la web del autor, donde él mismo ofrece a su vez su curso en multitud de formatos de manera libre y gratuita.

## Other Resources

- Una edición impresa está disponible para su compra en Amazon.
- Una edición Kindle está disponible para su compra en Amazon.
- La web del autor contiene mucha más información sobre el libro, así como PDFs, ebooks y videos descargables.

Copyright 2026

# 1 Comenzando el viaje

# Instalando Python

Para instalar Python en nuestro ordenador, sólo tendremos que ir a la web https://www.python.org/ y descargar el instalador.

Los pasos varían algo entre sistemas operativos.

¡Atención! En Windows, no olvides añadir Python al PATH en los primeros pasos de instalación:




Imagen - Instalando Python

Una vez hecho eso ya tendremos accesible Python desde la terminal.

Copyright 2026

https://www.youtube.com/embed/9fNKy9zOPkg

# ¿Por qué debería aprender a escribir programas?

Escribir programas (o programar) es una actividad muy creativa y gratificante. Puedes escribir programas por muchas razones, desde ganarte la vida, resolver un problema difícil de análisis de datos, hasta divertirte o ayudar a alguien más a resolver un problema. Este libro asume que **todos** debemos saber cómo programar, y que una vez que sepa cómo programar, descubrirás qué quieres hacer con tus nuevas habilidades.

Estamos rodeados en nuestra vida diaria de ordenadores que van desde portátiles hasta teléfonos móviles. Podemos pensar en estos ordenadores como nuestros "asistentes personales" que pueden hacerse cargo de muchas cosas en nuestro nombre. El hardware en nuestros ordenadores de hoy en día está esencialmente construido para hacernos continuamente la pregunta "¿Qué quieres que haga ahora?"

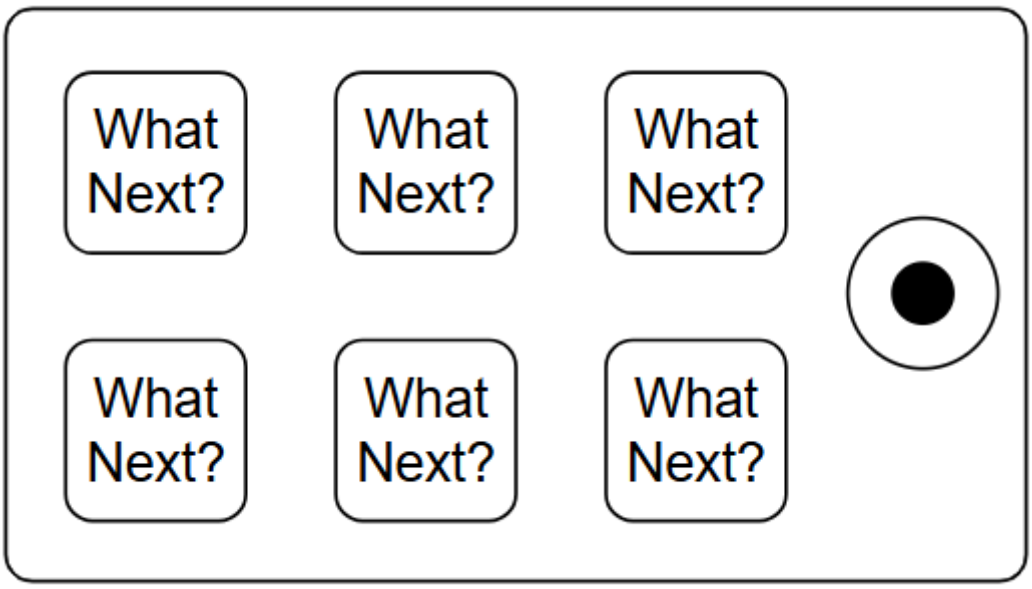



Imagen - Asistente personal digital

Los programadores agregan un sistema operativo y un conjunto de aplicaciones al hardware y terminamos con un Asistente digital personal que es bastante útil y capaz de ayudarnos a hacer muchas cosas diferentes.

Copyright 2026

Nuestros ordenadores son rápidos y tienen una gran cantidad de memoria y podrían ser muy útiles para nosotros si supiéramos el idioma con el que decirle al ordenador lo que nos gustaría "hacer a continuación". Si conociéramos este idioma, podríamos decirle al ordenador que realice tareas en nuestro nombre que son repetitivas. Curiosamente, las cosas que los ordenadores pueden hacer mejor son a menudo el tipo de cosas que a los humanos nos parecen aburridas.

Por ejemplo, mira los primeros tres párrafos de este capítulo, dime la palabra que se usa con más frecuencia y cuántas veces se usa dicha palabra. Si bien pudiste leer y entender las palabras en unos pocos segundos, contarlas es casi doloroso porque no es el tipo de problema que las mentes humanas están diseñadas para resolver. Para una ordenador es todo lo contrario, leer y entender texto de una hoja de papel es difícil, pero contar las palabras y decirle cuántas veces se usó la palabra más usada es muy fácil:

```
python words.py
    Enter file:words.txt
    to 16
```

Nuestro "asistente de análisis de información personal" rápidamente nos dijo que la palabra "to" se usó dieciséis veces en los primeros tres párrafos de este capítulo (válido para la versión inglesa de este libro).

Para usar ese poder de la máquina, necesitamos adquirir la habilidad para hablar el "lenguaje del ordenador". Una vez que aprendas este nuevo idioma, puedes delegar tareas mundanas a su compañero (el ordenador), lo que te deja más tiempo para hacer las cosas para las que estás preparado de manera única. Traes creatividad, intuición e inventiva a esta asociación.

# Creatividad y motivación

Si bien este libro no está dirigido a programadores profesionales, la programación profesional puede ser un trabajo muy gratificante tanto a nivel financiero como personal. Crear programas útiles, elegantes e inteligentes para que otros puedan usar es una actividad muy creativa. Su ordenador usualmente contiene muchos programas diferentes, de diferentes grupos de programadores, cada uno compitiendo por su atención e interés. Hacen todo lo posible para satisfacer sus necesidades y le brindan una excelente experiencia de usuario en el proceso. En algunas situaciones, cuando elige una pieza de software, los programadores reciben una compensación directa debido a su elección.

Si pensamos en los programas como la producción creativa de grupos de programadores, tal vez la siguiente figura sea una versión posible de nuestra PDA:

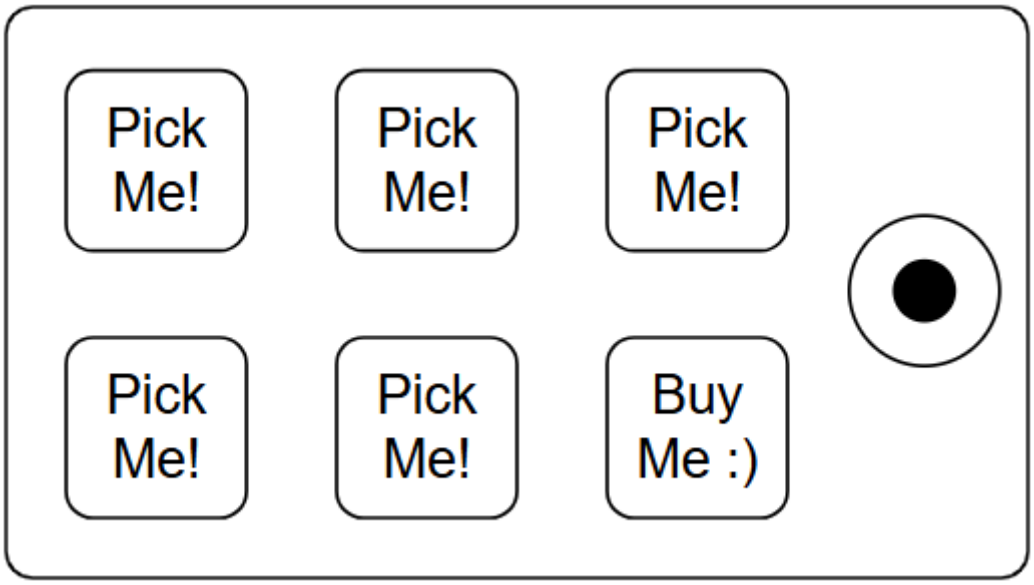
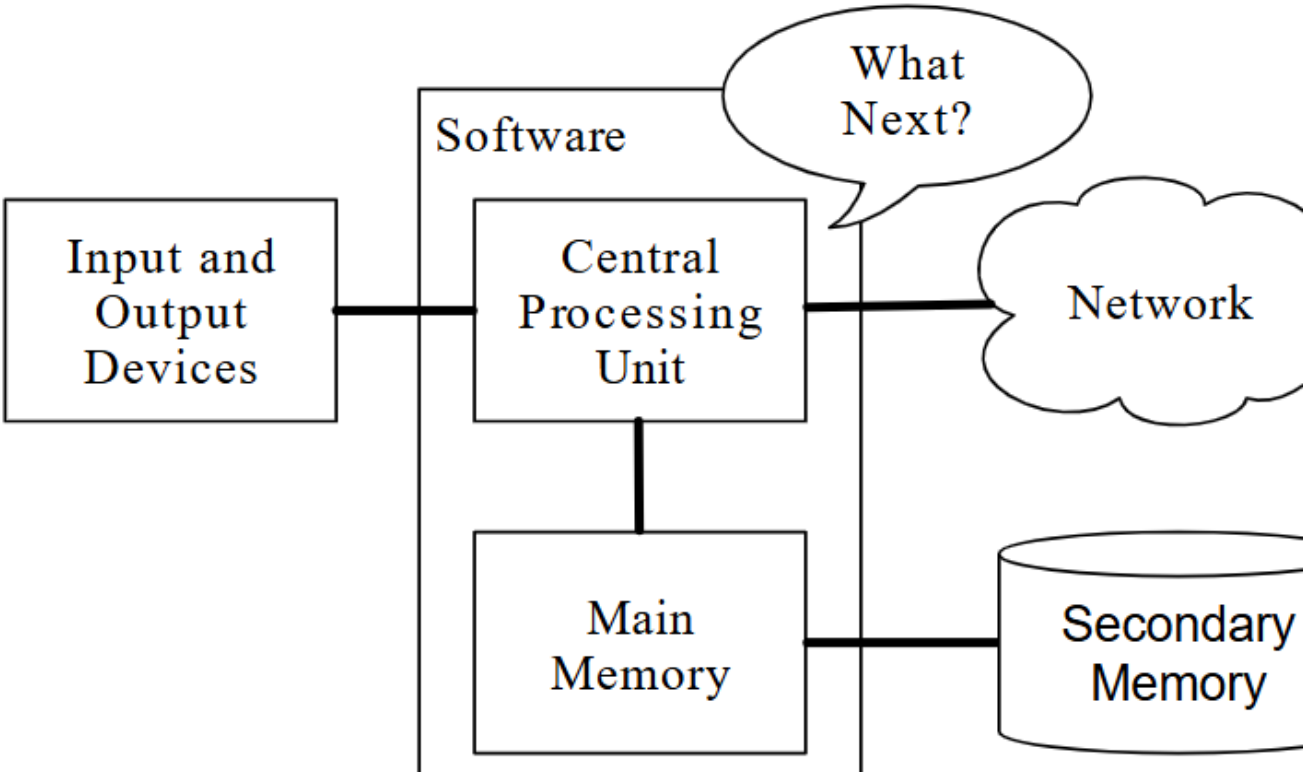



Imagen - Programadores hablando contigo

Por ahora, nuestra principal motivación no es ganar dinero o complacer a los usuarios finales, sino que seamos más productivos en el manejo de los datos y la información que encontraremos en nuestras vidas. Cuando comiences, serás el programador y el usuario final de tus programas. A medida que adquieras destreza como programador, tus pensamientos pueden dirigirse hacia el desarrollo de programas para otros.

# Arquitectura de hardware del ordenador

Antes de comenzar a aprender el idioma que hablamos para crear software, necesitamos aprender un poco sobre cómo se construyen los ordenadores. Si tuviera que desarmar su ordenador o teléfono celular y mirar adentro, encontraría las siguientes partes:




Copyr

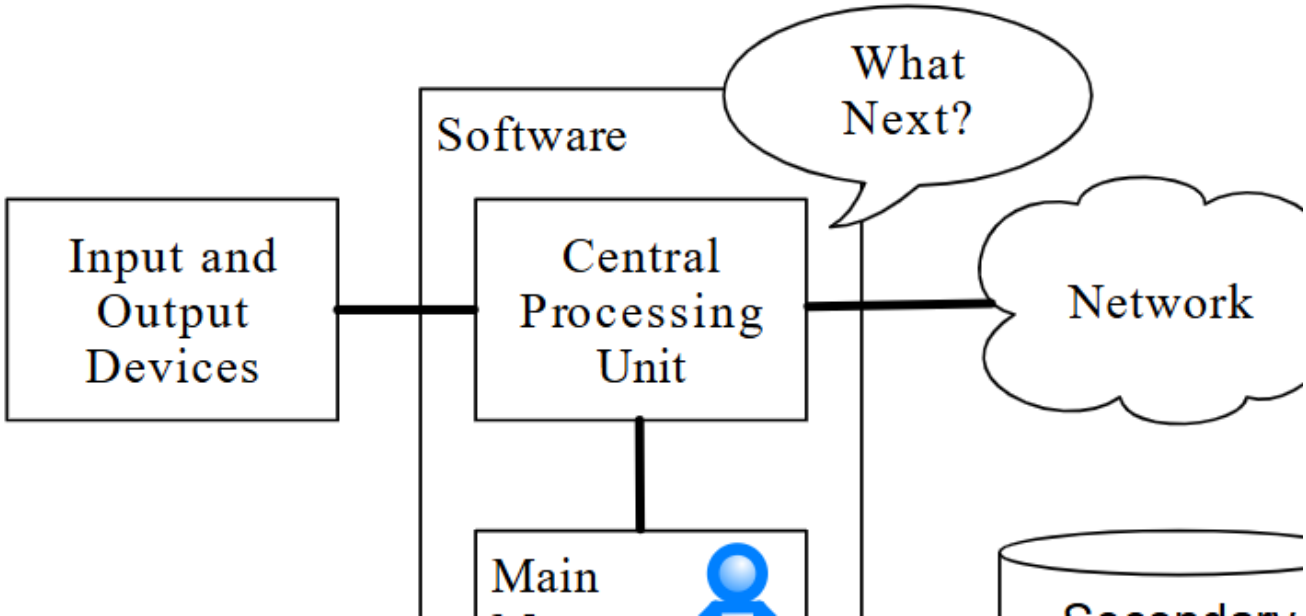
Imagen - Arquitectura de hardware

Las definiciones de alto nivel de estas partes son las siguientes:

- La **Unidad central de procesamiento** (o CPU) es la parte de el ordenador que está diseñada para estar obsesionada con "¿qué sigue?" Si tu ordenador tiene una clasificación de 3.0 Gigahercios, significa que la CPU preguntará "¿Qué sigue?" Tres mil millones de veces por segundo. Tendrás que aprender a hablar rápido para mantenerse al día con la CPU.
- La **memoria principal** se utiliza para almacenar información que la CPU necesita a toda prisa. La memoria principal es casi tan rápida como la CPU. Pero la información almacenada en la memoria principal se desvanece cuando se apaga el ordenador.
- La **memoria secundaria** también se usa para almacenar información, pero es mucho más lenta que la memoria principal. La ventaja de la memoria secundaria es que puede almacenar información incluso cuando no hay energía en el ordenador. Algunos ejemplos de memoria secundaria son las unidades de disco o la memoria flash (que generalmente se encuentran en memorias USB)
- Los **Dispositivos de entrada y salida** son simplemente nuestra pantalla, teclado, mouse, micrófono, altavoz, panel táctil, etc. Son todas las formas en que interactuamos con el ordenador.
- **Conexión de red** para recuperar información a través de una red. Podemos pensar en la red como un lugar muy lento para almacenar y recuperar datos que pueden no estar siempre "arriba". Entonces, en cierto sentido, la red es una forma más lenta y en ocasiones poco confiable de **Memoria secundaria**.

Si bien la mayoría de los detalles de cómo funcionan estos componentes es mejor dejarlos en manos de los desarrolladores, es útil tener un poco de terminología para que podamos hablar sobre estas partes diferentes a medida que escribimos nuestros programas.

Como programador, tu trabajo consiste en utilizar y organizar cada uno de estos recursos para resolver el problema que sea y analizar los datos que obtienes de la solución. En general, estarás "hablando" con la CPU y le dirás qué hacer a continuación. A veces le dirás a la CPU que use la memoria principal, la memoria secundaria, la red o los dispositivos de entrada/salida.




Copyr

Imagen - ¿Dónde estás?

Necesitas ser la persona que responde a la CPU "¿Qué sigue?". Pero sería muy incómodo reducirte a una altura de 5 mm e insertarte en el ordenador solo para que puedas emitir un comando tres mil millones de veces por segundo. Así que, en lugar de eso, debes escribir tus instrucciones con anticipación. Llamamos a estas instrucciones almacenadas un **programa**, y al hecho de escribir estas instrucciones para que sean correctas, **programación**.

# Entendiendo la programación

En el resto de este libro, trataremos de convertirte en una persona experta en el arte de la programación. Al final, serás un **programador** - quizás no un programador profesional, pero al menos tendrás las habilidades para analizar un problema de análisis de datos/información y desarrollar un programa para resolver el problema.

En cierto sentido, necesitas dos habilidades para ser un programador:

- Primero, necesitas saber el lenguaje de programación (Python) - necesitas conocer el vocabulario y la gramática. Debes poder escribir correctamente las palabras en este nuevo idioma y saber cómo construir "oraciones" bien formadas en este nuevo idioma.
- Segundo, necesitas "contar una historia". Al escribir una historia, combinas palabras y oraciones para transmitir una idea al lector. Hay una habilidad y arte en la construcción de la historia, y la habilidad en la escritura de la historia se mejora al escribir un poco y obtener algunos comentarios. En programación, nuestro programa es la "historia" y el problema que intenta resolver es la "idea".

Una vez que aprendas un lenguaje de programación como Python, te resultará mucho más fácil aprender un segundo lenguaje de programación como JavaScript o C++. El nuevo lenguaje de programación tiene un vocabulario y gramática muy diferentes, pero las habilidades de resolución de problemas serán las mismas en todos los lenguajes de programación.

Aprenderás el "vocabulario" y las "oraciones" de Python con bastante rapidez. Te llevará más tiempo poder escribir un programa coherente para resolver un problema completamente nuevo. Enseñamos la programación como enseñamos a escribir. Comenzamos a leer y explicar programas, luego escribimos programas simples y luego escribimos programas cada vez más complejos a lo largo del tiempo. En algún momento, "obtienes tu musa" y ves los patrones por tu cuenta y puedes ver más naturalmente cómo tomar un problema y escribir un programa que resuelva ese problema. Y una vez que llegas a ese punto, la programación se convierte en un proceso muy agradable y creativo.

Copyright 2026

Comenzamos con el vocabulario y la estructura de los programas de Python.

# Palabras y oraciones

A diferencia de las lenguas humanas, el vocabulario de Python es bastante pequeño. Llamamos a este "vocabulario" las "palabras reservadas". Estas son palabras que tienen un significado muy especial para Python. Cuando Python ve estas palabras en un programa de Python, tienen un solo significado para Python. Luego, a medida que escribes programas, inventarás sus propias palabras que tienen un significado para ti llamadas **variables**. Tendrás gran libertad para elegir tus nombres para sus variables, pero no puedes usar ninguna de las palabras reservadas de Python como nombre para una variable.

Cuando entrenamos a un perro, usamos palabras especiales como "sentarse", "quedarse" y "buscar". Cuando le hablas a un perro y no usas ninguna de las palabras reservadas, solo te miran con una expresión burlona hasta que dices una palabra reservada. Por ejemplo, si dices: "Me gustaría que más gente caminara para mejorar su salud general", lo que la mayoría de los perros escuchan es "bla bla bla **caminar** bla bla bla bla". Esto se debe a que "caminar" es una palabra reservada en lenguaje canino. Muchos podrían sugerir que el lenguaje entre humanos y gatos no tiene palabras reservadas[1].

Las palabras reservadas en el idioma donde los humanos hablan con Python incluyen lo siguiente:

```
and       del       global      not       with
as        elif      if          or        yield
assert    else      import      pass
break     except    in          raise
class     finally   is          return
continue  for       lambda      try
def       from      nonlocal    while
```

Eso es todo, y a diferencia de un perro, Python ya está completamente entrenado. Cuando dices "try", Python lo intentará cada vez que lo digas.

Aprenderemos estas palabras reservadas y cómo se usan a tiempo, pero por ahora nos enfocaremos en el equivalente de Python de "hablar" (en lenguaje humano a perro). Lo bueno de decirle a Python que hable es que incluso podemos decirle qué decir dándole un mensaje entre comillas:

```
print('Hello world!')
```

E incluso hemos escrito nuestra primera oración de Python sintácticamente correcta. Nuestra oración comienza con la función **print** (imprimir) seguida de una cadena de texto de nuestra elección entre comillas simples.

# Conversando con Python

Ahora que tenemos una palabra y una oración simple que conocemos en Python, necesitamos saber cómo iniciar una conversación con Python para probar nuestras nuevas habilidades lingüísticas.

Antes de poder conversar con Python, primero debes instalar el software de Python en tu ordenador y aprender a iniciar Python. Esto es demasiado detallado para este capítulo, por lo que te sugiero que consulte [www.py4e.com] (http://www.py4e.com), donde tengo instrucciones detalladas y recomendaciones para configurar e iniciar Python en sistemas Macintosh y Windows . En algún momento, estarás en una ventana de comandos o terminal y escribirás **python** y el intérprete de Python comenzará a ejecutarse en modo interactivo. Aparecerá de la siguiente manera:

```
Python 3.5.1 (v3.5.1:37a07cee5969, Dec  6 2015, 01:54:25)
[MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

El indicador `>>>` es la forma en que el intérprete de Python te pregunta: "¿Qué quieres que haga ahora?" Python está listo para tener una conversación contigo. Todo lo que tienes que saber es cómo hablar el idioma Python.

Digamos, por ejemplo, que no conocías ni las palabras ni las oraciones más simples del lenguaje Python. Es posible que desees utilizar la línea estándar que usan los astronautas cuando aterrizan en un planeta lejano y tratan de hablar con los habitantes del planeta:

```
>>> I come in peace, please take me to your leader
  File "<stdin>", line 1
    I come in peace, please take me to your leader
         ^
SyntaxError: invalid syntax
```

```
>>>
```

Esto no va tan bien. A menos que pienses en algo rápido, es probable que los habitantes del planeta te apuñalen con sus lanzas, te pongan a escupir, te asen al fuego y te coman para la cena.

Afortunadamente, trajiste una copia de este libro en tus viajes, y accedes a esta misma página e intentas nuevamente:

```
>>> print('Hello world!')
Hello world!
```

Esto se ve mucho mejor, así que intenta comunicar algo más:

```
>>> print('You must be the legendary god that comes from the sky')
You must be the legendary god that comes from the sky
>>> print('We have been waiting for you for a long time')
We have been waiting for you for a long time
>>> print('Our legend says you will be very tasty with mustard')
Our legend says you will be very tasty with mustard
>>> print 'We will have a feast tonight unless you say
  File "<stdin>", line 1
    print 'We will have a feast tonight unless you say
                                                      ^
SyntaxError: Missing parentheses in call to 'print'
>>>
```

La conversación transcurrió muy bien durante un tiempo y luego cometiste el más mínimo error al usar el lenguaje y Python sacó las lanzas.

En este punto, también debes darte cuenta de que, si bien Python es increíblemente complejo, poderoso y muy exigente con la sintaxis que utilizas para comunicarte con él, Python no es **inteligente**. Realmente solo estás teniendo una conversación contigo mismo, pero usando la sintaxis adecuada.

En cierto sentido, cuando utilizas un programa escrito por otra persona, la conversación es entre usted y los otros programadores con Python que actúa como intermediario. Python es una forma para que los creadores de programas expresen cómo se supone que la conversación debe continuar. Y en unos pocos capítulos más, serás uno de esos programadores que usan Python para hablar con los usuarios de tu programa.

Copyright 2026

Antes de dejar nuestra primera conversación con el intérprete de Python, probablemente debas saber la forma correcta de decir "adiós" cuando interactúas con los habitantes de Planet Python:

```
>>> good-bye
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'good' is not defined
>>> if you don't mind, I need to leave
  File "<stdin>", line 1
    if you don't mind, I need to leave
             ^
SyntaxError: invalid syntax
>>> quit()
```

Notarás que el error es diferente para los dos primeros intentos incorrectos. El segundo error es diferente porque **if** es una palabra reservada y Python vio la palabra reservada y pensó que estábamos tratando de decir algo, pero te equivocaste en la sintaxis de la oración.

La forma correcta de decir "adiós" a Python es ingresar **quit()** en el indicador interactivo `>>>`. Probablemente te haya tomado bastante tiempo adivinarlo, por lo que tener un libro a la mano probablemente te resulte útil.

# Terminología: intérprete y compilador

Python es un lenguaje **de alto nivel** destinado a ser relativamente sencillo para que los humanos lean y escriban y para que los ordenadores lean y procesen. Otros lenguajes de alto nivel incluyen Java, C ++, PHP, Ruby, Basic, Perl, JavaScript y muchos más. El hardware real dentro de la Unidad Central de Procesamiento (CPU) no comprende ninguno de estos lenguajes de alto nivel.

La CPU entiende un idioma que llamamos **lenguaje de máquina**. El lenguaje de máquina es muy simple y francamente muy tedioso de escribir porque está representado en ceros y unos:

```
00101000111010010010101010000001111
11100110000011101010010101101101
...
```

El lenguaje de máquina parece bastante simple en la superficie, dado que solo hay ceros y unos, pero su sintaxis es aún más compleja y mucho más compleja que Python. Muy pocos

programadores escriben lenguaje de máquina. En su lugar, creamos varios traductores para permitir que los programadores escriban en lenguajes de alto nivel como Python o JavaScript y estos traductores convierten los programas al lenguaje de máquina para su ejecución real por parte de la CPU.

Dado que el lenguaje de la máquina está vinculado al hardware de el ordenador, el lenguaje de la máquina no es **portátil** a través de diferentes tipos de hardware. Los programas escritos en lenguajes de alto nivel se pueden mover entre diferentes ordenadors utilizando un intérprete diferente en la nueva máquina o compilando el código para crear una versión del programa en la máquina.

Estos traductores de lenguaje de programación se dividen en dos categorías generales: (1) intérpretes y (2) compiladores.

Un **intérprete** lee el código fuente del programa como está escrito por el programador, analiza el código fuente e interpreta las instrucciones sobre la marcha. Python es un intérprete y cuando ejecutamos Python de forma interactiva, podemos escribir una línea de Python (una oración) y Python la procesa de inmediato y está lista para que escribamos otra línea de Python.

Algunas de las líneas le dicen a Python que quieres que recuerde algún valor para más adelante. Necesitamos elegir un nombre para que se recuerde ese valor y podemos usar ese nombre simbólico para recuperar el valor más adelante. Utilizamos el término **variable** para referirnos a las etiquetas que usamos para estos datos almacenados.

```
>>> x = 6
>>> print(x)
6
>>> y = x * 7
>>> print(y)
42
>>>
```

En este ejemplo, le pedimos a Python que recuerde el valor seis y use la etiqueta **x** para poder recuperar el valor más adelante. Verificamos que Python realmente haya recordado el valor utilizando **print**. Luego le pedimos a Python que recupere **x** y lo multiplique por siete y ponga el nuevo valor calculado en **y**. Luego le pedimos a Python que imprima el valor actualmente en **y**.

Aunque estamos escribiendo estos comandos en Python una línea a la vez, Python los trata como una secuencia ordenada de declaraciones, con declaraciones posteriores que pueden recuperar datos creados anteriomente. Estamos escribiendo nuestro primer párrafo simple con cuatro oraciones en un orden lógico y significativo.

Copyright 2026

Está en la naturaleza de un **intérprete** poder tener una conversación interactiva como se muestra arriba. A un **compilador** debemos entregarle todo el programa en un archivo, y luego ejecuta un proceso para traducir el código fuente de alto nivel al lenguaje de máquina y luego el compilador coloca el lenguaje de máquina resultante en un archivo para su posterior ejecución.

Si tienes un sistema Windows, a menudo estos programas de lenguaje de máquina ejecutable tienen un sufijo de ".exe" o ".dll" que significa "ejecutable" y "biblioteca de enlace dinámico" respectivamente. En Linux y Macintosh, no hay ningún sufijo que marque de forma única un archivo como ejecutable.

Si fueras a abrir un archivo ejecutable en un editor de texto, se vería completamente loco y sería ilegible:

```
^?ELF^A^A^A^@^@^@^@^@^@^@^@^@^B^@^C^@^A^@^@^@\xa0\x82
^D^H4^@^@^@\x90^]^@^@^@^@^@^@4^@ ^@^G^@(^@$^@!^@^F^@
^@^@4^@^@^@4\x80^D^H4\x80^D^H\xe0^@^@^@\xe0^@^@^@^E
^@^@^@^D^@^@^@^C^@^@^@^T^A^@^@^T\x81^D^H^T\x81^D^H^S
^@^@^@^S^@^@^@^D^@^@^@^A^@^@^@^A\^D^HQVhT\x83^D^H\xe8
....
```

No es fácil leer o escribir lenguaje de máquina, por lo que es bueno que tengamos **intérpretes** y **compiladores** que nos permiten escribir en lenguajes de alto nivel como Python o C.

Ahora, en este punto de nuestra discusión sobre compiladores e intérpretes, deberías preguntarte un poco sobre el intérprete de Python. ¿En qué idioma está escrito? ¿Está escrito en un lenguaje compilado? Cuando escribimos "python", ¿qué está sucediendo exactamente?

El intérprete de Python está escrito en un lenguaje de alto nivel llamado "C". Puede consultar el código fuente real del intérprete de Python en [www.python.org] (http://www.python.org). Entonces, Python es un programa en sí mismo y está compilado en código de máquina. Cuando instaló Python en su ordenador (o el proveedor lo instaló), hizo una copia en código de máquina del programa Python traducido en su sistema. En Windows, el código de máquina ejecutable para Python en sí mismo es probablemente un archivo con un nombre como:

```
C:\Python37\python.exe
```

Eso es más de lo que realmente necesitas saber para ser un programador de Python, pero a veces vale la pena responder esas pequeñas y molestas preguntas desde el principio.

# Escribiendo un programa

Escribir comandos en el intérprete de Python es una excelente manera de experimentar con las funciones de Python, pero no se recomienda para resolver problemas más complejos.

Cuando queremos escribir un programa, usamos un editor de texto para escribir las instrucciones de Python en un archivo, que se denomina un **script**. Por convención, los scripts de Python tienen nombres que terminan con `.py`.

Para ejecutar el script, debes decirle al intérprete de Python el nombre del archivo. En una ventana de comandos de Unix o Windows, escribirías `python hello.py` de la siguiente manera:

```
csev$ cat hello.py
print('Hello world!')
csev$ python hello.py
Hello world!
csev$
```

El "csev $" es el indicador del sistema operativo y el "cat hello.py" nos muestra que el archivo "hello.py" tiene un programa de Python de una línea para imprimir una cadena.

Llamamos al intérprete de Python y le decimos que lea el código fuente del archivo "hello.py" en lugar de pedirnos líneas interactivas de código de Python.

Notarás que no había necesidad de tener **quit()** al final del programa Python en el archivo. Cuando Python está leyendo el código fuente de un archivo, sabe que debe detenerse cuando llega al final del archivo.

# ¿Qué es un programa?

La definición de un **programa** en su forma más básica es una secuencia de sentencias de Python que se han creado para hacer algo. Incluso nuestro sencillo script **hello.py** es un programa. Es un programa de una línea y no es particularmente útil, pero en la definición más estricta, es un programa de Python.

Podría ser más fácil entender qué es un programa pensando en un problema para el cual se podría construir un programa, y luego mirando un programa que solucione ese problema.

Copyright 2026

Digamos que estás haciendo una investigación de Computación Social en las publicaciones de Facebook y estás interesado en la palabra más utilizada en una serie de publicaciones. Podrías imprimir la corriente de publicaciones de Facebook y estudiar detenidamente el texto buscando la palabra más común, pero eso llevaría mucho tiempo y sería muy propenso a errores. Sería inteligente escribir un programa Python para manejar la tarea de manera rápida y precisa, de modo que puedas pasar el fin de semana haciendo algo divertido.

Por ejemplo, mira el siguiente texto sobre un payaso y un automóvil. Calcula la palabra más común y cuántas veces aparece.

```
the clown ran after the car and the car ran into the tent
and the tent fell down on the clown and the car
```

Luego imagina que estás haciendo esta tarea mirando millones de líneas de texto. Francamente, sería más rápido aprender Python y escribir un programa Python para contar las palabras que escanearlas manualmente.

La noticia aún mejor es que ya se me ocurrió un programa simple para encontrar la palabra más común en un archivo de texto. Lo escribí, lo probé y ahora te lo doy para que lo uses y puedas ahorrar algo de tiempo.

https://trinket.io/embed/python3/d51dc614a8

Ni siquiera necesitas saber Python para usar este programa. Deberás leer el Capítulo 10 de este libro para comprender completamente las impresionantes técnicas de Python que se utilizaron para crear el programa. Tú eres el usuario final, simplemente utiliza el programa y disfruta de su inteligencia y de cómo te ahorró tanto esfuerzo manual. Simplemente escribe el código en un archivo llamado **words.py** y ejecútalo o descárgate el código fuente de https://www.py4e.com/code3/ y ejecútalo.

Este es un buen ejemplo de cómo Python actúa como intermediario entre tú (el usuario final) y yo (el programador). Python es una forma de intercambiar secuencias de instrucciones útiles (es decir, programas) en un lenguaje común que puede ser utilizado por cualquier persona que instale Python en su ordenador. Así que ninguno de los dos está hablando **con Python**, en lugar de eso, nos estamos comunicando unos con otros **a través de** Python.

Copyright 2026

# Los componentes básicos de los programas

En los siguientes capítulos, aprenderemos más sobre el vocabulario, la estructura de las oraciones, la estructura de los párrafos y la estructura de la historia de Python. Aprenderemos sobre las potentes capacidades de Python y cómo usarlas para crear programas útiles.

Hay algunos patrones conceptuales de bajo nivel que utilizamos para construir programas. Estas construcciones no son solo para programas de Python, sino que forman parte de todos los lenguajes de programación, desde el lenguaje de máquina hasta los lenguajes de alto nivel.

## entrada (input)

Obtener datos del "mundo exterior". Esto podría ser leer datos de un archivo, o incluso algún tipo de sensor como un micrófono o un GPS. En nuestros programas iniciales, nuestra entrada provendrá del usuario que escribe datos en el teclado.

## salida (output)

Muestra los resultados del programa en una pantalla o guárdalos en un archivo o tal vez envíalos a un dispositivo como un altavoz para reproducir música o leer texto.

## ejecución secuencial

Realiza las instrucciones una tras otra en el orden en que se encuentran en el script.

## ejecución condicional

Verifica ciertas condiciones y luego ejecuta u omite una secuencia de sentencias.

## ejecución iterativa

Realizar algún conjunto de declaraciones repetidamente, generalmente con alguna variación.

## reutilizar

Escribe un conjunto de instrucciones una vez y dales un nombre. Luego reutiliza esas instrucciones según sea necesario en todo el programa.

Suena casi demasiado simple para ser verdad, y por supuesto nunca es tan simple. Es como decir que caminar es simplemente "poner un pie delante del otro". El "arte" de escribir un programa es

Copyright 2026

componer y tejer estos elementos básicos juntos muchas veces para producir algo que sea útil para sus usuarios.

El programa de conteo de palabras de arriba usa directamente todos estos patrones excepto uno.

# ¿Qué podría salir mal?

Como vimos en nuestras primeras conversaciones con Python, debemos comunicarnos con mucha precisión cuando escribimos el código de Python. La desviación o error más pequeño hará que Python deje de mirar tu programa.

Los programadores principiantes a menudo toman el hecho de que Python no deja espacio para errores como evidencia de que Python es malo, odioso y cruel. Si bien a Python parece gustarle todo el mundo, Python en realidad los conoce personalmente y guarda rencor contra ellos. Debido a este rencor, Python toma nuestros programas perfectamente escritos y los rechaza como "no aptos" solo para atormentarnos.

```
>>> primt 'Hello world!'
  File "<stdin>", line 1
    primt 'Hello world!'
                        ^
SyntaxError: invalid syntax
>>> primt ('Hello world')
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
NameError: name 'primt' is not defined

>>> I hate you Python!
  File "<stdin>", line 1
    I hate you Python!
         ^
SyntaxError: invalid syntax
>>> if you come out of there, I would teach you a lesson
  File "<stdin>", line 1
    if you come out of there, I would teach you a lesson
               ^
SyntaxError: invalid syntax
```

```
>>>
```

Hay poco que ganar discutiendo con Python. Es solo una herramienta. No tiene emociones y está listo para servirte cuando lo necesite. Sus mensajes de error suenan ásperos, pero son solo la llamada de ayuda de Python. Ha examinado lo que escribiste y simplemente no puede entender lo que has ingresado.

Python es mucho más como un perro, te ama incondicionalmente, tiene algunas palabras clave que entiende, te mira con una mirada dulce en su cara (`>>>`), y te espera para que digas algo que entiendas. Cuando Python dice "SyntaxError: sintaxis no válida", simplemente mueve la cola y dice: "Parecías decir algo, pero no entiendo lo que quisiste decir, pero sigue hablando (`>>>`)."

A medida que sus programas se vuelven cada vez más sofisticados, encontrará tres tipos generales de errores:

## Errores sintácticos

Estos son los primeros errores que cometerás y los más fáciles de corregir. Un error de sintaxis significa que has violado las reglas de "sintaxis" de Python. Python hace todo lo posible para señalar la línea y el carácter en el que notó que estabas confundido. La única parte difícil de los errores de sintaxis es que, a veces, el error que debe solucionarse es, en realidad, más temprano en el programa que en el que Python notó que estabas confundido. Por lo tanto, la línea y el carácter que Python indica en un error de sintaxis pueden ser solo un punto de partida para tu investigación.

## Errores lógicos

Un error lógico es cuando su programa tiene una buena sintaxis pero hay un error en el orden de las declaraciones o quizás un error en la forma en que las declaraciones se relacionan entre sí. Un buen ejemplo de un error lógico podría ser: "tome un trago de su botella de agua, colóquelo en su mochila, camine hasta la biblioteca y luego vuelva a colocar la tapa en la botella".

## Errores semánticos

Un error semántico es cuando tu descripción de los pasos a seguir es sintácticamente perfecta y en el orden correcto, pero simplemente hay un error en el programa. El programa es perfectamente correcto, pero no hace lo que tú **querías** que haga. Un ejemplo simple sería si le dieras a una persona instrucciones para llegar a un restaurante y dijeras: "... cuando llegue a la intersección con la estación de servicio, gire a la izquierda y avance una milla y el restaurante es un edificio rojo a su izquierda". Su amigo llega tarde y lo llama para decirle que está en una granja y que está caminando detrás de un granero, sin ninguna señal de restaurante. Entonces dices "¿giraste a la izquierda o a la derecha en la estación de servicio?" y dicen: "Seguí tus instrucciones a la

perfección, las tengo escritas, dice: gira a la izquierda y ve una milla en la gasolinera". Luego dices: "Lo siento mucho, porque aunque mis instrucciones fueron sintácticamente correctas, tristemente contenían un error semántico pequeño pero no detectado".

Nuevamente, en los tres tipos de errores, Python simplemente está haciendo todo lo posible por hacer exactamente lo que tú pediste.

# El viaje del aprendizaje

A medida que avance en el resto del libro, no tengas miedo si los conceptos no parecen encajar bien a la primera. Cuando estabas aprendiendo a hablar, no fue un problema durante los primeros años que hicieras ruidos de gorgoteo. Y estuvo bien si tardaste seis meses en pasar de un vocabulario simple a oraciones simples y tardaste 5-6 años más en pasar de las oraciones a los párrafos, y algunos años más para poder escribir un cuento completo completo interesante en tú sólo.

Queremos que aprendas Python mucho más rápido, por lo que enseñamos todo al mismo tiempo en los próximos capítulos. Pero es como aprender un nuevo idioma que toma tiempo absorber y entender antes de que se sienta natural. Esto nos lleva a cierta confusión a medida que visitamos y revisamos los temas para tratar de que veas el panorama general mientras definimos los pequeños fragmentos que conforman ese panorama general. Si bien el libro está escrito de manera lineal, y si estás tomando un curso, progresarás de manera lineal, no dudes en ser muy no lineal en la forma en que abordas el material. Mira hacia adelante y hacia atrás y lee con ligereza. Al hojear material más avanzado sin comprender completamente los detalles, puedes comprender mejor el "¿por qué?" de la programación. Al revisar el material anterior e incluso rehacer ejercicios anteriores, te darás cuenta de que realmente aprendiste mucho, incluso si el material que estás mirando actualmente parece un poco impenetrable.

Por lo general, cuando estás aprendiendo tu primer lenguaje de programación, hay algunos maravillosos momentos "¡Ah Hah!" en los que puedes mirar desde una roca con un martillo y un cincel, alejarte, y ver que de hecho estás construyendo una hermosa escultura.

Si algo parece particularmente difícil, generalmente merece la pena permanecer despierto toda la noche y mirarlo fijamente. Toma un descanso, toma una siesta, toma un refrigerio, explícale a alguien (o quizás a tu perro) con qué está teniendo problemas y luego vuelve con los ojos frescos. Te aseguro que una vez que aprendas los conceptos de programación en el libro, mirarás hacia atrás y verás que todo fue realmente fácil y elegante, y que simplemente te tomó un poco de tiempo absorberlo.

# Ejercicios (voluntarios y sin corrección)

**Ejercicio 1**: ¿Cuál es la función de la memoria secundaria en un ordenador?

a) Ejecutar todos los cálculos y la lógica del programa
b) Recuperar páginas web a través de Internet
c) Almacenar información a largo plazo, incluso más allá de un ciclo de alimentación
d) Tomar información del usuario

**Solución**

a

**Ejercicio 2**: ¿Cuál de los siguientes contiene "código de máquina"?

a) El intérprete de Python
b) El teclado
c) Archivo fuente de Python
d) Un documento de procesamiento de texto

**Solución**

a

**Ejercicio 3**: ¿Qué está mal en el siguiente código?

```
>>> primt 'Hello world!'
  File "<stdin>", line 1
    primt 'Hello world!'
                  ^
SyntaxError: invalid syntax
>>>
```

**Solución**

primt tendría que ser print

**Ejercicio 4**: ¿En qué parte de el ordenador hay una variable como "x" almacenada después de que termina la siguiente línea de Python?

```
x = 123
```

a) Unidad de procesamiento central
b) Memoria principal
c) Memoria secundaria
d) Dispositivos de entrada
e) Dispositivos de salida

**Solución**

c

**Ejercicio 5**: ¿Qué imprimirá el siguiente programa?

```
x = 43
x = x + 1
print(x)
```

a) 43
b) 44
c) x + 1
d) Error porque x = x + 1 no es posible matemáticamente

https://trinket.io/embed/python/47afa56dd668

**Ejercicio 6**: Explique cada uno de los siguientes ejemplos con un ejemplo de una capacidad humana: (1) Unidad central de procesamiento, (2) Memoria principal, (3) Memoria secundaria, (4) Dispositivo de entrada y (5) Dispositivo de salida. Por ejemplo, "¿Cuál es el equivalente humano a una unidad central de procesamiento?"

**Ejercicio 7** ¿Cómo arreglas un "Error de sintaxis"

Copyright 2026

# 2 Variables

## Valores y tipos

Un **valor** es una de las cosas básicas con las que trabaja un programa, como una letra o un número. Los valores que hemos visto hasta ahora son `1`, `2` y "¡Hola, mundo!"

Estos valores pertenecen a diferentes tipos: `2` es un número entero (`int` de integer en inglés) y "¡Hola, mundo!" es una **cadena** (`string`), llamada así porque contiene una "cadena" de letras. Usted (y el intérprete) pueden identificar cadenas porque están entre comillas.

La declaración `print` también funciona para enteros. Usamos el comando `python` para iniciar el intérprete.

```
python
>>> print(4)
4
```

Si no estás seguro del tipo de valor que tiene, el intérprete te puede informar.

```
>>> type('Hello, World!')
<class 'str'>
>>> type(17)
<class 'int'>
```

Las cadenas pertenecen al tipo `str` y los enteros pertenecen al tipo `int`. Los números con un punto decimal pertenecen a un tipo llamado `float`, porque estos números están representados en un formato llamado **punto flotante**.

```
>>> type(3.2)
<class 'float'>
```

¿Qué pasa con los valores como "17" y "3.2"? Parecen números, pero están entre comillas como cadenas.

Copyright 2026

```
>>> type('17')
<class 'str'>
>>> type('3.2')
<class 'str'>
```

Son cadenas.

Cuando escribe un entero grande, puede verse tentado a usar comas entre grupos de tres dígitos, como en `1,000,000`. Este no es un entero legal en Python, pero es legal:

```
>>> print(1,000,000)
1 0 0
```

Bueno, ¡eso no es lo que esperábamos en absoluto! Python interpreta `1,000,000` como una secuencia de enteros separados por comas, que imprime con espacios entre ellos.

Este es el primer ejemplo que hemos visto de un error semántico: el código se ejecuta sin producir un mensaje de error, pero no hace lo "correcto".

# Variables

Una de las características más poderosas de un lenguaje de programación es la capacidad de manipular **variables**. Una variable es un nombre que se refiere a un valor.

Una **asignación** crea nuevas variables y les da valores:

```
>>> message = 'And now for something completely different'
>>> n = 17
>>> pi = 3.1415926535897931
```

Este ejemplo hace tres asignaciones. El primero asigna una cadena a una nueva variable llamada `message`; el segundo asigna el número entero `17` a `n`; el tercero asigna el valor (aproximado) de **π** a `pi`.

Para mostrar el valor de una variable, puedes usar una declaración de impresión:

```
>>> print(n)
17
```

```
>>> print(pi)
3.141592653589793
```

El tipo de una variable es el tipo del valor al que se refiere.

```
>>> type(message)
<class 'str'>
>>> type(n)
<class 'int'>
>>> type(pi)
<class 'float'>
```

# Nombres de variables y palabras clave

Los programadores generalmente eligen nombres para sus variables que son significativos y documentan para qué se utiliza la variable.

Los nombres de las variables pueden ser arbitrariamente largos. Pueden contener letras y números, pero no pueden comenzar con un número. Es legal usar letras mayúsculas, pero es una buena idea comenzar los nombres de las variables con una letra minúscula (verás porqué más adelante).

El carácter de subrayado (`_`) puede aparecer en un nombre. A menudo se usa en nombres con varias palabras, como `my_name` o `airspeed_of_unladen_swallow`. Los nombres de las variables pueden comenzar con un carácter de subrayado, pero generalmente evitamos hacer esto a menos que estemos escribiendo el código de la biblioteca para que otros lo utilicen.

Si le das a una variable un nombre ilegal, obtendrás un error de sintaxis:

```
>>> 76trombones = 'big parade'
SyntaxError: invalid syntax
>>> more@ = 1000000
SyntaxError: invalid syntax
>>> class = 'Advanced Theoretical Zymurgy'
SyntaxError: invalid syntax
```

`76trombones` es ilegal porque comienza con un número. `more@` es ilegal porque contiene un carácter ilegal, `@`. Pero, ¿qué pasa con `class`?

Resulta que `class` es una de las **palabras reservadas** de Python. El intérprete usa palabras clave para reconocer la estructura del programa, y no se pueden usar como nombres de variables.

Python reserva 33 palabras clave:

```
and       del       from      None      True
as        elif      global    nonlocal  try
assert    else      if        not       while
break     except    import    or        with
class     False     in        pass      yield
continue  finally   is        raise
def       for       lambda    return
```

Es posible que desees mantener esta lista a mano. Si el intérprete se queja de uno de sus nombres de variables y no sabe porqué, vea si está en esta lista.

# Declaraciones

Una **instrucción** es una unidad de código que el intérprete de Python puede ejecutar. Hemos visto dos tipos de declaraciones: declaraciones de impresión (print()) y una asignación.

Cuando escribe una declaración en modo interactivo, el intérprete lo ejecuta y muestra el resultado, si lo hay.

Un script usualmente contiene una secuencia de sentencias. Si hay más de una declaración, los resultados aparecen uno por uno a medida que se ejecutan las declaraciones.

Por ejemplo, el guión.

```
print(1)
x = 2
print(x)
```

produce la salida

```
1
2
```

Copyright 2026

La sentencia de asignación no produce salida.

# Operadores y operandos

**Los operadores** son símbolos especiales que representan cálculos como la suma y la multiplicación. Los valores a los que se aplica el operador se denominan **operandos**.

Los operadores `+`, `-`, `*`, `/` y `**` realizan suma, resta, multiplicación, división y exponenciación, como en los siguientes ejemplos:

```
20+32   hour-1   hour*60+minute   minute/60   5**2   (5+9)*(15-7)
```

Ha habido un cambio en el operador de división entre Python 2.x y Python 3.x. En Python 3.x, el resultado de esta división es un resultado de punto flotante:

```
>>> minute = 59
>>> minute/60
0.9833333333333333
```

El operador de división en Python 2.0 dividiría dos enteros y truncaría el resultado a un entero:

```
>>> minute = 59
>>> minute/60
0
```

Para obtener la misma respuesta en Python 3.0 use división dividida (// integer).

En Python 3.0, la división de enteros funciona mucho más como cabría esperar si ingresara la expresión en una calculadora.

```
>>> minute = 59
>>> minute//60
0
```

# Expresiones

Copyright 2026

Una **expresión** es una combinación de valores, variables y operadores. Un valor en sí mismo se considera una expresión, y también lo es una variable, por lo que las siguientes son todas expresiones legales (asumiendo que a la variable `x` se le ha asignado un valor):

```
17
x
x + 17
```

Si escribes una expresión en modo interactivo, el intérprete **la evalúa** y muestra el resultado:

```
>>> 1 + 1
2
```

¡Pero en un guión, una expresión por sí misma no hace nada! Esta es una fuente común de confusión para los principiantes.

**Ejercicio 1**: escriba las siguientes declaraciones en el intérprete de Python para ver qué hacen:

```
5
x = 5
x + 1
```

# Orden de operaciones

Cuando aparece más de un operador en una expresión, el orden de evaluación depende de las **reglas de precedencia**. Para los operadores matemáticos, Python sigue la convención matemática. El acrónimo **PEMDAS** es una forma útil de recordar las reglas:

- **P**arentheses tienen la mayor prioridad y se pueden usar para forzar a una expresión a evaluar en el orden que desee. Como las expresiones entre paréntesis se evalúan primero, `2 * (3-1)` es 4, y `(1 + 1) ** (5-2)` es 8. También puede usar paréntesis para hacer que una expresión sea más fácil de leer , como en `(minuto * 100)/60`, incluso si no cambia el resultado.
- **E**xponentiation tiene la siguiente prioridad más alta, por lo que `2**1+1` es 3, no 4, y `3*1**3` es 3, no 27.
- **M**ultiplication y **D**ivision tienen la misma precedencia, que es mayor que **A**ddition y **S**ubtraction, que también tienen la misma precedencia. Entonces `2*3-1` es 5, no 4, y `6+4/2` es 8.0, no 5.

- Los operadores con la misma precedencia son evaluados de izquierda a derecha. Entonces, la expresión `5-3-1` es 1, no 3, porque el `5-3` sucede primero y luego se le resta `1` de `2`.

En caso de duda, pon siempre paréntesis en tus expresiones para asegurarte de que los cálculos se realicen en el orden que deseas.

# Operador de módulo

El **operador de módulo** funciona en enteros y produce el resto cuando el primer operando se divide por el segundo. En Python, el operador de módulo es un signo de porcentaje (`%`). La sintaxis es la misma que para otros operadores:

```
>>> quotient = 7 // 3
>>> print(quotient)
2
>>> remainder = 7 % 3
>>> print(remainder)
1
```

Así que 7 dividido por 3 es 2 con 1 sobrante.

El operador de módulo resulta ser sorprendentemente útil. Por ejemplo, puede verificar si un número es divisible por otro: si `x%y` es cero, entonces `x` es divisible por `y`.

También puede extraer el dígito o dígitos más a la derecha de un número. Por ejemplo, `x%10` produce el dígito más a la derecha de `x` (en base 10). De manera similar, `x%100` produce los dos últimos dígitos.

# Operaciones con cadenas

El operador `+` trabaja con cadenas, pero no es una suma en el sentido matemático. En su lugar, realiza **concatenación**, lo que significa unir las cadenas uniéndolas de extremo a extremo. Por ejemplo:

```
>>> first = 10
>>> second = 15
```

```
>>> print(first+second)
25
>>> first = '100'
>>> second = '150'
>>> print(first + second)
100150
```

La salida de este programa es `100150`.

# Preguntando al usuario por la entrada

A veces nos gustaría tomar el valor para una variable del usuario a través de su teclado. Python proporciona una función llamada `input` que obtiene información desde el teclado[1]. Cuando se llama a esta función, el programa se detiene y espera a que el usuario escriba algo. Cuando el usuario presiona `Return` o `Enter`, el programa se reanuda y `input` devuelve lo que el usuario escribió como una cadena.

```
>>> input = input()
Some silly stuff
>>> print(input)
Some silly stuff
```

Antes de recibir información del usuario, es una buena idea imprimir un mensaje que indique al usuario qué debe ingresar. Puede pasar una cadena a `input` para que se muestre al usuario antes de hacer una pausa para ingresar:

```
>>> name = input('What is your name?\n')
What is your name?
Chuck
>>> print(name)
Chuck
```

La secuencia `\n` al final de la solicitud representa una **nueva línea**, que es un carácter especial que causa un salto de línea. Es por eso que la entrada del usuario aparece debajo del indicador.

Si esperas que el usuario escriba un número entero, puedes intentar convertir el valor de retorno a `int` utilizando la función `int()`:

```
>>> prompt = 'What...is the airspeed velocity of an unladen swallow?\n'
>>> speed = input(prompt)
What...is the airspeed velocity of an unladen swallow?
17
>>> int(speed)
17
>>> int(speed) + 5
22
```

Pero si el usuario escribe algo más que una cadena de dígitos, obtendrás un error:

```
>>> speed = input(prompt)
What...is the airspeed velocity of an unladen swallow?
What do you mean, an African or a European swallow?
>>> int(speed)
ValueError: invalid literal for int() with base 10:
```

Veremos cómo manejar este tipo de error más adelante.

# Comentarios

A medida que los programas se hacen más grandes y más complicados, se vuelven más difíciles de leer. Los lenguajes formales son densos, y a menudo es difícil mirar un fragmento de código y descubrir qué está haciendo, o por qué.

Por esta razón, es una buena idea agregar notas a tus programas para explicar en lenguaje natural lo que está haciendo el programa. Estas notas se llaman **comentarios**, y en Python comienzan con el símbolo `#`:

```
# compute the percentage of the hour that has elapsed
percentage = (minute * 100) / 60
```

En este caso, el comentario aparece solo en una línea. También puedes poner comentarios al final de una línea:

```
percentage = (minute * 100) / 60     # percentage of an hour
```

Todo desde el `\#` hasta el final de la línea se ignora. No tiene efecto en el programa.

Los comentarios son más útiles cuando documentan características no obvias del código. Es razonable suponer que el lector puede averiguar lo que hace el código; es mucho más útil explicar por qué.

Este comentario es redundante e inútil:

```
v = 5     # assign 5 to v
```

Este comentario contiene información útil que no está en el código:

```
v = 5     # velocity in meters/second.
```

Los buenos nombres de variables pueden reducir la necesidad de comentarios, pero los nombres largos pueden hacer que las expresiones complejas sean difíciles de leer, por lo que hay que buscar un punto de equilibrio.

# Eligiendo nombres de variables mnemónicas

Siempre que sigas las reglas simples de los nombres de variables y evites las palabras reservadas, tienes muchas opciones al nombrar tus variables. Al principio, esta opción puede ser confusa tanto cuando lees un programa como cuando escribes tus propios programas. Por ejemplo, los siguientes tres programas son idénticos en términos de lo que logran, pero son muy diferentes cuando los lees y tratas de entenderlos.

```
a = 35.0
b = 12.50
c = a * b
print(c)
```

```
hours = 35.0
rate = 12.50
pay = hours * rate
print(pay)
```

```
x1q3z9ahd = 35.0
x1q3z9afd = 12.50
x1q3p9afd = x1q3z9ahd * x1q3z9afd
print(x1q3p9afd)
```

El intérprete de Python ve a estos tres programas como **exactamente iguales** pero los humanos ven y entienden estos programas de manera muy diferente. Los humanos entenderán más rápidamente la **intención** del segundo programa porque el programador ha elegido nombres de variables que reflejan su intención con respecto a qué datos se almacenarán en cada variable.

Llamamos a estos nombres de variables sabiamente elegidos "nombres de variables mnemónicas". La palabra **mnemotécnia**[2] significa "ayuda de memoria". Elegimos nombres de variables mnemónicas para ayudarnos a recordar porqué creamos la variable en primer lugar.

Si bien todo esto suena bien, y es una muy buena idea usar nombres de variables mnemónicas, los nombres de variables mnemotécnicas pueden interferir en la capacidad de un programador principiante para analizar y comprender el código. Esto se debe a que los programadores principiantes aún no han memorizado las palabras reservadas (solo hay 33 de ellas) y, a veces, las variables con nombres que son demasiado descriptivas comienzan a parecer parte del lenguaje y no solo los nombres de variables bien elegidos.

Eche un vistazo rápido al siguiente código de ejemplo de Python que recorre algunos datos. Cubriremos los bucles pronto, pero por ahora trataremos de descifrar lo que esto significa:

```
for word in words:
    print(word)
```

¿Que está sucediendo aquí? ¿Cuáles de los tokens (para, palabra, en, etc.) son palabras reservadas y cuáles son solo nombres de variables? ¿Python entiende en un nivel fundamental la noción de palabras? Los programadores principiantes tienen problemas para separar qué partes del código **deben** ser las mismas que en este ejemplo y qué partes del código son simplemente elecciones hechas por el programador.

El siguiente código es equivalente al código anterior:

```
for slice in pizza:
    print(slice)
```

Es más fácil para el programador principiante mirar este código y saber qué partes son palabras reservadas definidas por Python y qué partes son simplemente nombres de variables elegidos por

el programador. Es bastante claro que Python no tiene una comprensión fundamental de la pizza y las rebanadas y el hecho de que una pizza consiste en un conjunto de una o más rebanadas.

Pero si nuestro programa realmente trata de leer datos y buscar palabras en los datos, `pizza` y `slice` son nombres de variables muy poco mnemotécnicos. Elegirlos como nombres de variables distrae del significado del programa.

Después de un período de tiempo bastante corto, conocerá las palabras reservadas más comunes y comenzará a ver las palabras reservadas saltando hacia usted:

**for** word **in** words: **print**(word)

Las partes del código definidas por Python (`for`, `in`, `print` y `:`) están en negrita y las variables elegidas por el programador (`word` y `words`) no están en negrita . Muchos editores de texto conocen la sintaxis de Python y colorearán las palabras reservadas de manera diferente para darle pistas sobre cómo mantener separadas sus variables y palabras reservadas. Después de un tiempo, comenzarás a leer Python y determinar rápidamente qué es una variable y qué es una palabra reservada.

# Depuración

En este punto, el error de sintaxis que probablemente cometas es un nombre de variable ilegal, como `class` y `yield`, que son palabras clave, o `odd~job` y `US$`, que contienen caracteres ilegales.

Si colocas un espacio en el nombre de una variable, Python cree que son dos operandos sin un operador:

```
>>> bad name = 5
SyntaxError: invalid syntax
```

```
>>> month = 09
  File "<stdin>", line 1
    month = 09
              ^
SyntaxError: invalid token
```

Para los errores de sintaxis, los mensajes de error no ayudan mucho. Los mensajes más comunes son `SyntaxError: syntax invalid` y `SyntaxError: invalid token`, ninguno de los cuales es muy informativo.

El error de tiempo de ejecución que es más probable que cometa es un "use before def;" es decir, tratar de usar una variable antes de que haya asignado un valor. Esto puede suceder si escribes mal el nombre de una variable:

```
>>> principal = 327.68
>>> interest = principle * rate
NameError: name 'principle' is not defined
```

Los nombres de las variables distinguen entre mayúsculas y minúsculas, por lo que `LaTeX` no es lo mismo que `latex`.

En este punto, la causa más probable de un error semántico es el orden de las operaciones. Por ejemplo, para evaluar `1/2π`, puede tener la tentación de escribir

```
>>> 1.0/2.0 * pi
```

Pero la división ocurre primero, así que obtendrías `π/2`, ¡que no es lo mismo! Python no tiene forma de saber lo que querías escribir, por lo que en este caso no recibes un mensaje de error; acabas de obtener la respuesta incorrecta.

# Ejercicios

**Ejercicio 2**: escribe un programa que use `input` para pedirle a un usuario su nombre y luego le de la bienvenida.

```
Enter your name: Chuck
Hello Chuck
```

**Ejercicio 3**: escribe un programa para pedirle al usuario las horas y la tarifa por hora para calcular el pago bruto.

```
Enter Hours: 35
Enter Rate: 2.75
Pay: 96.25
```

No nos preocuparemos por asegurarnos de que nuestra paga tenga exactamente dos dígitos después del lugar decimal por ahora. Si lo deseas, puede jugar con la función Python `round` incorporada para redondear correctamente la paga resultante a dos decimales.

Copyright 2026

**Ejercicio 4**: Supongamos que ejecutamos las siguientes instrucciones de asignación:

```
width = 17
height = 12.0
```

Para cada una de las siguientes expresiones, escriba el valor de la expresión y el tipo (del valor de la expresión).

1. `width//2`
2. `width/ 2.0`
3. `height/ 3`
4. `1+2\*5`

Use el intérprete de Python para verificar sus respuestas.

**Ejercicio 5**: escribe un programa que solicite al usuario una temperatura en grados Celsius, convierta la temperatura a Fahrenheit e imprima la temperatura convertida.

---

> [1]. En Python 2.0, esta función se denominó `raw_input`. ↩

> [2]. Consulte http://en.wikipedia.org/wiki/Mnemonic para obtener una descripción ampliada de la palabra "mnemonic". ↩

# 3 Condicionales

## Expresiones booleanas

Una **expresión booleana** es una expresión que es verdadera o falsa. Los siguientes ejemplos usan el operador `==`, que compara dos operandos y produce `True` si son iguales y `False` de lo contrario:

```
>>> 5 == 5
True
>>> 5 == 6
False
```

`True` y `False` son valores especiales que pertenecen a la clase `bool`; no son cadenas.

```
>>> type(True)
<class 'bool'>
>>> type(False)
<class 'bool'>
```

El operador `==` es uno de los **operadores de comparación**; los otros son:

```
x != y               # x is not equal to y
x > y                # x is greater than y
x < y                # x is less than y
x >= y               # x is greater than or equal to y
x <= y               # x is less than or equal to y
x is y               # x is the same as y
x is not y           # x is not the same as y
```

Aunque es probable que estas operaciones te resulten familiares, los símbolos de Python son diferentes de los símbolos matemáticos para las mismas operaciones. Un error común es usar un único signo igual (`=`) en lugar de un doble signo igual (`==`). Recuerda que `=` es un operador de asignación y `==` es un operador de comparación. No hay tal cosa como `=<` o `=>`.

Copyright 2026

# Operadores lógicos

Hay tres **operadores lógicos**: `and`, `or`, y `not`. La semántica (significado) de estos operadores es similar a su significado en inglés. Por ejemplo,

```
x > 0 and x < 10
```

es cierto solo si `x` es mayor que 0 **y** menor que 10.

`n%2 == 0 or n% 3 == 0` es verdadero si **cualquiera** de las dos condiciones es verdadera, es decir, si el número es divisible por 2 **o** 3.

Finalmente, el operador `not` niega una expresión booleana, por lo que `not(x>y)` es verdadero si `x>y` es falso; es decir, si `x` es menor o igual que `y`.

Estrictamente hablando, los operandos de los operadores lógicos deben ser expresiones booleanas, pero Python no es muy estricto. Cualquier número distinto de cero se interpreta como "verdadero".

```
>>> 17 and True
True
```

Esta flexibilidad puede ser útil, pero hay algunas sutilezas que pueden ser confusas. Es posible que desee evitarlo hasta que esté seguro de saber lo que está haciendo.

# Ejecución condicional

Para escribir programas útiles, casi siempre necesitamos la capacidad de verificar las condiciones y cambiar el comportamiento del programa en consecuencia. Las **declaraciones condicionales** nos dan esta habilidad. La forma más simple es la instrucción `if`:

```
if x > 0 :
    print('x is positive')
```

La expresión booleana después de la instrucción `if` se llama la **condición**. Terminamos la instrucción `if` con un carácter de dos puntos (:) y la(s) línea(s) después de la instrucción if deben estar sangradas.

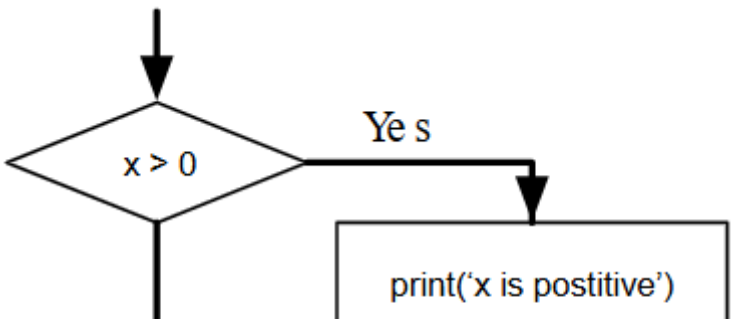


Imagen - Si logica

Si la condición lógica es verdadera, entonces se ejecuta la instrucción con sangría. Si la condición lógica es falsa, la instrucción con sangría se omite.

Las declaraciones `if` tienen la misma estructura que las definiciones de funciones o `for` loops [1]. La declaración consiste en una línea de encabezado que termina con el carácter de dos puntos (:) seguido de un bloque con sangría. Las declaraciones como esta se denominan **declaraciones compuestas** porque se extienden a lo largo de más de una línea.

No hay límite en el número de declaraciones que pueden aparecer en el cuerpo, pero debe haber al menos una. Ocasionalmente, es útil tener un cuerpo sin declaraciones (generalmente como un marcador de posición para el código que aún no ha escrito). En ese caso, puedes usar la instrucción `pass`, que no hace nada.

```
if x < 0 :
    pass          # need to handle negative values!
```

Si ingresa una instrucción `if` en el intérprete de Python, el indicador cambiará de tres puntos a tres puntos para indicar que se encuentra en medio de un bloque de declaraciones, como se muestra a continuación:

```
>>> x = 3
>>> if x < 10:
...     print('Small')
...
Small
>>>
```

# Ejecución alternativa

Una segunda forma de la sentencia `if` es **ejecución alternativa**, en la que hay dos posibilidades y la condición determina cuál se ejecuta. La sintaxis se ve así:

```
if x%2 == 0 :
    print('x is even')
else :
    print('x is odd')
```

Si el resto cuando `x` se divide por 2 es 0, entonces sabemos que `x` es par, y el programa muestra un mensaje a tal efecto. Si la condición es falsa, se ejecuta el segundo conjunto de sentencias.

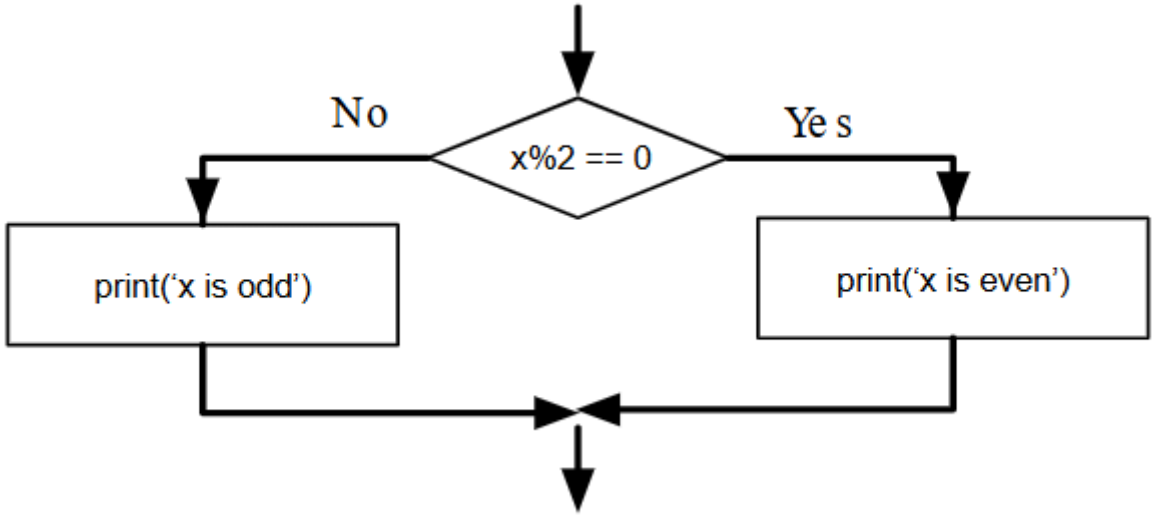



Imagen - Lógica de If-Then-Else

Dado que la condición debe ser verdadera o falsa, se ejecutará exactamente una de las alternativas. Las alternativas se denominan **ramas**, porque son ramas en el flujo de ejecución.

# Condicionales encadenados

A veces hay más de dos posibilidades y necesitamos más de dos ramas. Una forma de expresar un cálculo como ese es un **condicional encadenado**:

```
if x < y:
    print('x is less than y')
elif x > y:
    print('x is greater than y')
else:
    print('x and y are equal')
```

"elif" es una abreviatura de "else if". De nuevo, exactamente una rama será ejecutada.

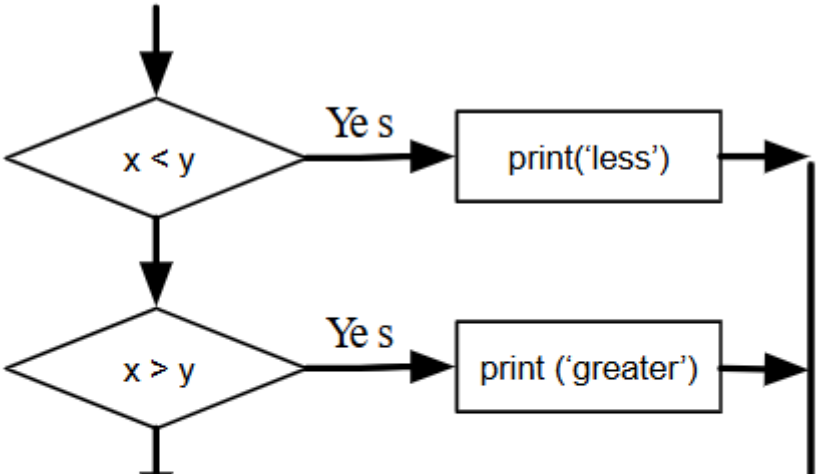


Imagen - Lógica de elif

No hay límite en el número de declaraciones `elif`. Si hay una cláusula `else`, tiene que estar al final, pero no tiene necesariamente que haber una.

```
if choice == 'a':
    print('Bad guess')
elif choice == 'b':
    print('Good guess')
elif choice == 'c':
    print('Close, but not correct')
```

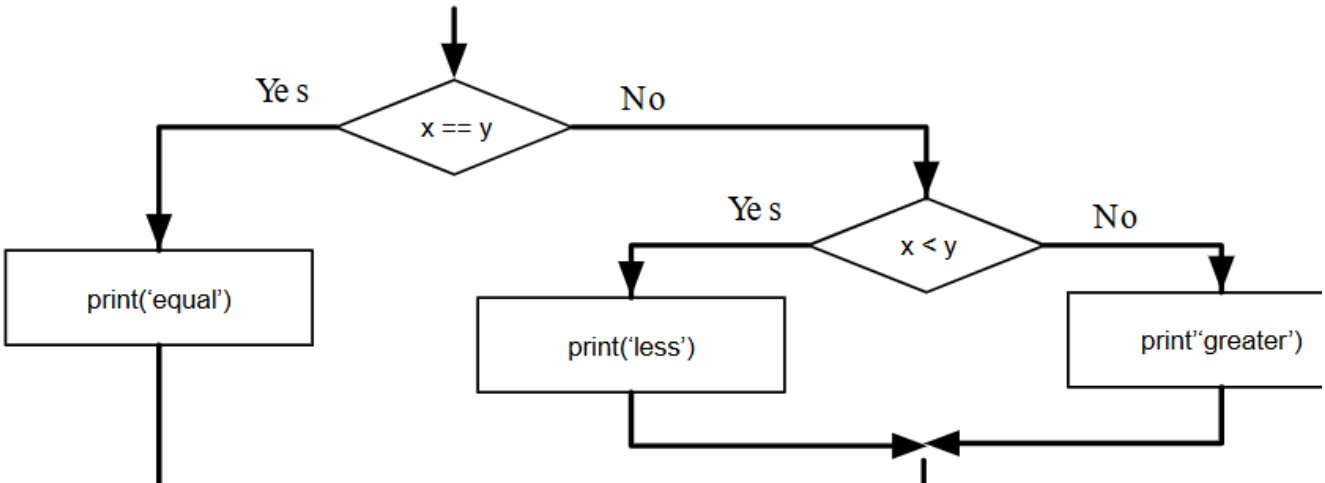
Cada condición se comprueba en orden. Si el primero es falso, se marca el siguiente, y así sucesivamente. Si uno de ellos es verdadero, la rama correspondiente se ejecuta y la declaración finaliza. Incluso si más de una condición es verdadera, solo se ejecuta la primera rama verdadera.

# Condicionales anidados

Un condicional también puede ser anidado dentro de otro. Podríamos haber escrito el ejemplo de tres ramas como este:

```
if x == y:
    print('x and y are equal')
else:
    if x < y:
        print('x is less than y')
    else:
        print('x is greater than y')
```

El condicional exterior contiene dos ramas. La primera rama contiene una declaración simple. La segunda rama contiene otra instrucción 'if', que tiene dos ramas propias. Esas dos ramas son ambas declaraciones simples, aunque también podrían haber sido declaraciones condicionales.




Copyr

Imagen - Nested If Statements

Aunque la sangría de los enunciados hace que la estructura sea aparente, **los condicionales anidados** se vuelven difíciles de leer muy rápidamente. En general, es una buena idea evitarlos cuando puedas.

Los operadores lógicos a menudo proporcionan una forma de simplificar sentencias condicionales anidadas. Por ejemplo, podemos reescribir el siguiente código usando un solo condicional:

```
if 0 < x:
    if x < 10:
        print('x is a positive single-digit number.')
```

La instrucción `print` se ejecuta solo si superamos ambas condiciones, por lo que podemos obtener el mismo efecto con el operador `y`:

```
if 0 < x and x < 10:
    print('x is a positive single-digit number.')
```

# Capturando excepciones utilizando try y except

Anteriormente vimos un segmento de código donde usamos las funciones `input` y `int` para leer y analizar un número entero ingresado por el usuario. También vimos lo peligroso que podría ser hacer esto:

```
>>> prompt = "What...is the airspeed velocity of an unladen swallow?\n"
>>> speed = input(prompt)
What...is the airspeed velocity of an unladen swallow?
What do you mean, an African or a European swallow?
>>> int(speed)
ValueError: invalid literal for int() with base 10:
>>>
```

Cuando ejecutamos estas declaraciones en el intérprete de Python, obtenemos un nuevo mensaje del intérprete, pensamos "oops" y pasamos a nuestra siguiente declaración.

Sin embargo, si coloca este código en una secuencia de comandos de Python y se produce este error, la secuencia de comandos se detiene inmediatamente dejando una traza de lo que se estaba ejecutando cuando el programa falló. No ejecuta la siguiente sentencia.

Aquí hay un programa de ejemplo para convertir una temperatura Fahrenheit a una temperatura Celsius:

https://trinket.io/embed/python3/27bbe8b7ab

Si ejecutamos este código y le damos una entrada no válida, simplemente falla con un mensaje de error hostil:

```
python fahren.py
Enter Fahrenheit Temperature:72
22.22222222222222
```

```
python fahren.py
Enter Fahrenheit Temperature:fred
Traceback (most recent call last):
  File "fahren.py", line 2, in <module>
    fahr = float(inp)
ValueError: could not convert string to float: 'fred'
```

Existe una estructura de ejecución condicional incorporada en Python para manejar estos tipos de errores esperados e inesperados llamados "try/except". La idea de `try` y `except` es que usted sabe que alguna secuencia de instrucciones puede tener un problema y que desea agregar algunas instrucciones para que se ejecuten si se produce un error. Estas declaraciones adicionales (el bloque de excepción) se ignoran si no hay ningún error.

Puede pensar en las funciones `try` y `except` en Python como una “póliza de seguro” en una secuencia de declaraciones.

Podemos reescribir nuestro convertidor de temperatura de la siguiente manera:

https://trinket.io/embed/python3/5dbec1550b

Python comienza ejecutando la secuencia de instrucciones en el bloque `try`. Si todo va bien, salta el bloque `except` y continúa. Si ocurre una excepción en el bloque `try`, Python salta del bloque `try` y ejecuta la secuencia de instrucciones en el bloque `except`.

```
python fahren2.py
Enter Fahrenheit Temperature:72
22.22222222222222
```

```
python fahren2.py
Enter Fahrenheit Temperature:fred
Please enter a number
```

Manejar una excepción con una declaración `try` se llama **capturar** una excepción. En este ejemplo, la cláusula `except` imprime un mensaje de error. En general, detectar una excepción le da la oportunidad de solucionar el problema, o volver a intentarlo, o al menos finalizar el programa con gracia.

# Evaluación de cortocircuito de expresiones lógicas

Cuando Python está procesando una expresión lógica como `x>= 2 and (x/y)>2`, evalúa la expresión de izquierda a derecha. Debido a la definición de `y`, si `x` es menor que 2, la expresión `x>= 2` es `False` y, por lo tanto, toda la expresión es `False` independientemente de si `(x/y)>` se evalúa como "True" o "False".

Cuando Python detecta que no hay nada que ganar al evaluar el resto de una expresión lógica, detiene su evaluación y no realiza los cálculos en el resto de la expresión lógica. Cuando la evaluación de una expresión lógica se detiene porque ya se conoce el valor general, se llama **cortocircuitar** la evaluación.

Si bien esto puede parecer un punto delicado, el comportamiento de cortocircuito conduce a una técnica inteligente llamada **patrón guardián**. Considere la siguiente secuencia de código en el intérprete de Python:

```
>>> x = 6
>>> y = 2
>>> x >= 2 and (x/y) > 2
```

```
True
>>> x = 1
>>> y = 0
>>> x >= 2 and (x/y) > 2
False
>>> x = 6
>>> y = 0
>>> x >= 2 and (x/y) > 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>>
```

El tercer cálculo falló porque Python estaba evaluando `(x/y)` y `y` era cero, lo que causa un error de tiempo de ejecución. Pero el segundo ejemplo no falló porque la primera parte de la expresión `x>=2` se evaluó como `Falso` por lo que el `(x/y)` nunca se ejecutó debido a que **se cortocircuitó** la evaluación y no hubo error.

Podemos construir la expresión lógica para colocar estratégicamente una evaluación **de guardia** justo antes de la evaluación que podría causar un error de la siguiente manera:

```
>>> x = 1
>>> y = 0
>>> x >= 2 and y != 0 and (x/y) > 2
False
>>> x = 6
>>> y = 0
>>> x >= 2 and y != 0 and (x/y) > 2
False
>>> x >= 2 and (x/y) > 2 and y != 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>>
```

En la primera expresión lógica, `x>=2` es `False`, por lo que la evaluación se detiene en `y`. En la segunda expresión lógica, `x >= 2` es `True` pero `y!=0` es `False` por lo que nunca alcanzamos `(x/y)`.

Copyright 2026

En la tercera expresión lógica, el `y!=0` es **después del** cálculo `(x/y)` por lo que la expresión falla con un error.

En la segunda expresión, decimos que `y!=0` actúa como un **guard** para asegurarnos de que solo ejecutamos `(x/y)` si `y` no es cero.

# Depuración

El seguimiento que Python muestra cuando se produce un error contiene mucha información, pero puede ser abrumador. Las partes más útiles suelen ser:

- Qué tipo de error fue, y
- Donde ocurrió.

Los errores de sintaxis son generalmente fáciles de encontrar, pero hay algunos enrevesados. Los errores de espacios en blanco pueden ser complicados porque los espacios y las pestañas son invisibles y estamos acostumbrados a ignorarlos.

```
>>> x = 5
>>>  y = 6
  File "<stdin>", line 1
    y = 6
    ^
IndentationError: unexpected indent
```

En este ejemplo, el problema es que la segunda línea está sangrada por un espacio. Pero el mensaje de error apunta a `y`, lo que es engañoso. En general, los mensajes de error indican dónde se descubrió el problema, pero el error real puede ser anterior en el código, a veces en una línea anterior.

En general, los mensajes de error te indican dónde se descubrió el problema, pero a menudo no es donde se causó.

# Ejercicios (voluntario sin corrección)

**Ejercicio 1** reescribe tu cálculo de pago para darle al empleado 1.5 veces la tarifa por hora por las horas trabajadas por encima de las 40 horas.

```
Enter Hours: 45
Enter Rate: 10
Pay: 475.0
```

**Ejercicio 2** reescribe tu programa de pago usando `try` y `except` para que su programa maneje la entrada no numérica correctamente imprimiendo un mensaje y saliendo del programa. A continuación se muestran dos ejecuciones del programa:

```
Enter Hours: 20
Enter Rate: nine
Error, please enter numeric input
```

```
Enter Hours: forty
Error, please enter numeric input
```

**Ejercicio 3** Escribe un programa para solicitar una puntuación entre 0.0 y 1.0. Si la puntuación está fuera de rango, imprime un mensaje de error. Si la puntuación está entre 0.0 y 1.0, imprime una calificación usando la siguiente tabla:

```
Score   Grade
>= 0.9     A
>= 0.8     B
>= 0.7     C
>= 0.6     D
< 0.6    F
~~~
```

Introduce la puntuación: 0,95 A ~ ~

```
Enter score: perfect
Bad score
```

```
Enter score: 10.0
Bad score
```

```
Enter score: 0.75
C
```

```
Enter score: 0.5
F
```

Ejecute el programa repetidamente como se muestra arriba para probar los diferentes valores de entrada.

---

> [1]. Aprenderemos sobre las funciones en el Capítulo 4 y los bucles en el Capítulo 5. ↩

# 4 Funciones

## Llamadas de función

En el contexto de la programación, una **función** es una secuencia de instrucciones con nombre que realiza un cálculo. Cuando defines una función, especifica el nombre y la secuencia de instrucciones. Más tarde, puedes "llamar" a la función por su nombre. Ya hemos visto un ejemplo de una llamada a una función:

```
>>> type(32)
<class 'int'>
```

El nombre de la función es `type`. La expresión entre paréntesis se llama el **argumento** de la función. El argumento es un valor o variable que estamos pasando a la función como entrada. El resultado, para la función `type`, es el tipo del argumento.

Es común decir que una función "toma" un argumento y "devuelve" un resultado. El resultado se llama el **valor de retorno**.

## Funciones incorporadas (built-in functions)

Python proporciona una serie de funciones integradas importantes que podemos usar sin necesidad de proporcionar la definición de la función. Los creadores de Python escribieron un conjunto de funciones para resolver problemas comunes y las incluyeron en Python para que las utilicemos.

Las funciones `max` y `min` nos dan los valores más grandes y más pequeños en una lista, respectivamente:

```
>>> max('Hello world')
'w'
>>> min('Hello world')
```

Copyright 2026

```
' '
>>>
```

La función `max` nos dice el "carácter más grande" en la cadena (que resulta ser la letra "w") y la función `min` nos muestra el carácter más pequeño (que es un espacio).

Otra función incorporada muy común es la función `len` que nos dice cuántos elementos hay en su argumento. Si el argumento a `len` es una cadena, devuelve el número de caracteres en la cadena.

```
>>> len('Hello world')
11
>>>
```

Estas funciones no se limitan a mirar las cadenas. Pueden operar en cualquier conjunto de valores, como veremos en capítulos posteriores.

Debe tratar los nombres de las funciones incorporadas como palabras reservadas (es decir, evitar usar "max" como nombre de variable).

# Funciones de conversión de tipos

Python también proporciona funciones integradas que convierten valores de un tipo a otro. La función `int` toma cualquier valor y lo convierte en un entero, si puede, o se queja de lo contrario:

```
>>> int('32')
32
>>> int('Hello')
ValueError: invalid literal for int() with base 10: 'Hello'
```

`int` puede convertir valores de coma flotante en enteros, pero no se redondea; corta la parte de la fracción

```
>>> int(3.99999)
3
>>> int(-2.3)
-2
```

`float` convierte números enteros y cadenas en números de punto flotante:

```
>>> float(32)
32.0
>>> float('3.14159')
3.14159
```

Finalmente, `str` convierte su argumento en una cadena:

```
>>> str(32)
'32'
>>> str(3.14159)
'3.14159'
```

# Números aleatorios

Dadas las mismas entradas, la mayoría de los programas de ordenador generan las mismas salidas cada vez, por lo que se dice que son **deterministas**. El determinismo suele ser algo bueno, ya que esperamos que el mismo cálculo dé el mismo resultado. Para algunas aplicaciones, sin embargo, queremos que el ordenador sea impredecible. Los juegos son un ejemplo obvio, pero hay más.

Hacer que un programa sea realmente no determinista resulta no ser tan fácil, pero hay formas de que al menos parezca no determinista. Uno de ellos es utilizar **algoritmos** que generan **números pseudoaleatorios**. Los números pseudoaleatorios no son realmente aleatorios porque son generados por un cálculo determinista, pero con solo mirar los números es casi imposible distinguirlos de los aleatorios.

El módulo "aleatorio" proporciona funciones que generan números pseudoaleatorios (que a partir de ahora llamaré "aleatorios").

La función `random` devuelve un float aleatorio entre 0.0 y 1.0 (incluyendo 0.0 pero no 1.0). Cada vez que llamas a la función `random`, obtienes el siguiente número en una larga serie. Para ver una muestra, ejecuta este bucle:

```
import random

for i in range(10):
    x = random.random()
    print(x)
```

Este programa produce la siguiente lista de 10 números aleatorios entre 0.0 y hasta, pero sin incluir 1.0.

```
0.11132867921152356
0.5950949227890241
0.04820265884996877
0.841003109276478
0.997914947094958
0.04842330803368111
0.7416295948208405
0.510535245390327
0.27447040171978143
0.028511805472785867
```

**Ejercicio 1**: Ejecuta el programa en su sistema y vea qué números obtiene. Ejecute el programa más de una vez y vea qué números obtiene.

La función `random` es solo una de las muchas funciones que manejan números aleatorios. La función `randint` toma los parámetros `low` y `high`, y devuelve un número entero entre `low` y `high` (incluyendo ambos).

```
>>> random.randint(5, 10)
5
>>> random.randint(5, 10)
9
```

Para elegir un elemento de una secuencia al azar, puedes usar `choice`:

```
>>> t = [1, 2, 3]
>>> random.choice(t)
2
>>> random.choice(t)
3
```

El módulo `random` también proporciona funciones para generar valores aleatorios a partir de distribuciones continuas que incluyen Gaussian, exponential, gamma y algunas más.

Copyright 2026

# Funciones matemáticas

Python tiene un módulo `math` que proporciona la mayoría de las funciones matemáticas conocidas. Antes de que podamos usar el módulo, tenemos que importarlo:

```
>>> import math
```

Esta declaración crea un **objeto de módulo** llamado math. Si imprime el objeto de módulo, obtiene alguna información al respecto:

```
>>> print(math)
<module 'math' (built-in)>
```

El objeto módulo contiene las funciones y variables definidas en el módulo. Para acceder a una de las funciones, debe especificar el nombre del módulo y el nombre de la función, separados por un punto.

```
>>> ratio = signal_power / noise_power
>>> decibels = 10 * math.log10(ratio)

>>> radians = 0.7
>>> height = math.sin(radians)
```

El primer ejemplo calcula la base logarítmica 10 de la relación señal-ruido. El módulo matemático también proporciona una función llamada `log` que calcula los logaritmos base `e`.

El segundo ejemplo encuentra el seno de 'radians'. El nombre de la variable es un indicio de que `sin` y las otras funciones trigonométricas (`cos`, `tan`, etc.) toman argumentos en radianes. Para convertir de grados a radianes, divide por 360 y multiplica por 2π:

```
>>> degrees = 45
>>> radians = degrees / 360.0 * 2 * math.pi
>>> math.sin(radians)
0.7071067811865476
```

La expresión `math.pi` obtiene la variable `pi` del módulo matemático. El valor de esta variable es una aproximación de **π**, con una precisión de aproximadamente 15 dígitos.

Copyright 2026

Si sabes trigonometría, puedes verificar el resultado anterior comparándolo con la raíz cuadrada de dos dividido por dos:

```
>>> math.sqrt(2)/2.0
0.7071067811865476
```

# Agregar nuevas funciones

Hasta ahora, solo hemos estado usando las funciones que vienen con Python, pero también es posible agregar nuevas funciones. Una **definición de función** especifica el nombre de una nueva función y la secuencia de sentencias que se ejecutan cuando se llama a la función. Una vez que definimos una función, podemos reutilizarla una y otra vez a lo largo de nuestro programa.

Aquí hay un ejemplo:

```
def print_lyrics():
    print("I'm a lumberjack, and I'm okay.")
    print('I sleep all night and I work all day.')
```

`def` es una palabra clave que indica que esta es una definición de función. El nombre de la función es `print_lyrics`. Las reglas para los nombres de funciones son las mismas que para los nombres de variables: las letras, los números y algunos signos de puntuación son legales, pero el primer carácter no puede ser un número. No puedes usar una palabra clave como el nombre de una función, y debes evitar tener una variable y una función con el mismo nombre.

Los paréntesis vacíos después del nombre indican que esta función no acepta ningún argumento. Más adelante construiremos funciones que toman argumentos como sus entradas.

La primera línea de la definición de la función se llama el **encabezado**; el resto se llama el **cuerpo** . El encabezado debe terminar con dos puntos y el cuerpo debe estar sangrado. Por convención, la sangría es siempre de cuatro espacios. El cuerpo puede contener cualquier número de declaraciones.

Las cadenas en las declaraciones impresas están entre comillas. Las comillas simples y las comillas dobles hacen lo mismo; la mayoría de las personas usan comillas simples, excepto en casos como éste, donde aparece una comilla simple (que también es un apóstrofe) en la cadena.

Si escribes una definición de función en modo interactivo, el intérprete imprime puntos suspensivos (...) para hacerle saber que la definición no está completa:

```
>>> def print_lyrics():
...     print("I'm a lumberjack, and I'm okay.")
...     print('I sleep all night and I work all day.')
...
```

Para finalizar la función, debe ingresar una línea vacía (esto no es necesario en un script).

La definición de una función crea una variable con el mismo nombre.

```
>>> print(print_lyrics)
<function print_lyrics at 0xb7e99e9c>
>>> print(type(print_lyrics))
<class 'function'>
```

El valor de `print_lyrics` es un **objeto**, que tiene el tipo "función ".

La sintaxis para llamar a la nueva función es la misma que para las funciones incorporadas:

```
>>> print_lyrics()
I'm a lumberjack, and I'm okay.
I sleep all night and I work all day.
```

Una vez que hayas definido una función, puedes usarla dentro de otra función. Por ejemplo, para repetir el refrán anterior, podríamos escribir una función llamada `repeat_lyrics`:

```
def repeat_lyrics():
    print_lyrics()
    print_lyrics()
```

Y luego llamar a "repeat_lyrics":

```
>>> repeat_lyrics()
I'm a lumberjack, and I'm okay.
I sleep all night and I work all day.
I'm a lumberjack, and I'm okay.
I sleep all night and I work all day.
```

Copyright 2026

Pero así no es realmente cómo va la canción.

# Definiciones y usos

Recopilando los fragmentos de código de la sección anterior, todo el programa se ve así:

https://trinket.io/embed/python3/3b61a1059e

Este programa contiene dos definiciones de funciones: `print_lyrics` y `repeat_lyrics`. Las definiciones de funciones se ejecutan igual que otras declaraciones, pero el efecto es crear objetos de funciones. Las sentencias dentro de la función no se ejecutan hasta que se llama a la función, y la definición de la función no genera salida.

Como es de esperar, debes crear una función antes de poder ejecutarla. En otras palabras, la definición de la función debe ejecutarse antes de la primera vez que se llama.

**Ejercicio 2** Mueve la última línea de este programa a la parte superior, para que la llamada a la función aparezca antes de las definiciones. Ejecuta el programa y observa qué mensaje de error recibe.

**Ejercicio 3** Mueve la llamada de la función a la parte inferior y mueve la definición de `print_lyrics` después de la definición de `repeat_lyrics`. ¿Qué pasa cuando ejecutas este programa?

# Flujo de ejecución

Para garantizar que una función se define antes de su primer uso, debes conocer el orden en que se ejecutan las instrucciones, lo que se denomina **flujo de ejecución**.

La ejecución siempre comienza en la primera declaración del programa. Las declaraciones se ejecutan de una en una, en orden de arriba a abajo.

Las definiciones de la función no alteran el flujo de ejecución del programa, pero recuerde que las sentencias dentro de la función no se ejecutan hasta que se llama a la función.

Una llamada de función es como un desvío en el flujo de ejecución. En lugar de ir a la siguiente instrucción, el flujo salta al cuerpo de la función, ejecuta todas las declaraciones allí y luego vuelve

a retomar donde se detuvo. Eso suena bastante simple, hasta que recuerdas que una función puede llamar a otra. Mientras se encuentra en medio de una función, el programa podría tener que ejecutar las instrucciones en otra función. ¡Pero mientras ejecuta esa nueva función, el programa podría tener que ejecutar otra función!

Afortunadamente, Python es bueno para mantener un registro de dónde está, por lo que cada vez que una función se completa, el programa retoma el lugar donde se quedó en la función que lo llamó. Cuando llega al final del programa, termina.

¿Cuál es la moraleja de este sórdido cuento? Cuando lees un programa, no siempre quieres leer de arriba a abajo. A veces tiene más sentido si sigues el flujo de ejecución.

# Parámetros y argumentos

Algunas de las funciones incorporadas que hemos visto requieren argumentos. Por ejemplo, cuando llamas `math.sin` pasas un número como argumento. Algunas funciones toman más de un argumento: `math.pow` toma dos, la base y el exponente.

Dentro de la función, los argumentos se asignan a variables llamadas **parámetros**. Aquí hay un ejemplo de una función definida por el usuario que toma un argumento:

```
def print_twice(bruce):
    print(bruce)
    print(bruce)
```

Esta función asigna el argumento a un parámetro llamado `bruce`. Cuando se llama a la función, imprime el valor del parámetro (cualquiera que sea) dos veces.

Esta función funciona con cualquier valor que se pueda imprimir.

```
>>> print_twice('Spam')
Spam
Spam
>>> print_twice(17)
17
17
>>> import math
>>> print_twice(math.pi)
```

```
3.141592653589793
3.141592653589793
```

Las mismas reglas de composición que se aplican a las funciones integradas también se aplican a las funciones definidas por el usuario, por lo que podemos usar cualquier tipo de expresión como argumento para `print_twice`:

```
>>> print_twice('Spam '*4)
Spam Spam Spam Spam
Spam Spam Spam Spam
>>> print_twice(math.cos(math.pi))
-1.0
-1.0
```

El argumento se evalúa antes de llamar a la función, por lo que en los ejemplos las expresiones "Spam '* 4 y math.cos (math.pi)` solo se evalúan una vez.

También puedes usar una variable como argumento:

```
>>> michael = 'Eric, the half a bee.'
>>> print_twice(michael)
Eric, the half a bee.
Eric, the half a bee.
```

El nombre de la variable que pasamos como argumento (`michael`) no tiene nada que ver con el nombre del parámetro (`bruce`). No importa cómo se llamó el valor de regreso a casa (en la persona que llama). Aquí en `print_twice`, llamamos a todos `bruce`.

# Funciones fructíferas y funciones nulas

Algunas de las funciones que utilizamos, como las funciones matemáticas, producen resultados. A falta de un nombre mejor, las llamo **funciones fructíferas**. Otras funciones, como `print_twice`, realizan una acción pero no devuelven un valor. Se les llama **funciones nulas** (void functions).

Cuando llama a una función fructífera, casi siempre quiere hacer algo con el resultado; por ejemplo, puede asignarlo a una variable o usarlo como parte de una expresión:

```
x = math.cos(radians)
golden = (math.sqrt(5) + 1) / 2
```

Cuando llamas a una función en modo interactivo, Python muestra el resultado:

```
>>> math.sqrt(5)
2.23606797749979
```

Pero en un script, si llama a una función fructífera y no almacena el resultado de la función en una variable, el valor de retorno desaparece en la nada.

```
math.sqrt(5)
```

Este script calcula la raíz cuadrada de 5, pero como no almacena el resultado en una variable ni muestra el resultado, no es muy útil.

Las funciones nulas pueden mostrar algo en la pantalla o tener algún otro efecto, pero no tienen un valor de retorno. Si intenta asignar el resultado a una variable, obtiene un valor especial llamado `None`.

```
>>> result = print_twice('Bing')
Bing
Bing
>>> print(result)
None
```

El valor `None` no es el mismo que la cadena `None`. Es un valor especial que tiene su propio tipo:

```
>>> print(type(None))
<class 'NoneType'>
```

Para devolver un resultado de una función, usamos la declaración `return` en nuestra función. Por ejemplo, podríamos hacer una función muy simple llamada `addtwo` que suma dos números y devuelve un resultado.

https://trinket.io/embed/python3/a50501b656

Cuando este script se ejecute, la instrucción `print` imprimirá" 8 "porque la función `addtwo` fue llamada con 3 y 5 como argumentos. Dentro de la función, los parámetros `a` y `b` fueron 3 y 5 respectivamente. La función calculó la suma de los dos números y la colocó en la variable de función local denominada `added`. Luego usó la declaración `return` para enviar el valor calculado al código de llamada como el resultado de la función, que se asignó a la variable `x` y se imprimió.

# ¿Para qué crear funciones?

Puede que no esté claro por qué vale la pena dividir un programa en funciones. Hay varias razones:

- La creación de una nueva función le brinda la oportunidad de nombrar un grupo de declaraciones, lo que hace que su programa sea más fácil de leer, comprender y depurar.
- Las funciones pueden hacer que un programa sea más pequeño eliminando el código repetitivo. Más tarde, si realiza un cambio, solo tendrá que hacerlo en un lugar.
- La división de un programa largo en funciones le permite depurar las partes una a la vez y luego ensamblarlas en un todo de trabajo.
- Las funciones bien diseñadas suelen ser útiles para muchos programas. Una vez que escribes y depuras una, puedes reutilizarla.

A lo largo del resto del libro, a menudo usaremos una definición de función para explicar un concepto. Parte de la habilidad de crear y usar funciones es hacer que una función capture adecuadamente una idea como "encontrar el valor más pequeño en una lista de valores". Más adelante, te mostraremos el código que encuentra el más pequeño en una lista de valores y lo presentaremos como una función llamada `min` que toma una lista de valores como su argumento y devuelve el valor más pequeño de la lista.

# Depurando

Si estás utilizando un editor de texto para escribir tus scripts, es posible que tengas problemas con los espacios y las tabulaciones. La mejor manera de evitar estos problemas es usar espacios exclusivamente (sin tabulaciones). La mayoría de los editores de texto que conocen Python lo hacen de forma predeterminada, pero algunos no.

Las tabulaciones y los espacios suelen ser invisibles, lo que los hace difíciles de depurar, así que trata de encontrar un editor que administre la sangría por ti.

Además, no olvides guardar tu programa antes de ejecutarlo. Algunos entornos de desarrollo lo hacen automáticamente, pero otros no. En ese caso, el programa que estás viendo en el editor de

texto no es el mismo que el programa que estás ejecutando.

La depuración puede llevar mucho tiempo si continúas ejecutando el mismo programa incorrecto una y otra vez.

Asegúrate de que el código que estás viendo es el código que estás ejecutando. Si no estás seguro, coloca algo como `print("hola")` al principio del programa y vuelve a ejecutarlo. ¡Si no ves `hola`, no estás ejecutando el programa correcto!

# Ejercicios

**Ejercicio 4** ¿Cuál es el propósito de la palabra clave "def" en Python?

a) Es una jerga que significa "el siguiente código es realmente genial"
b) Indica el inicio de una función
c) Indica que la siguiente sección de código con sangría debe almacenarse para más adelante
d) `b` y `c` son verdaderas
e) Ninguna de las anteriores

**Ejercicio 5** ¿Qué imprimirá el siguiente programa de Python?

```
def fred():
   print("Zap")

def jane():
   print("ABC")

jane()
fred()
jane()
```

a) Zap ABC jane fred jane
b) Zap ABC Zap
c) ABC Zap jane
d) ABC Zap ABC
e) Zap Zap Zap

**Ejercicio 6** reescribe su cálculo de pago con tiempo y medio para horas extras y crea una función llamada `computepay` que tome dos parámetros (`hours` y `rate`).

```
Enter Hours: 45
Enter Rate: 10
Pay: 475.0
```

**Ejercicio 7** reescribe el programa de calificación del capítulo anterior utilizando una función llamada `computegrade` que tome una puntuación como parámetro y devuelva una nota como una cadena.

```
Score   Grade
> 0.9     A
> 0.8     B
> 0.7     C
> 0.6     D
<= 0.6    F
```

```
Program Execution:
```

```
Enter score: 0.95
A
```

```
Enter score: perfect
Bad score
```

```
Enter score: 10.0
Bad score
```

```
Enter score: 0.75
C
```

```
Enter score: 0.5
F
```

Ejecuta el programa repetidamente para probar los diferentes valores de entrada.

Copyright 2026

# 5 Bucles

## Actualizando variables

Un patrón común en las asignaciones es una instrucción de asignación que actualiza una variable, donde el nuevo valor de la variable depende de la antigua.

```
python
x = x + 1
```

Esto significa que "obtenga el valor actual de `x`, agregue 1 y luego actualice `x` con el nuevo valor".

Si intentas actualizar una variable que no existe, obtienes un error, porque Python evalúa el lado derecho antes de asignar un valor a `x`:

```
>>> x = x + 1
NameError: name 'x' is not defined
```

Antes de poder actualizar una variable, debes **inicializarla**:

```
>>> x = 0
>>> x = x + 1
```

Actualizar una variable agregando 1 se llama **incrementar**.

## La declaración 'while'

Los ordenadores se utilizan a menudo para automatizar tareas repetitivas. Repetir tareas idénticas o similares sin cometer errores es algo que a los ordenadores se les da muy bien y a las personas se les da muy mal. Debido a que la iteración es tan común, Python proporciona varias características de lenguaje para que sea más fácil.

Copyright 2026

Una forma de iteración en Python es la instrucción `while`. Aquí hay un programa simple que cuenta a partir de cinco y luego dice "¡Explosión!".

```
n = 5
while n > 0:
    print(n)
    n = n - 1
print('Blastoff!')
```

Casi se puede leer la instrucción `while` como si fuera inglés. Significa que "Mientras `n` es mayor que 0, muestra el valor de `n` y luego reduce el valor de `n` en 1. Cuando llegues a 0, sal de la instrucción `while` y muestra la palabra `Blastoff !`"

Más formalmente, aquí está el flujo de ejecución para una instrucción `while`:

1. Evalúa la condición, obteniendo "Verdadero" o "Falso".
2. Si la condición es falsa, salga de la instrucción `while` y continúa con la ejecución en la siguiente instrucción.
3. Si la condición es verdadera, ejecuta el cuerpo y luego vuelve al paso 1.

Este tipo de flujo se llama un bucle porque el tercer paso vuelve a la parte superior. Cada vuelta del bucle la llamamos una **iteración**. Para el bucle anterior, diríamos que "tenía cinco iteraciones", lo que significa que el cuerpo del bucle se ejecutó cinco veces.

El cuerpo del bucle debe cambiar el valor de una o más variables para que finalmente la condición se vuelva falsa y el bucle finalice. Llamamos a la variable que cambia cada vez que el bucle se ejecuta y controla cuando el bucle termina la **variable de iteración**. Si no hay una variable de iteración, el bucle se repetirá para siempre, dando como resultado un **bucle infinito**.

# Bucles infinitos

Una fuente inagotable de diversión para los programadores es la observación de que las instrucciones en el champú, "Enjabonar, enjuagar, repetir", son un bucle infinito porque no hay una **variable de iteración** que indique cuántas veces se debe ejecutar el bucle.

En el caso de `countdown`, podemos probar que el bucle termina porque sabemos que el valor de `n` es finito, y podemos ver que el valor de `n` se reduce cada vez que pasa por el bucle, por lo que eventualmente tiene que llegar a 0. Otras veces, un bucle es obviamente infinito porque no tiene ninguna variable de iteración.

# "Infinite loops" y `break`

A veces no sabes que es hora de terminar un ciclo hasta que lleguas a la mitad del cuerpo. En ese caso, puedes escribir un bucle infinito a propósito y luego usar la instrucción `break` para saltar fuera del bucle.

Este bucle es obviamente un **bucle infinito** porque la expresión lógica en la instrucción `while` es simplemente la constante lógica `True`:

```
n = 10
while True:
    print(n, end=' ')
    n = n - 1
print('Done!')
```

Si cometes el error y ejecuta este código, aprenderás rápidamente cómo detener un proceso de Python fuera de control en su sistema o dónde encontrarás el botón de apagado en su ordenador. Este programa se ejecutará para siempre o hasta que la batería se agote porque la expresión lógica en la parte superior del bucle siempre es verdadera.

Si bien este es un bucle infinito disfuncional, aún podemos usar este patrón para crear bucles útiles, siempre y cuando agreguemos cuidadosamente `break` cuando hayamos alcanzado la condición de salida.

Por ejemplo, supongamos que desea recibir información del usuario hasta que escriban `done`. Podrías escribir:

https://trinket.io/embed/python3/7a1dd00756

La condición del bucle es `True`, que siempre es verdadera, por lo que el bucle se ejecuta repetidamente hasta que llega a la instrucción break.

Cada vez que itera, le pide al usuario que de un input. Si el usuario escribe `done`, la instrucción `break` sale del bucle. De lo contrario, el programa hace eco de lo que sea que el usuario escriba y vuelve a la parte superior del bucle. Aquí hay una muestra de ejecución:

Copyright 2026

```
> hello there
hello there
> finished
finished
> done
Done!
```

Esta forma de escribir bucles `while` es común porque puede verificar la condición en cualquier parte del bucle (no solo en la parte superior) y puede expresar la condición de detención afirmativamente ("detenerse cuando esto sucede") en lugar de negativamente ("seguir adelante hasta que eso suceda").

# Finalización de iteraciones con `continue`

A veces te encuentras en una iteración de un bucle y deseas finalizar la iteración actual y pasar de inmediato a la siguiente iteración. En ese caso, puedes usar la instrucción `continue` para saltar a la siguiente iteración sin terminar el cuerpo del bucle para la iteración actual.

Aquí hay un ejemplo de un bucle que copia su entrada hasta que el usuario escribe "hecho", pero trata las líneas que comienzan con el carácter de hash como líneas que no se imprimen (como los comentarios de Python).

https://trinket.io/embed/python3/f1f5dc61e1

Aquí hay una muestra de ejecución de este nuevo programa con `continue` agregado.

```
> hello there
hello there
> # don't print this
> print this!
print this!
> done
Done!
```

Todas las líneas se imprimen, excepto las que comienza con el signo hash porque cuando se ejecuta `continue`, finaliza la iteración actual y vuelve a la instrucción `while` para iniciar la siguiente iteración, omitiendo la instrucción `print`.

# Bucles definidos usando `for`

A veces queremos recorrer un **conjunto** de cosas como una lista de palabras, las líneas de un archivo o una lista de números. Cuando tenemos una lista de cosas para recorrer, podemos construir un bucle **definido** usando una instrucción `for`. Llamamos a la frase `while` un bucle **indefinido** porque simplemente se repite hasta que alguna condición se convierte en `Falsa`, mientras que el bucle `for` se repite en un conjunto conocido de elementos, por lo que se ejecuta a través de tantas iteraciones como Elementos del conjunto.

La sintaxis de un bucle `for` es similar al bucle `while` en que hay una sentencia `for` y un cuerpo de bucle:

```
friends = ['Joseph', 'Glenn', 'Sally']
for friend in friends:
    print('Happy New Year:', friend)
print('Done!')
```

En términos de Python, la variable `friends` es una lista [1] de tres cadenas y el bucle `for` recorre la lista y ejecuta el cuerpo una vez para cada una de las tres cadenas en la lista que da como resultado esta salida:

```
Happy New Year: Joseph
Happy New Year: Glenn
Happy New Year: Sally
Done!
```

La traducción de este bucle `for` al inglés no es tan directa como el `while`, pero si piensas en los amigos como un **conjunto**, quedaría así: "Ejecute las declaraciones en el cuerpo del bucle `for` una vez para cada amigo **en** el conjunto llamado amigos ".

Mirando el bucle `for`, **for** y **in** son palabras clave reservadas de Python, y `friend` y `friends` son variables.

Copyright 2026

```
for friend in friends:
    print('Happy New Year:', friend)
```

En particular, `friend` es la **variable de iteración** para el bucle for. La variable `friend` cambia para cada iteración del bucle y se controla cuando se completa el bucle `for`. La **variable de iteración** pasa sucesivamente por las tres cadenas almacenadas en la variable `friends`.

# Patrones de bucle

A menudo, utilizamos un bucle `for` o `while` para revisar una lista de elementos o el contenido de un archivo y estamos buscando algo como el valor más grande o más pequeño de los datos que escaneamos.

Estos bucles generalmente se componen de:

- Una o más variables inicializadas antes de que comience el ciclo.
- Realización de algunos cálculos en cada elemento del cuerpo del bucle, posiblemente cambiando las variables en el cuerpo del bucle
- Observación las variables resultantes cuando se completa el bucle.

Usaremos una lista de números para demostrar los conceptos y la construcción de estos patrones de bucle.

# Bucles de suma y recuento

Por ejemplo, para contar el número de elementos en una lista, escribiríamos el siguiente bucle `for` :

```
count = 0
for itervar in [3, 41, 12, 9, 74, 15]:
    count = count + 1
print('Count: ', count)
```

Establecemos la variable `count` en cero antes de que comience el bucle, luego escribimos un bucle `for` para ejecutar la lista de números. Nuestra variable **iterativa** se llama `itervar` y, aunque no usamos `itervar` en el bucle, controla el bucle y hace que el cuerpo se ejecute una vez para cada uno de los valores de la lista.

En el cuerpo del bucle, agregamos 1 al valor actual de `count` para cada uno de los valores en la lista. Mientras el bucle se está ejecutando, el valor de `count` es el número de valores que hemos visto "hasta ahora".

Una vez que el bucle se completa, el valor de `count` es el número total de elementos. El número total "cae en nuestro regazo" al final del bucle. Construimos el bucle para que tengamos lo deseado cuando el bucle finalice.

Otro bucle similar que calcula el total de un conjunto de números es el siguiente:

```
total = 0
for itervar in [3, 41, 12, 9, 74, 15]:
    total = total + itervar
print('Total: ', total)
```

En este bucle nosotros **sí** usamos la **variable de iteración**. En lugar de simplemente agregar uno al `recuento` como en el bucle anterior, agregamos el número real (3, 41, 12, etc.) al total acumulado durante cada iteración de bucle. Antes de que el bucle comience, `total` es cero porque aún no hemos visto ningún valor. Durante el bucle, `total` es el total acumulado, y al final del bucle `total` es el total general de todos los valores en el lista.

A medida que se ejecuta el bucle, `total` acumula la suma de los elementos; una variable utilizada de esta manera a veces se llama un **acumulador**.

Ni el bucle de conteo ni el bucle de suma son particularmente útiles en la práctica porque hay funciones incorporadas `len()` y `sum()` que computan el número de elementos en una lista y el total de los elementos en la lista respectivamente .

# Bucles máximos y mínimos

Para encontrar el valor más grande en una lista o secuencia, construimos el siguiente bucle:

```
largest = None
print('Before:', largest)
for itervar in [3, 41, 12, 9, 74, 15]:
    if largest is None or itervar > largest :
        largest = itervar
    print('Loop:', itervar, largest)
print('Largest:', largest)
```

Cuando el programa se ejecuta, la salida es la siguiente:

```
Before: None
Loop: 3 3
Loop: 41 41
Loop: 12 41
Loop: 9 41
Loop: 74 74
Loop: 15 74
Largest: 74
```

La variable `largest` se comprende mejor como el "valor más grande que hemos visto hasta ahora". Antes del bucle, asignamos a `largest` la constante `None`. `None` es un valor constante especial que podemos almacenar en una variable para marcar la variable como "vacía".

Antes de que comience el bucle, el valor más grande que hemos visto hasta ahora es 'Ninguno', ya que todavía no hemos visto ningún valor. Mientras el bucle se está ejecutando, si `largest` es `None` entonces tomamos el primer valor que vemos como el más grande hasta ahora.

Después de la primera iteración, `largest` ya no es `None`, así que la segunda parte de la expresión lógica compuesta que comprueba, `itervar > largest`, se dispara solo cuando vemos un valor que es más grande que el "más grande hasta ahora". Cuando vemos un nuevo valor "aún más grande", tomamos ese nuevo valor para "más grande". Puede ver en la salida del programa que "más grande" progresa de 3 a 41 a 74.

Al final del ciclo, hemos escaneado todos los valores y la variable `largest` ahora contiene el valor más grande en la lista.

Para calcular el número más pequeño, el código es muy similar con un pequeño cambio:

```
smallest = None
print('Before:', smallest)
for itervar in [3, 41, 12, 9, 74, 15]:
    if smallest is None or itervar < smallest:
        smallest = itervar
    print('Loop:', itervar, smallest)
print('Smallest:', smallest)
```

Copyright 2026

Nuevamente, 'smallest' es el "más pequeño hasta ahora" antes, durante y después de que se ejecute el bucle. Cuando el bucle se ha completado, `smallest` contiene el valor mínimo en la lista.

De nuevo, al contar y sumar, las funciones incorporadas `max()` y `min()` hacen que la escritura de estos bucles exactos sea innecesaria.

La siguiente es una versión simple de la función `min()` incorporada en Python:

```
def min(values):
    smallest = None
    for value in values:
        if smallest is None or value < smallest:
            smallest = value
    return smallest
```

En la versión de la función anterior, eliminamos todas las declaraciones `print` para que sean equivalentes a la función `min` que ya está incorporada en Python.

# Depurando

A medida que comienzas a escribir programas más grandes, puedes pasar más tiempo depurando. Más código significa más posibilidades de cometer un error y más lugares para que los errores se oculten.

Una forma de reducir el tiempo de depuración es "depuración por bisección". Por ejemplo, si hay 100 líneas en tu programa y las verificas una cada vez, te tomaría 100 pasos.

En su lugar, intenta romper el problema por la mitad. Busca en la mitad del programa, o cerca de él, un valor intermedio que puedas verificar. Agrega una declaración `print` (o algo que puedas verificar) y ejecuta el programa.

Si la verificación del punto medio es incorrecta, el problema debe estar en la primera mitad del programa. Si es correcto, el problema está en la segunda mitad.

Cada vez que realizas una verificación como esta, reduces a la mitad el número de líneas en las que tiene que buscar. Después de seis pasos (que es mucho menos que 100), se reduciría a una o dos líneas de código, al menos en teoría.

En la práctica, no siempre está claro cuál es la "mitad del programa" y no siempre es posible verificarlo. No tiene sentido contar líneas y encontrar el punto medio exacto. En su lugar, piensa en los lugares del programa en los que podría haber errores y en los lugares donde es fácil poner un chequeo. Luego, elije un lugar donde pienses que las probabilidades son casi las mismas de que el error esté antes o después de la comprobación.

# Ejercicios

**Ejercicio 1** escribe un programa que lea números repetidamente hasta que el usuario ingrese "listo". Una vez que se ingrese "listo", imprima el total, el recuento y el promedio de los números. Si el usuario ingresa algo que no sea un número, detecta su error usando `try` y `except` e imprime un mensaje de error y salta al siguiente número.

```
Enter a number: 4
Enter a number: 5
Enter a number: bad data
Invalid input
Enter a number: 7
Enter a number: done
16 3 5.333333333333333
```

**Ejercicio 2** escribe otro programa que solicite una lista de números como arriba y al final imprima el máximo y el mínimo de los números en lugar del promedio.

---

> [1]. Examinaremos las listas con más detalle en un capítulo posterior. ↩

# 6 Cadenas

## Una cadena es una secuencia

Una cadena es una **secuencia** de caracteres. Puede acceder a los caracteres uno a la vez con el operador de corchete:

```
>>> fruit = 'banana'
>>> letter = fruit[1]
```

La segunda declaración extrae el caracter en la posición de índice 1 de la variable `fruit` y lo asigna a la variable `letter`.

La expresión entre corchetes se llama un índice. El índice indica qué caracter de la secuencia desea.

Pero puede que no consigas lo que esperas:

```
>>> print(letter)
a
```

Para la mayoría de las personas, la primera letra de "banana" es `b`, no `a`. Pero en Python, el índice es un desplazamiento desde el principio de la cadena, y el índice de la primera letra es cero.

```
>>> letter = fruit[0]
>>> print(letter)
b
```

Entonces `b` es la 0ª letra ("zero-eth") de "banana ", `a` es la 1ª letra ("one-eth"), y `n` es la 2ª letra ("two-eth").

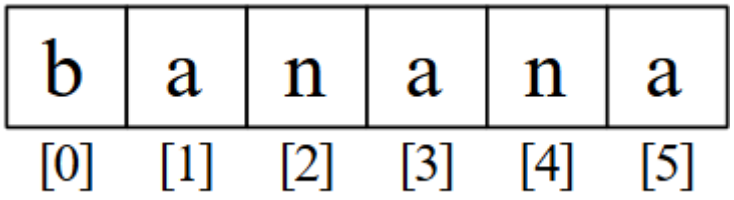


Imagen - String Indexes

Puedes usar cualquier expresión, incluidas las variables y los operadores, como un índice, pero el valor del índice debe ser un entero. De lo contrario obtendrás:

```
>>> letter = fruit[1.5]
TypeError: string indices must be integers
```

# Obteniendo la longitud de una cadena usando `len`

`len` es una función incorporada que devuelve el número de caracteres en una cadena:

```
>>> fruit = 'banana'
>>> len(fruit)
6
```

Para obtener la última letra de una cadena, puedes tener la tentación de probar algo como esto:

```
>>> length = len(fruit)
>>> last = fruit[length]
IndexError: string index out of range
```

La razón para el `IndexError` es que no hay una letra en `banana` con el índice 6. Ya que comenzamos a contar desde cero, las seis letras están numeradas del 0 al 5. Para obtener el último caracter, debes restar 1 de `lenght`:

```
>>> last = fruit[length-1]
>>> print(last)
a
```

Alternativamente, puede usar índices negativos, que cuentan hacia atrás desde el final de la cadena. La expresión `fruit[-1]` extrae la última letra, `fruit[-2]` extrae la penúltima, y así sucesivamente.

Copyright 2026

# Atravesar una cadena con un bucle

Muchos algoritmos implican procesar una cadena, caracter a caracter. A menudo comienzan al principio, seleccionan un caracter por iteración, le hacen algo y continúan hasta el final. Este patrón de procesamiento se denomina **recorrido**. Una forma de escribir un recorrido es con un bucle `while`:

```
index = 0
while index < len(fruit):
    letter = fruit[index]
    print(letter)
    index = index + 1
```

Este bucle atraviesa la cadena y muestra cada letra en una línea separada. La condición del bucle es `index < len(fruit)`, por lo que cuando `index` es igual a la longitud de la cadena, la condición es falsa, y el cuerpo del bucle no se ejecuta. El último caracter accedido es el que tiene el índice `len(fruit)-1`, que es el último caracter de la cadena.

**Ejercicio 1** escribe un bucle `while` que comience en el último caracter de la cadena y avance hacia el primer caracter de la cadena, imprimiendo cada letra en una línea separada.

Otra forma de escribir un recorrido es con un bucle `for`:

```
for char in fruit:
    print(char)
```

Cada vez que pasa por el bucle, el siguiente caracter de la cadena se asigna a la variable `char`. El bucle continúa hasta que no quedan caracteres.

# String slices (segmentando cadenas)

Un trozo de una cadena se llama **segmento**. Seleccionar segmento es similar a seleccionar un caracter:

```
>>> s = 'Monty Python'
>>> print(s[0:5])
Monty
```

Copyright 2026

```
>>> print(s[6:12])
Python
```

El operador devuelve el segmento desde el caracter "n-eth" al caracter "m-eth", incluido el primero pero excluyendo el último.

Si omites el primer índice (antes de los dos puntos), el segmento comienza al principio de la cadena. Si omites el segundo índice, la segmento llega hasta el final de la cadena:

```
>>> fruit = 'banana'
>>> fruit[:3]
'ban'
>>> fruit[3:]
'ana'
```

Si el primer índice es mayor o igual que el segundo, el resultado es una **cadena vacía**, representada por dos comillas:

```
>>> fruit = 'banana'
>>> fruit[3:3]
''
```

Una cadena vacía no contiene caracteres y tiene una longitud de 0, pero aparte de eso, es la misma que cualquier otra cadena.

**Ejercicio 2**: Dado que `fruit` es una cadena, ¿qué significa `fruit[:]`?

# Las cadenas son inmutables

Es tentador utilizar el operador en el lado izquierdo de una tarea, con la intención de cambiar un caracter en una cadena. Por ejemplo:

```
>>> greeting = 'Hello, world!'
>>> greeting[0] = 'J'
TypeError: 'str' object does not support item assignment
```

El "objeto" en este caso es la cadena y el "elemento" es el caracter que intentó asignar. Por ahora, un **objeto** es lo mismo que un valor, pero refinaremos esa definición más adelante. Un **elemento**

es uno de los valores en una secuencia.

El motivo del error es que las cadenas son **inmutables**, lo que significa que no puede cambiar una cadena existente. Lo mejor que puedes hacer es crear una nueva cadena que sea una variación del original:

```
>>> greeting = 'Hello, world!'
>>> new_greeting = 'J' + greeting[1:]
>>> print(new_greeting)
Jello, world!
```

Este ejemplo concatena una nueva primera letra en una porción de "greeting". No tiene efecto en la cadena original.

# Bucles y conteos

El siguiente programa cuenta el número de veces que aparece la letra `a` en una cadena:

```
word = 'banana'
count = 0
for letter in word:
    if letter == 'a':
        count = count + 1
print(count)
```

Este programa demuestra otro algoritmo llamado **contador**. La variable `count` se inicializa a 0 y luego se incrementa cada vez que se encuentra un `a`. Cuando el bucle sale, `count` contiene el resultado: el número total de `a`s.

**Ejercicio 3**

Encapsula este código en una función llamada `count`, y generalízalo para que acepte la cadena y la letra como argumentos.

# El operador `in`

La palabra `in` es un operador booleano que toma dos cadenas y devuelve `True` si la primera aparece como una subcadena en la segunda:

```
>>> 'a' in 'banana'
True
>>> 'seed' in 'banana'
False
```

# Comparación de cadenas

Los operadores de comparación trabajan en cadenas. Para ver si dos cadenas son iguales:

```
if word == 'banana':
    print('All right, bananas.')
```

Otras operaciones de comparación son útiles para poner las palabras en orden alfabético:

```
if word < 'banana':
    print('Your word,' + word + ', comes before banana.')
elif word > 'banana':
    print('Your word,' + word + ', comes after banana.')
else:
    print('All right, bananas.')
```

Python no maneja las letras mayúsculas y minúsculas de la misma manera que las personas. Todas las letras mayúsculas vienen antes de todas las letras minúsculas, así que:

```
Your word, Pineapple, comes before banana.
```

Una forma común de abordar este problema es convertir las cadenas a un formato estándar, como en minúsculas, antes de realizar la comparación. Tenlo en cuenta en caso de que tengas que defenderte contra un hombre armado con una piña.

# métodos 'de cadena'

Las cadenas son un ejemplo de objetos de Python. Un objeto contiene tanto datos (la propia cadena real) como **métodos**, que son efectivamente funciones que están integradas en el objeto y están disponibles para cualquier **instancia** (copia) del objeto.

Python tiene una función llamada `dir` que lista los métodos disponibles para un objeto. La función `type` muestra el tipo de un objeto y la función `dir` muestra los métodos disponibles.

```
>>> stuff = 'Hello world'
>>> type(stuff)
<class 'str'>
>>> dir(stuff)
['capitalize', 'casefold', 'center', 'count', 'encode',
'endswith', 'expandtabs', 'find', 'format', 'format_map',
'index', 'isalnum', 'isalpha', 'isdecimal', 'isdigit',
'isidentifier', 'islower', 'isnumeric', 'isprintable',
'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower',
'lstrip', 'maketrans', 'partition', 'replace', 'rfind',
'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip',
'split', 'splitlines', 'startswith', 'strip', 'swapcase',
'title', 'translate', 'upper', 'zfill']
>>> help(str.capitalize)
Help on method_descriptor:

capitalize(...)
    S.capitalize() -> str

    Return a capitalized version of S, i.e. make the first character
    have upper case and the rest lower case.
>>>
```

Mientras que la función `dir` enumera los métodos, y puedes usar `help` para obtener alguna documentación simple sobre un método, una mejor fuente de documentación para los métodos de cadena sería https://docs.python.org/3.7/library/stdtypes.html#string-methods.

Llamar a un **método** es similar a llamar a una función (toma argumentos y devuelve un valor) pero la sintaxis es diferente. Llamamos a un método agregando el nombre del método al nombre de la variable utilizando el punto como delimitador.

Por ejemplo, el método `upper` toma una cadena y devuelve una nueva cadena con todas las letras en mayúsculas:

En lugar de la sintaxis de la función `upper(word)`, utiliza la sintaxis del método `word.upper()`.

```
>>> word = 'banana'
>>> new_word = word.upper()
>>> print(new_word)
BANANA
```

Esta forma de notación de puntos especifica el nombre del método, `upper`, y la cadena a la que aplicar el método. Los paréntesis vacíos indican que este método no toma ningún argumento.

Una llamada al método se llama **invocación**; en este caso, diríamos que estamos invocando `upper` en `word`.

Por ejemplo, hay un método de cadena llamado `find` que busca la posición de una cadena dentro de otra:

```
>>> word = 'banana'
>>> index = word.find('a')
>>> print(index)
1
```

En este ejemplo, invocamos `find` en `word` y pasamos la letra que buscamos como parámetro.

El método `find` puede encontrar subcadenas y caracteres:

```
>>> word.find('na')
2
```

Puede tomar como segundo argumento el índice donde debe comenzar:

```
>>> word.find('na', 3)
4
```

Una tarea común es eliminar los espacios en blanco (espacios, tabuladores o nuevas líneas) desde el principio y el final de una cadena usando el método `strip`:

Copyright 2026

```
>>> line = '  Here we go  '
>>> line.strip()
'Here we go'
```

Algunos métodos como **startswith** devuelven valores booleanos.

```
>>> line = 'Have a nice day'
>>> line.startswith('Have')
True
>>> line.startswith('h')
False
```

Notarás que `startswith` requiere mayúsculas y minúsculas, por lo que a veces tomamos una línea y las mapeamos en minúsculas antes de realizar cualquier comprobación utilizando el método `lower`.

```
>>> line = 'Have a nice day'
>>> line.startswith('h')
False
>>> line.lower()
'have a nice day'
>>> line.lower().startswith('h')
True
```

En el último ejemplo, se llama al método `lower` y luego usamos `startswith` para ver si la cadena en minúsculas resultante comienza con la letra "h". Mientras tengamos cuidado con el orden, podemos realizar múltiples llamadas de método en una sola expresión.

**Ejercicio 4**

Existe un método de cadena llamado `count` que es similar a la función en el ejercicio anterior. Lee la documentación de este método en https://docs.python.org/3.7/library/stdtypes.html#string-methods y escribe una invocación que cuente el número de veces que aparece la letra a en "banana".

# Parseando cadenas

A menudo, queremos buscar en una cadena y encontrar una subcadena. Por ejemplo, si se nos presentara una serie de líneas con el siguiente formato:

`From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008`

y quisiéramos extraer solo la segunda mitad de la dirección (es decir, `uct.ac.za`) de cada línea, podemos hacerlo usando el método `find` y el corte de cadena.

Primero, encontraremos la posición del signo en la cadena. Luego encontraremos la posición del primer espacio **después de** el signo de entrada. Y luego usaremos el corte de cadena para extraer la parte de la cadena que estamos buscando.

```
>>> data = 'From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008'
>>> atpos = data.find('@')
>>> print(atpos)
21
>>> sppos = data.find(' ',atpos)
>>> print(sppos)
31
>>> host = data[atpos+1:sppos]
>>> print(host)
uct.ac.za
>>>
```

Usamos una versión del método `find` que nos permite especificar una posición en la cadena donde queremos que `find` comience a buscar. Cuando cortamos, extraemos los caracteres de "uno más allá del signo de entrada hasta **pero sin incluir** el caracter de espacio".

La documentación para el método `find` está disponible en
https://docs.python.org/3.7/library/stdtypes.html#string-methods.

# Operador de formato

El operador de formato, `%` nos permite construir cadenas, reemplazando partes de las cadenas con los datos almacenados en variables. Cuando se aplica a enteros, `%` es el operador de módulo. Pero cuando el primer operando es una cadena, `%` es el operador de formato.

Copyright 2026

El primer operando es la **cadena de formato**, que contiene una o más **secuencias de formato** que especifican cómo se formatea el segundo operando. El resultado es una cadena.

Por ejemplo, la secuencia de formato "%d" significa que el segundo operando debe formatearse como un entero ("d" significa "dígito"):

```
>>> camels = 42
>>> '%d' % camels
'42'
```

El resultado es la cadena "42", que no debe confundirse con el valor entero "42".

Una secuencia de formato puede aparecer en cualquier parte de la cadena, por lo que puede incrustar un valor en una oración:

```
>>> camels = 42
>>> 'I have spotted %d camels.' % camels
'I have spotted 42 camels.'
```

Si hay más de una secuencia de formato en la cadena, el segundo argumento debe ser una tupla [1]. Cada secuencia de formato se empareja con un elemento de la tupla, en orden.

El siguiente ejemplo utiliza "%d" para formatear un número entero, "%g" para formatear un número de punto flotante (no pregunte por qué) y "%s" para formatear una cadena:

```
>>> 'In %d years I have spotted %g %s.' % (3, 0.1, 'camels')
'In 3 years I have spotted 0.1 camels.'
```

El número de elementos en la tupla debe coincidir con el número de secuencias de formato en la cadena. Los tipos de elementos también deben coincidir con las secuencias de formato:

```
>>> '%d %d %d' % (1, 2)
TypeError: not enough arguments for format string
>>> '%d' % 'dollars'
TypeError: %d format: a number is required, not str
```

En el primer ejemplo, no hay suficientes elementos; en el segundo, el elemento es del tipo incorrecto.

El operador de formato es poderoso, pero puede ser difícil de usar. Puedes leer más sobre esto en

https://docs.python.org/3.7/library/stdtypes.html#printf-style-string-formatting.

# Depuración

Una habilidad que deberías cultivar a medida que programas siempre es preguntarte: "¿Qué podría salir mal aquí?" o alternativamente, "¿Qué locura podría hacer nuestro usuario para bloquear nuestro programa (aparentemente) perfecto?"

Por ejemplo, mira el programa que usamos para demostrar el bucle `while` en el capítulo sobre iteración:

https://trinket.io/embed/python3/406b93ae34

Mira lo que sucede cuando el usuario ingresa una línea de entrada vacía:

```
> hello there
hello there
> # don't print this
> print this!
print this!
>
Traceback (most recent call last):
  File "copytildone.py", line 3, in <module>
    if line[0] == '#':
IndexError: string index out of range
```

El código funciona bien hasta que se presenta una línea vacía. Entonces no hay caracter `0`, por lo que obtenemos un error. Existen dos soluciones para hacer que la línea tres sea "segura" incluso si la línea está vacía.

Una posibilidad es simplemente usar el método `startswith` que devuelve `False` si la cadena está vacía.

```
    if line.startswith('#'):
```

Otra forma es escribir de forma segura la instrucción `if` utilizando el patrón **guardian** y asegurarse de que la segunda expresión lógica se evalúa solo cuando hay al menos un caracter en la cadena:

```
    if len(line) > 0 and line[0] == '#':
```

# Ejercicios

**Ejercicio 5** toma el siguiente código de Python que almacena una cadena: `

`str = 'X-DSPAM-Confidence: 0.8475'`

Usa `find` y el corte de cadena para extraer la parte de la cadena después del caracter de dos puntos y luego use la función `float` para convertir la cadena extraída en un número de punto flotante.

**Ejercicio 6**

Lee la documentación de los métodos de cadena en

https://docs.python.org/3.7/library/stdtypes.html#string-methods

Es posible que desees experimentar con algunos de ellos para asegurarte de que comprendes cómo funcionan. `strip` y `replace` son particularmente útiles.

La documentación utiliza una sintaxis que puede ser confusa. Por ejemplo, en `find (sub [, start [, end]])`, los corchetes indican argumentos opcionales. Entonces se requiere `sub`, pero `start` es opcional, y si incluye `start`, entonces `end` es opcional.

---

> [1]. Una tupla es una secuencia de valores separados por comas dentro de un par de paréntesis. Cubriremos las tuplas en el Capítulo 10 ↩

# 7 Archivos

## Persistencia

Hasta ahora, hemos aprendido cómo escribir programas y comunicar nuestras intenciones a la **Unidad de procesamiento central** mediante la ejecución condicional, las funciones y las iteraciones. Hemos aprendido cómo crear y usar estructuras de datos en la **Memoria principal**. La CPU y la memoria son donde nuestro software funciona y se ejecuta. Es donde sucede todo el "pensamiento".

Pero si recuerdas de nuestras discusiones sobre la arquitectura del hardware, una vez que se apaga la alimentación, cualquier cosa almacenada en la CPU o la memoria principal se borra. Hasta ahora, nuestros programas solo han sido ejercicios transitorios y divertidos para aprender Python.

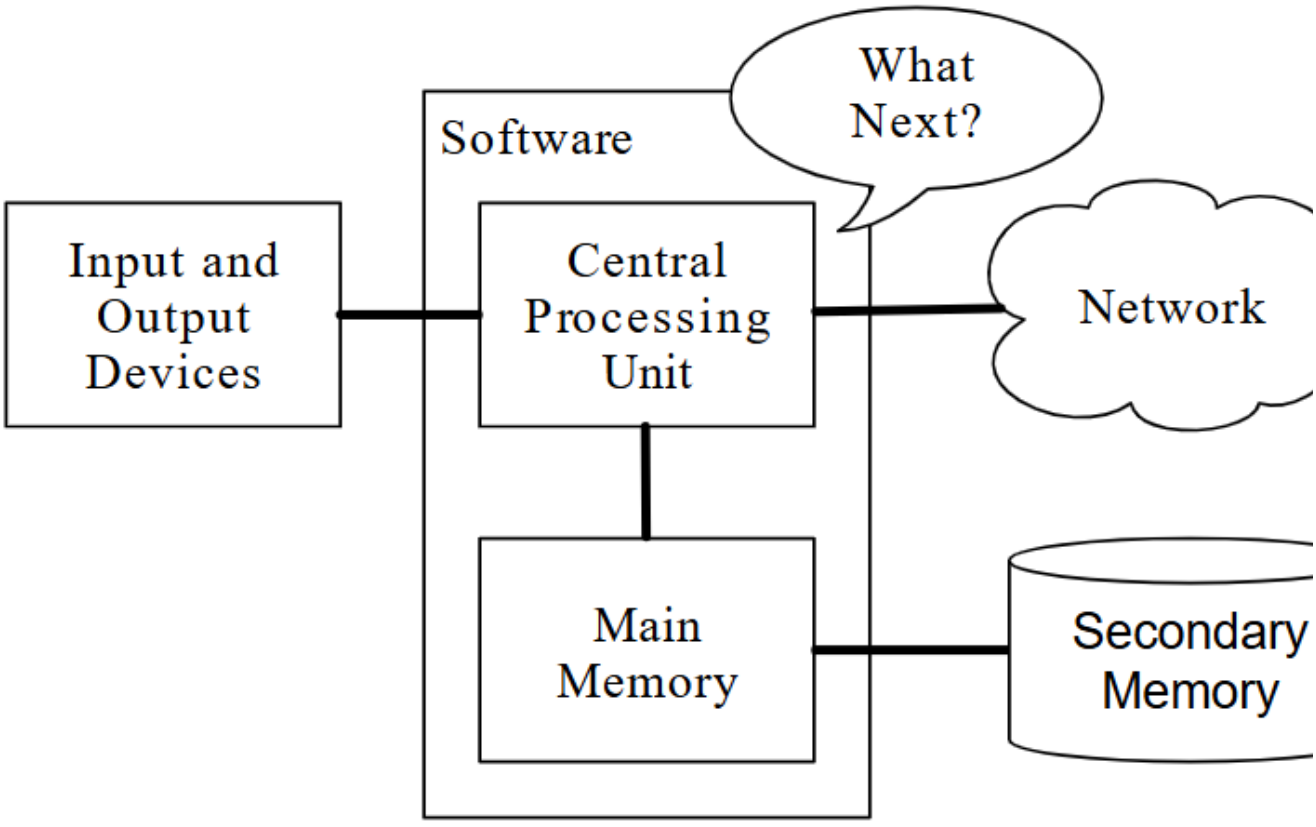



Imagen - Memoria secundaria

En este capítulo, comenzamos a trabajar con **Memoria secundaria** (o archivos). La memoria secundaria no se borra cuando se apaga la alimentación. O en el caso de una unidad flash USB, los

datos que escribimos de nuestros programas se pueden eliminar del sistema y transportar a otro sistema.

Nos centraremos principalmente en leer y escribir archivos de texto como los que creamos en un editor de texto. Más adelante veremos cómo trabajar con archivos de base de datos que son archivos binarios, diseñados específicamente para ser leídos y escritos a través del software de base de datos.

# Apertura de archivos

Cuando queremos leer o escribir un archivo (digamos en su disco duro), primero debemos **abrir** el archivo. Al abrir el archivo se comunica con su sistema operativo, que sabe dónde se almacenan los datos de cada archivo. Cuando abres un archivo, le estás pidiendo al sistema operativo que busque el archivo por su nombre y se asegure de que exista. En este ejemplo, abrimos el archivo `mbox.txt`, que se debe almacenar en la misma carpeta en la que se encuentra cuando inicia Python. Puedes descargar este archivo desde www.py4e.com/code3/mbox.txt

```
>>> fhand = open('mbox.txt')
>>> print(fhand)
<_io.TextIOWrapper name='mbox.txt' mode='r' encoding='cp1252'>
```

Si el `open` tiene éxito, el sistema operativo nos devuelve un **archivo manejador**. El identificador de archivo no es la información real contenida en el archivo, sino que es un "identificador" que podemos usar para leer los datos. Se le otorga un identificador si el archivo solicitado existe y tienes los permisos adecuados para leer el archivo.

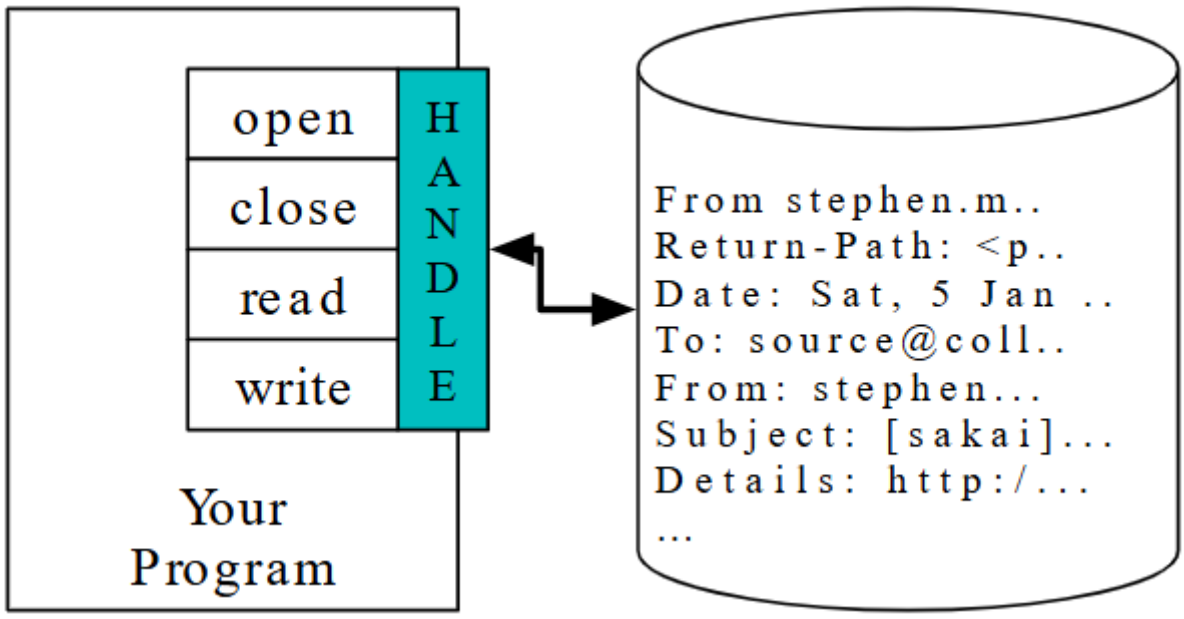


Imagen - A File Handle

Si el archivo no existe, `open` fallará con un rastreo y no obtendrás un identificador para acceder al contenido del archivo:

```
>>> fhand = open('stuff.txt')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: 'stuff.txt'
```

Más adelante usaremos `try` y `except` para lidiar con la situación en la que intentamos abrir un archivo que no existe.

# Archivos y líneas de texto

Un archivo de texto puede considerarse como una secuencia de líneas, al igual que una cadena de Python puede considerarse como una secuencia de caracteres. Por ejemplo, esta es una muestra de un archivo de texto que registra la actividad de correo de varias personas en un equipo de desarrollo de proyectos de código abierto:

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
Return-Path: <postmaster@collab.sakaiproject.org>
Date: Sat, 5 Jan 2008 09:12:18 -0500
To: source@collab.sakaiproject.org
From: stephen.marquard@uct.ac.za
Subject: [sakai] svn commit: r39772 - content/branches/
Details: http://source.sakaiproject.org/viewsvn/?view=rev&amp;rev=39772
...
```

El archivo completo de interacciones de correo está disponible en

www.py4e.com/code3/mbox.txt

y una versión abreviada del archivo está disponible en

www.py4e.com/code3/mbox-short.txt

Estos archivos están en un formato estándar para un archivo que contiene varios mensajes de correo. Las líneas que comienzan con "From" separan los mensajes y las líneas que comienzan con "From:" son parte de los mensajes. Para obtener más información sobre el formato mbox, consulte en.wikipedia.org/wiki/Mbox.

Para dividir el archivo en líneas, hay un carácter especial que representa el "final de la línea" llamado el carácter **nueva línea**.

En Python, representamos el carácter **nueva línea** como una barra invertida-n en constantes de cadena. A pesar de que se ve como dos caracteres, en realidad es un solo caracter. Cuando miramos la variable al ingresar "cosas" en el intérprete, nos muestra el `\n` en la cadena, pero cuando usamos `print` para mostrar la cadena, vemos la cadena dividida en dos líneas por el caracter de **nueva línea**.

```
>>> stuff = 'Hello\nWorld!'
>>> stuff
'Hello\nWorld!'
>>> print(stuff)
Hello
World!
>>> stuff = 'X\nY'
>>> print(stuff)
X
Y
>>> len(stuff)
3
```

También puede ver que la longitud de la cadena `X\nY` es **tres** caracteres porque el carácter de nueva línea es un solo carácter.

Entonces, cuando miramos las líneas en un archivo, necesitamos **imaginarnos** que hay un carácter invisible especial llamado `nueva línea` que marca el final de la línea.

Así que el caracter de nueva línea separa los caracteres del archivo en líneas.

# Leyendo archivos

Mientras que el **manejador de archivo** no contiene los datos para el archivo, es bastante fácil construir un bucle `for` para leer y contar cada una de las líneas en un archivo:

https://trinket.io/embed/python3/971f4a72f3

Podemos usar el identificador de archivo como el iterable en nuestro bucle `for`. Nuestro bucle `for` simplemente cuenta el número de líneas en el archivo y las imprime. La traducción aproximada del bucle `for` al inglés es, "para cada línea en el archivo representado por el identificador de archivo, incrementa la variable `count`.

La razón por la que la función `abrir` no lee el archivo completo es que el archivo puede ser bastante grande con muchos gigabytes de datos. La instrucción `open` toma la misma cantidad de tiempo independientemente del tamaño del archivo. El bucle `for` en realidad hace que los datos se lean del archivo.

Cuando se lee el archivo utilizando un bucle `for` de esta manera, Python se encarga de dividir los datos en el archivo en líneas separadas utilizando el carácter de nueva línea. Python lee cada línea asignándola a la variable `line` en cada iteración del bucle `for`.

Debido a que el bucle `for` lee los datos una línea cada vez, puede leer y contar de manera eficiente las líneas en archivos muy grandes sin quedarse sin memoria principal para almacenar los datos. El programa anterior puede contar las líneas en cualquier archivo de tamaño usando muy poca memoria ya que cada línea se lee, se cuenta y luego se descarta.

Si sabes que el archivo es relativamente pequeño en comparación con el tamaño de tu memoria principal, puedes leer todo el archivo en una cadena usando el método `read` en el identificador de archivo.

```
>>> fhand = open('mbox-short.txt')
>>> inp = fhand.read()
>>> print(len(inp))
94626
>>> print(inp[:20])
From stephen.marquar
```

En este ejemplo, el contenido completo (todos los 94626 caracteres) del archivo `mbox-short.txt` se lee directamente en la variable `inp`. Utilizamos el corte de cadena para imprimir los primeros 20 caracteres de los datos de cadena almacenados en `inp`.

Cuando el archivo se lee de esta manera, todos los caracteres, incluidas todas las líneas y los caracteres de nueva línea, son una cadena grande en la variable **inp**. Recuerda que la función `read` solo debe usarse si los datos del archivo caben cómodamente en la memoria principal de su ordenador.

Si el archivo es demasiado grande para la memoria principal, debes escribir tu programa para leer el archivo en trozos usando un bucle `for` o `while`.

# Buscando a través de un archivo

Cuando buscas datos en un archivo, es un patrón muy común leer un archivo, ignorando la mayoría de las líneas y solo procesando líneas que cumplen con una condición particular. Podemos combinar el patrón para leer un archivo con métodos de cadena para construir mecanismos de búsqueda simples.

Por ejemplo, si quisiéramos leer un archivo y solo imprimir líneas que comenzaran con el prefijo "De:", podríamos usar el método de cadena **startswith** para seleccionar solo aquellas líneas con el prefijo deseado:

https://trinket.io/embed/python3/064568c5b1

Cuando este programa se ejecuta, obtenemos el siguiente resultado:

```
From: stephen.marquard@uct.ac.za

From: louis@media.berkeley.edu

From: zqian@umich.edu

From: rjlowe@iupui.edu
...
```

El resultado se ve muy bien ya que las únicas líneas que estamos viendo son aquellas que comienzan con "De:", pero ¿por qué vemos las líneas en blanco adicionales? Esto se debe a ese carácter invisible **de nueva línea**. Cada una de las líneas termina con una nueva línea, por lo que la instrucción `print` imprime la cadena en la línea de la variable que incluye una nueva línea y luego `print` agrega **otra** nueva línea, resultando en el efecto de doble espaciado.

Podríamos usar el corte de línea para imprimir todo menos el último carácter, pero un enfoque más simple es usar el método **rstrip** que elimina el espacio en blanco del lado derecho de una cadena de la siguiente manera:

https://trinket.io/embed/python3/8436ab534f

Cuando este programa se ejecuta, obtenemos el siguiente resultado:

```
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: cwen@iupui.edu
...
```

A medida que los programas de procesamiento de archivos se vuelven más complicados, es posible que desees estructurar tus bucles de búsqueda utilizando `continue`. La idea básica del bucle de búsqueda es que estás buscando líneas "interesantes" y que omites líneas "no interesantes". Cuando encontramos una línea interesante, hacemos algo con esa línea.

Podemos estructurar el bucle para seguir el patrón de omitir líneas no interesantes como sigue:

https://trinket.io/embed/python3/c175b5a15f

La salida del programa es la misma. En inglés, las líneas no interesantes son aquellas que no comienzan con "From:", que omitimos usando `continue`. Para las líneas "interesantes" (es decir, aquellas que comienzan con "From:") realizamos el procesamiento pertinente.

Podemos usar el método de cadena `find` para simular una búsqueda de editor de texto que encuentra líneas donde la cadena de búsqueda está en cualquier lugar de la línea. Dado que `find` busca una aparición de una cadena dentro de otra cadena y devuelve la posición de la cadena o -1 si no se encontró la cadena, podemos escribir el siguiente bucle para mostrar las líneas que contienen la cadenav "@uct.ac.za" (es decir, provienen de la Universidad de Ciudad del Cabo en Sudáfrica):

https://trinket.io/embed/python3/25e3e59816

Que produce el siguiente resultado:

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
X-Authentication-Warning: set sender to stephen.marquard@uct.ac.za using -f
From: stephen.marquard@uct.ac.za
Author: stephen.marquard@uct.ac.za
From david.horwitz@uct.ac.za Fri Jan  4 07:02:32 2008
X-Authentication-Warning: set sender to david.horwitz@uct.ac.za using -f
From: david.horwitz@uct.ac.za
Author: david.horwitz@uct.ac.za
...
```

# Permitiendo que el usuario elija el nombre del archivo

Realmente no queremos tener que editar nuestro código Python cada vez que deseamos procesar un archivo diferente. Sería más útil pedirle al usuario que ingrese la cadena del nombre del archivo cada vez que se ejecute el programa para que puedan usar nuestro programa en diferentes archivos sin cambiar el código Python.

Esto es bastante simple de hacer leyendo el nombre del archivo del usuario usando `input` de la siguiente manera:

https://trinket.io/embed/python3/e92683e754

Leemos el nombre del archivo del usuario, lo colocamos en una variable llamada `fname` y abrimos ese archivo. Ahora podemos ejecutar el programa repetidamente en diferentes archivos.

```
python search6.py
Enter the file name: mbox.txt
```

```
There were 1797 subject lines in mbox.txt

python search6.py
Enter the file name: mbox-short.txt
There were 27 subject lines in mbox-short.txt
```

Antes de echar un vistazo a la siguiente sección, observa el programa anterior y pregúntate: "¿Qué podría salir mal aquí?" o "¿Qué podría hacer nuestro usuario amigable que haría que nuestro pequeño y agradable programa fallara en su ejecución, mostrándonos no tan geniales a los ojos de nuestros usuarios?"

# Usando `try, except,` y `open`

Te dije que no miraras. Esta es tu última oportunidad.

¿Qué pasa si nuestro usuario escribe algo que no es un nombre de archivo?

```
python search6.py
Enter the file name: missing.txt
Traceback (most recent call last):
  File "search6.py", line 2, in <module>
    fhand = open(fname)
FileNotFoundError: [Errno 2] No such file or directory: 'missing.txt'

python search6.py
Enter the file name: na na boo boo
Traceback (most recent call last):
  File "search6.py", line 2, in <module>
    fhand = open(fname)
FileNotFoundError: [Errno 2] No such file or directory: 'na na boo boo'
```

No te rías. Los usuarios eventualmente harán todo lo posible para romper tus programas. De hecho, una parte importante de cualquier equipo de desarrollo de software es una persona o grupo llamado **Aseguradora de calidad** (o control de calidad para abreviar) cuyo trabajo consiste en hacer las cosas más locas posibles en un intento de romper el software que el programador ha creado.

El equipo de control de calidad es responsable de encontrar las fallas en los programas antes de que entreguemos el programa a los usuarios finales que pueden estar comprando el software o pagando nuestro salario para escribir el software. Así que el equipo de control de calidad es el mejor amigo del programador.

Así que ahora que vemos la falla en el programa, podemos arreglarlo con elegancia usando la estructura `try`/`except`. Debemos suponer que la llamada `open` puede fallar y agregar un código de recuperación cuando el `open` falla de la siguiente manera:

https://trinket.io/embed/python3/ee7849d86b

La función `exit` termina el programa. Es una función que llamamos que nunca vuelve. Ahora, cuando nuestro usuario (o equipo de control de calidad) escribe tonterías o nombres de archivos incorrectos, los "capturamos" y recuperamos con gracia:

```
python search7.py
Enter the file name: mbox.txt
There were 1797 subject lines in mbox.txt

python search7.py
Enter the file name: na na boo boo
File cannot be opened: na na boo boo
```

Proteger la llamada `open` es un buen ejemplo del uso correcto de `try` y `except` en un programa Python. Usamos el término "Pythonic" cuando estamos haciendo algo a la "manera Python". Podríamos decir que el ejemplo anterior es la forma Pythonic de abrir un archivo.

Una vez que adquieras más experiencia en Python, puede comprometerse con otros programadores de Python para decidir cuál de las dos soluciones equivalentes a un problema es "más Pythonic". El objetivo de ser "más Pythonic" capta la noción de que la programación es parte ingeniería y parte arte. No siempre estamos interesados en hacer que algo funcione, también queremos que nuestra solución sea elegante y que nuestros pares la aprecien como elegante.

# Escribiendo archivos

Para escribir un archivo, debes abrirlo con el modo "w" (write) como segundo parámetro:

```
>>> fout = open('output.txt', 'w')
>>> print(fout)
<_io.TextIOWrapper name='output.txt' mode='w' encoding='cp1252'>
```

Si el archivo ya existe, abrirlo en modo de escritura borra los datos antiguos y comienza de nuevo, ¡así que ten cuidado! Si el archivo no existe, se crea uno nuevo.

El método `write` del objeto de identificador de archivo coloca datos en el archivo, devolviendo el número de caracteres escritos. El modo de escritura predeterminado es texto para escribir (y leer) cadenas.

```
>>> line1 = "This here's the wattle,\n"
>>> fout.write(line1)
24
```

Nuevamente, el objeto de archivo mantiene un registro de dónde está, por lo que si llama a `write` de nuevo, agrega los nuevos datos al final.

Debemos asegurarnos de administrar los extremos de las líneas a medida que escribimos en el archivo insertando explícitamente el carácter de nueva línea cuando queremos terminar una línea. La instrucción `print` agrega automáticamente una nueva línea, pero el método `write` no agrega la nueva línea automáticamente.

```
>>> line2 = 'the emblem of our land.\n'
>>> fout.write(line2)
24
```

Cuando hayas terminado de escribir, debes cerrar el archivo para asegurarte de que el último bit de datos se haya escrito físicamente en el disco y que no se pierda si se corta la alimentación.

```
>>> fout.close()
```

Podríamos cerrar los archivos que abrimos para leer también, pero podemos ser un poco descuidados si solo abrimos algunos archivos ya que Python se asegura de que todos los archivos abiertos se cierren cuando finalice el programa. Cuando estamos escribiendo archivos, queremos cerrar explícitamente los archivos para no dejar nada al azar.

# depuración

Cuando estás leyendo y escribiendo archivos, puedes tener problemas con el espacio en blanco. Estos errores pueden ser difíciles de depurar porque los espacios, las pestañas y las nuevas líneas son normalmente invisibles:

```
>>> s = '1 2\t 3\n 4'
>>> print(s)
1 2  3
 4
```

La función incorporada `repr` puede ayudar. Toma cualquier objeto como un argumento y devuelve una representación de cadena del objeto. Para cadenas, representa caracteres de espacio en blanco con secuencias de barra invertida:

```
>>> print(repr(s))
'1 2\t 3\n 4'
```

Esto puede ser útil para la depuración.

Otro problema que podrías encontrar es que los diferentes sistemas usan caracteres diferentes para indicar el final de una línea. Algunos sistemas usan `\n`. Otros utilizan un carácter de retorno, representado como `\r`. Algunos usan ambos. Si mueves archivos entre diferentes sistemas, estas inconsistencias pueden causar problemas.

Para la mayoría de los sistemas, hay aplicaciones para convertir de un formato a otro. Puede encontrarlos (y leer más sobre este problema) en wikipedia.org/wiki/Newline. O, por supuesto, podrías escribir uno tú mismo.

# Ejercicios

**Ejercicio 1** escribe un programa para leer un archivo e imprime el contenido del mismo (línea por línea) todo en mayúsculas. Ejecutando el programa se verá como sigue:

```
python shout.py
Enter a file name: mbox-short.txt
FROM STEPHEN.MARQUARD@UCT.AC.ZA SAT JAN  5 09:14:16 2008
RETURN-PATH: <POSTMASTER@COLLAB.SAKAIPROJECT.ORG>
RECEIVED: FROM MURDER (MAIL.UMICH.EDU [141.211.14.90])
```

```
BY FRANKENSTEIN.MAIL.UMICH.EDU (CYRUS V2.3.8) WITH LMTPA;
SAT, 05 JAN 2008 09:14:16 -0500
```

Puedes descargar el archivo desde

www.py4e.com/code3/mbox-short.txt

**Ejercicio 2** escribe un programa para solicitar un nombre de archivo. Luego lee el archivo y busca las líneas del formulario:

`X-DSPAM-Confidence: 0.8475`

Cuando encuentra una línea que comienza con "X-DSPAM-Confidence:", separa la línea para extraer el número de punto flotante en la línea. Cuenta estas líneas y luego calcula el total de los valores de confianza de correo no deseado de estas líneas. Cuando llegues al final del archivo, imprime la confianza promedio del spam.

```
Enter the file name: mbox.txt
Average spam confidence: 0.894128046745

Enter the file name: mbox-short.txt
Average spam confidence: 0.750718518519
```

Prueba tu programa en los archivos `mbox.txt` y `mbox-short.txt`.

**Ejercicio 3** A veces, cuando los programadores se aburren y quieren divertirse un poco, agregan un **Huevo de Pascua** inofensivo a su programa. Modifica el programa que solicita al usuario el nombre del archivo para que imprima un mensaje divertido cuando el el usuario escriba el nombre exacto del archivo "na na boo boo". El programa debería comportarse normalmente para todos los demás archivos que existen y no existen. Aquí hay una muestra de ejecución del programa:

```
python egg.py
Enter the file name: mbox.txt
There were 1797 subject lines in mbox.txt

python egg.py
Enter the file name: missing.tyxt
File cannot be opened: missing.tyxt
```

```
python egg.py
Enter the file name: na na boo boo
NA NA BOO BOO TO YOU - You have been punk'd!
```

No te alentamos a que pongas Huevos de Pascua en tus programas; esto es solo un ejercicio

# 8 Listas

## Una lista es una secuencia

Como una cadena, una **lista** es una secuencia de valores. En una cadena, los valores son caracteres; en una lista, pueden ser de cualquier tipo. Los valores en la lista se denominan **elementos**.

Hay varias formas de crear una nueva lista; lo más simple es encerrar los elementos entre corchetes (`[` y `]`):

```
[10, 20, 30, 40]
['crunchy frog', 'ram bladder', 'lark vomit']
```

El primer ejemplo es una lista de cuatro enteros. El segundo es una lista de tres cadenas. Los elementos de una lista no tienen que ser del mismo tipo. La siguiente lista contiene una cadena, un decimal, un entero y... ¡otra lista!:

```
['spam', 2.0, 5, [10, 20]]
```

Una lista dentro de otra lista está **anidada**.

Una lista que no contiene elementos se llama una lista vacía; puedes crear una con corchetes vacíos, `[]`.

```
empty_list = []
```

Como es de esperar, puedes asignar valores de lista a variables.

## Las listas son mutables

La sintaxis para acceder a los elementos de una lista es la misma que para acceder a los caracteres de una cadena: el operador de corchete. La expresión dentro de los paréntesis especifica el índice. Recuerda que los índices comienzan por 0:

Copyright 2026

```
>>> print(cheeses[0])
Cheddar
```

A diferencia de las cadenas, las listas son mutables porque puede cambiar el orden de los elementos en una lista o reasignar un elemento en una lista. Cuando el operador de corchetes aparece en el lado izquierdo de una asignación, identifica el elemento de la lista que se asignará.

```
>>> numbers = [17, 123]
>>> numbers[1] = 5
>>> print(numbers)
[17, 5]
>>>
```

El elemento one-eth de `numbers`, que era 123, ahora es 5.

Puede pensar en una lista como una relación entre índices y elementos. Esta relación se llama **mapeo**; cada índice "mapea" a uno de los elementos.

Los índices de lista funcionan de la misma manera que los índices de cadena:

- Cualquier `entero` puede ser usado como un índice.
- Si intentas leer o escribir un elemento que no existe, obtienes un 'IndexError'.
- Si un índice tiene un valor negativo, cuenta hacia atrás desde el final de la lista.

El operador `in` también funciona en listas.

```
>>> cheeses = ['Cheddar', 'Edam', 'Gouda']
>>> 'Edam' in cheeses
True
>>> 'Brie' in cheeses
False
>>>
```

# Atravesando una lista

La forma más común de recorrer los elementos de una lista es con un bucle `for`. La sintaxis es la misma que para las cadenas:

```
for cheese in cheeses:
    print(cheese)
```

Esto funciona bien si solo necesitas leer los elementos de la lista. Pero si deseas escribir o actualizar los elementos, necesitas los índices. Una forma común de hacerlo es combinar las funciones `range` y `len`:

```
for i in range(len(numbers)):
    numbers[i] = numbers[i] * 2
```

Este bucle atraviesa la lista y actualiza cada elemento. `len` devuelve el número de elementos en la lista. `range` devuelve una lista de índices de 0 a **n-1**, donde **n** es la longitud de la lista. Cada vez que pasa por el bucle, `i` obtiene el índice del siguiente elemento. La declaración de asignación en el cuerpo usa `i` para leer el valor antiguo del elemento y para asignar el nuevo valor.

Un bucle `for` sobre una lista vacía nunca ejecuta el cuerpo:

```
for x in empty:
    print('This never happens.')
```

Aunque una lista puede contener otra lista, la lista anidada todavía cuenta como un elemento único. La longitud de esta lista es cuatro:

```
['spam', 1, ['Brie', 'Roquefort', 'Pol le Veq'], [1, 2, 3]]
```

# Operaciones de lista

El operador `+` concatena las listas:

```
>>> a = [1, 2, 3]
>>> b = [4, 5, 6]
>>> c = a + b
>>> print(c)
[1, 2, 3, 4, 5, 6]
>>>
```

Del mismo modo, el operador `*` repite una lista un número determinado de veces:

```
>>> [0] * 4
[0, 0, 0, 0]
>>> [1, 2, 3] * 3
[1, 2, 3, 1, 2, 3, 1, 2, 3]
>>>
```

El primer ejemplo se repite cuatro veces. El segundo ejemplo repite la lista tres veces.

# Listar segmentos

El operador de corte también funciona en listas:

```
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> t[1:3]
['b', 'c']
>>> t[:4]
['a', 'b', 'c', 'd']
>>> t[3:]
['d', 'e', 'f']
>>>
```

Si omites el primer índice, la división comienza al principio. Si omites el segundo, la rebanada va al final. Entonces, si omite ambos, la porción es una copia de toda la lista.

```
>>> t[:]
['a', 'b', 'c', 'd', 'e', 'f']
```

Dado que las listas son mutables, a menudo es útil hacer una copia antes de realizar operaciones de plegado, huso o mutilación de listas.

Un operador de sector en el lado izquierdo de una asignación puede actualizar varios elementos:

```
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> t[1:3] = ['x', 'y']
>>> print(t)
['a', 'x', 'y', 'd', 'e', 'f']
```

Copyright 2026

```
>>>
```

# Métodos de lista

Python proporciona métodos que operan en listas. Por ejemplo, `append` agrega un nuevo elemento al final de una lista:

```
>>> t = ['a', 'b', 'c']
>>> t.append('d')
>>> print(t)
['a', 'b', 'c', 'd']
```

`extend` toma una lista como argumento y agrega todos los elementos:

```
>>> t = ['a', 'b', 'c']
>>> t.extend(['d', 'f'])
>>> print(t)
['a', 'b', 'c', 'd', 'f']
```

Este ejemplo deja `t2` sin modificar.

`sort` organiza los elementos de la lista de bajo a alto:

```
>>> t = ['d', 'c', 'e', 'b', 'a']
>>> t.sort()
>>> print(t)
['a', 'b', 'c', 'd', 'e']
```

La mayoría de los métodos de lista son nulos; modifican la lista y devuelven `None`. Si accidentalmente escribes `t = t.sort ()`, quedarás decepcionado con el resultado.

# Eliminando elementos

Hay varias formas de eliminar elementos de una lista. Si conoces el índice del elemento que deseas, puede usar `pop`:

```
>>> t = ['a', 'b', 'c']
>>> x = t.pop(1)
>>> print(t)
['a', 'c']
>>> print(x)
b
>>>
```

`pop` modifica la lista y devuelve el elemento que fue eliminado. Si no proporcionas un índice, elimina y devuelve el último elemento.

Si no necesitas el valor eliminado, puede usar el operador `del`:

```
>>> t = ['a', 'b', 'c']
>>> del t[1]
>>> print(t)
['a', 'c']
>>>
```

Si conoces el elemento que deseas eliminar (pero no el índice), puedes usar `remove`:

```
>>> t = ['a', 'b', 'c']
>>> t.remove('b')
>>> print(t)
['a', 'c']
>>>
```

El valor de retorno de `remove` es `None`.

Para eliminar más de un elemento, puedes usar `del` con un índice de división:

```
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> del t[1:5]
>>> print(t)
['a', 'f']
>>>
```

Copyright 2026

Como es habitual, la división selecciona todos los elementos hasta el segundo índice, pero sin incluirlo.

# Listas y funciones

Hay una serie de funciones incorporadas que se pueden usar en listas que te permiten mirar rápidamente una lista sin escribir sus propios bucles:

```
>>> nums = [3, 41, 12, 9, 74, 15]
>>> print(len(nums))
6
>>> print(max(nums))
74
>>> print(min(nums))
3
>>> print(sum(nums))
154
>>> print(sum(nums)/len(nums))
25.666666666666668
```

La función `sum()` solo funciona cuando los elementos de la lista son números. Las otras funciones ( `max()`, `len()`, etc.) funcionan con listas de cadenas y otros tipos que pueden ser comparables.

Podríamos reescribir un programa anterior que calculó el promedio de una lista de números ingresados por el usuario usando una lista.

Primero, el programa para calcular un promedio sin una lista:

https://trinket.io/embed/python3/e02540c91c

En este programa, tenemos las variables `count` y `total` para mantener el número y el total acumulado de los números del usuario, ya que repetidamente pedimos al usuario un número.

Simplemente podríamos recordar cada número cuando el usuario lo ingresó y usar funciones integradas para calcular la suma y el conteo al final.

https://trinket.io/embed/python3/d5f9bf95ff

Hacemos una lista vacía antes de que comience el ciclo, y luego cada vez que tenemos un número, lo agregamos a la lista. Al final del programa, simplemente calculamos la suma de los números en la lista y la dividimos por el recuento de los números en la lista para obtener el promedio.

# Listas y cadenas

Una cadena es una secuencia de caracteres y una lista es una secuencia de valores, pero una lista de caracteres no es lo mismo que una cadena. Para convertir de una cadena a una lista de caracteres, puedes usar `list`:

```
>>> s = 'spam'
>>> t = list(s)
>>> print(t)
['s', 'p', 'a', 'm']
```

Como `list` es el nombre de una función incorporada, debes evitar usarla como nombre de variable. También evito la letra `l` porque se parece demasiado al número `1`. Así que por eso uso `t`.

La función `list` rompe una cadena en letras individuales. Si deseas dividir una cadena en palabras, puede usar el método `split`:

```
>>> s = 'pining for the fjords'
>>> t = s.split()
>>> print(t)
['pining', 'for', 'the', 'fjords']
>>> print(t[2])
the
```

Una vez que hayas usado `split` para dividir la cadena en una lista de palabras, puedes usar el operador de índice (corchete) para mirar una palabra en particular en la lista.

Puedes llamar a `split` con un argumento opcional llamado **dellimiter** que especifica qué caracteres usar como límites de palabras. El siguiente ejemplo usa un guión como delimitador:

```
>>> s = 'spam-spam-spam'
>>> delimiter = '-'
>>> s.split(delimiter)
['spam', 'spam', 'spam']
>>>
```

`join` es el inverso de `split`. Toma una lista de cadenas y concatena los elementos. `join` es un método de cadena, por lo que debes invocarlo en el delimitador y pasar la lista como parámetro:

```
>>> t = ['pining', 'for', 'the', 'fjords']
>>> delimiter = ' '
>>> delimiter.join(t)
'pining for the fjords'
```

En este caso, el delimitador es un carácter de espacio, por lo que `join` pone un espacio entre las palabras. Para concatenar cadenas sin espacios, puedes usar la cadena vacía "" como un delimitador.

# Parseando líneas

Por lo general, cuando estamos leyendo un archivo queremos hacer algo en las líneas que no sea simplemente imprimir toda la línea. A menudo queremos encontrar las "líneas interesantes" y luego **analizar** la línea para encontrar algunas **partes** interesantes de la línea. ¿Qué pasaría si quisiéramos imprimir el día de la semana desde esas líneas que comienzan con "From"?

`From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008`

El método de `split` es muy efectivo cuando se enfrenta a este tipo de problemas. Podemos escribir un pequeño programa que busque líneas donde la línea comience con "From", `dividir` esas líneas, y luego imprimir la tercera palabra en la línea:

https://trinket.io/embed/python3/106858f1a8

Aquí también usamos la forma contraída de la declaración `if` donde colocamos el `continue` en la misma línea que el `if`. Esta forma contraída de `if` funciona de la misma manera que si `continue` estuviera en la siguiente línea y con sangría.

El programa produce la siguiente salida:

```
Sat
Fri
Fri
Fri
    ...
```

Más adelante, aprenderemos técnicas cada vez más sofisticadas para elegir las líneas en las que trabajar y cómo separamos esas líneas para encontrar la información exacta que estamos buscando.

# Objetos y valores

Si ejecutamos estas sentencias de asignación:

```
a = 'banana'
b = 'banana'
```

sabemos que `a` y `b` se refieren a una cadena, pero no sabemos si se refieren a la *misma* cadena. Hay dos estados posibles:




Imagen - Variables y objetos

En un caso, `a` y `b` se refieren a dos objetos diferentes que tienen el mismo valor. En el segundo caso, se refieren al mismo objeto.

Para verificar si dos variables se refieren al mismo objeto, puede usar el operador `is`.

```
>>> a = 'banana'
>>> b = 'banana'
>>> a is b
True
```

En este ejemplo, Python solo creó un objeto de cadena, y tanto `a` como `b` se refieren a él.

Pero cuando creas dos listas, obtienes dos objetos:

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a is b
False
>>>
```

En este caso, diríamos que las dos listas son **equivalentes**, porque tienen los mismos elementos, pero no **idénticas**, porque no son el mismo objeto. Si dos objetos son idénticos, también son equivalentes, pero si son equivalentes, no son necesariamente idénticos.

Hasta ahora, hemos estado utilizando indistintamente "objeto" y "valor", pero es más preciso decir que un objeto tiene un valor. Si ejecuta `a = [1,2,3]`, `a` se refiere a un objeto de lista cuyo valor es una secuencia particular de elementos. Si otra lista tiene los mismos elementos, diríamos que tiene el mismo valor.

# Aliasing

Si `a` se refiere a un objeto y usted asigna `b = a`, entonces ambas variables se refieren al mismo objeto:

```
>>> a = [1, 2, 3]
>>> b = a
>>> b
[1, 2, 3]
>>> a[2] = 4
>>> b
[1, 2, 4]
```

La asociación de una variable con un objeto se denomina **referencia**. En este ejemplo, hay dos referencias al mismo objeto.

Un objeto con más de una referencia tiene más de un nombre, por lo que decimos que el objeto tiene un **alias**.

Copyright 2026

Si el objeto con alias es mutable, los cambios realizados con un alias afectan al otro:

```
>>> b[0] = 17
>>> print(a)
[17, 2, 3]
```

Aunque este comportamiento puede ser útil, es propenso a errores. En general, es más seguro evitar los alias cuando trabaja con objetos mutables.

Para objetos inmutables como cadenas, el aliasing no es tan problemático.

# Lista de argumentos

Cuando pasas una lista a una función, la función obtiene una referencia a la lista. Si la función modifica un parámetro de lista, la función ve el cambio. Por ejemplo, `delete_head` elimina el primer elemento de una lista:

```
def delete_head(t):
    del t[0]
```

Así es como se usa:

```
>>> letters = ['a', 'b', 'c']
>>> delete_head(letters)
>>> print(letters)
['b', 'c']
>>>
```

El parámetro `t` y la variable `letters` son alias para el mismo objeto.

Es importante distinguir entre operaciones que modifican listas y operaciones que crean nuevas listas. Por ejemplo, el método `append` modifica una lista, pero el operador `+` crea una nueva lista:

```
>>> t1 = [1, 2]
>>> t2 = t1.append(3)
>>> print(t1)
[1, 2, 3]
```

Copyright 2026

```
>>> print(t2)
None
>>> t3 = t1 + [3]
>>> print(t3)
[1, 2, 3, 3]
>>> t2 is t3
False
>>>
```

Esta diferencia es importante cuando escribes funciones que se supone que modifican las listas. Por ejemplo, esta función **no** elimina el encabezado de una lista:

```
def bad_delete_head(t):
    t = t[1:]              # WRONG!
```

El operador de sector crea una nueva lista y la asignación hace que `t` se refiera a ella, pero nada de eso tiene ningún efecto en la lista que se pasó como argumento.

Una alternativa es escribir una función que crea y devuelve una nueva lista. Por ejemplo, `tail` devuelve todos menos el primer elemento de una lista:

```
def tail(t):
    return t[1:]
```

Esta función deja la lista original sin modificar. Así es como se usa:

```
>>> letters = ['a', 'b', 'c']
>>> rest = tail(letters)
>>> print(rest)
['b','c']
```

**Ejercicio 1**:

Escriba una función llamada `chop` que tome una lista y la modifique, eliminando el primer y último elemento y devuelva `None`.

Luego escribe una función llamada `middle` que tome una lista y devuelva una nueva lista que contiene todos los elementos, excepto el primero y el último.

# Depuración

El uso descuidado de listas (y otros objetos mutables) puede llevar a largas horas de depuración. Aquí hay algunos escollos comunes y maneras de evitarlos:

> No olvides que la mayoría de los métodos de lista modifican el objeto y devuelven `None`. Esto es lo contrario de los métodos de cadena, que devuelven una cadena nueva y dejan el original inalterado.

Si estás acostumbrado a escribir un código de cadena como este:

```
word = word.strip()
```

Es tentador escribir un código de lista como este:

```
t = t.sort () # WRONG!
```

Debido a que `sort` devuelve `None`, la siguiente operación que realice con `t` es probable que falle.

Antes de utilizar los métodos de lista y los operadores, debes leer la documentación detenidamente y luego probarlos en modo interactivo. Los métodos y operadores que las listas comparten con otras secuencias (como cadenas) se documentan en https://docs.python.org/3.7/library/stdtypes.html#string-methods. Los métodos y operadores que solo se aplican a secuencias mutables están documentados en https://docs.python.org/3.7/library/stdtypes.html#mutable-sequence-types.

> Escoge una sintaxis y quédate con ella.
> Parte del problema con las listas es que hay demasiadas formas de hacer las cosas. Por ejemplo, para eliminar un elemento de una lista, puedes usar `pop`, `remove`, `del`, o incluso una asignación de sector.

Para agregar un elemento, puede usar el método `append` o el operador `+`. Pero no olvides que estos son correctos:

```
    t.append(x)
    t = t + [x]
```

Copyright 2026

Y estos son erróneos:

```
t.append([x])          # WRONG!
t = t.append(x)        # WRONG!
t + [x]                # WRONG!
t = t + x              # WRONG!
```

Prueba cada uno de estos ejemplos en modo interactivo para asegurartse de que comprendes lo que hacen. Observa que solo el último causa un error en tiempo de ejecución; los otros tres son legales, pero no hacen lo que queremos.

> Haz copias para evitar el aliasing.
> Si deseas utilizar un método como `sort` que modifica el objeto, pero también necesita conservar la lista original, puede hacer una copia.

```
orig = t[:]
t.sort()
```

En este ejemplo, también podrías usar la función incorporada `sorted`, que devuelve una nueva lista ordenada y deja el original inalterado. ¡Pero en ese caso, debes evitar usar `sorted` como nombre de variable!

> Listas, `split` y archivos
> Cuando leemos y analizamos archivos, hay muchas oportunidades de encontrar información que puede bloquear nuestro programa, por lo que es una buena idea revisar el patrón **guardian** cuando se trata de escribir programas que leen un archivo y buscan una "aguja en el pajar".

Revisemos nuestro programa que busca el día de la semana en las líneas de nuestro archivo:

`From stephen.marquard@uct.ac.zaSatJan 5 09:14:16 2008`

Ya que estamos dividiendo esta línea en palabras, podríamos prescindir del uso de `startswith` y simplemente mirar la primera palabra de la línea para determinar si estamos interesados en ella. Podemos usar `continue` para omitir las líneas que no tienen "From" como la primera palabra de la siguiente manera:

```
fhand = open('mbox-short.txt')
for line in fhand:
    words = line.split()
    if words[0] != 'From' : continue
    print(words[2])
```

Esto parece mucho más simple y ni siquiera necesitamos hacer el `rstrip` para eliminar la nueva línea al final del archivo. Pero, ¿es mejor?

```
python search8.py
Sat
Traceback (most recent call last):
    File "search8.py", line 5, in <module>
    if words[0] != 'From' : continue
IndexError: list index out of range
```

Funciona y vemos el día desde la primera línea (Sat), pero luego el programa falla con un error de rastreo. ¿Qué salió mal? ¿Qué datos desordenados hicieron que nuestro programa elegante, inteligente y muy Pythonic fallara?

Puedes mirarlo fijamente durante un largo tiempo y resolverlo o pedirle ayuda a alguien, pero el enfoque más rápido e inteligente es agregar una declaración `print()`. El mejor lugar para agregar la declaración de impresión es justo antes de la línea donde falló el programa e imprimir los datos que parecen estar causando la falla.

Ahora este enfoque puede generar una gran cantidad de líneas de salida, pero al menos tendrás inmediatamente alguna pista sobre el problema en cuestión. Así que agregamos una impresión de la variable `words` justo antes de la línea cinco. Incluso agregamos un prefijo "Debug:" a la línea para que podamos mantener nuestra salida regular separada de nuestra salida de depuración.

```
for line in fhand:
    words = line.split()
    print('Debug:', words)
    if words[0] != 'From' : continue
    print(words[2])
```

Cuando ejecutamos el programa, una gran cantidad de salida se desplaza fuera de la pantalla, pero al final, vemos nuestra salida de depuración y el rastreo, por lo que sabemos lo que sucedió justo antes del rastreo.

```
Debug: ['X-DSPAM-Confidence:', '0.8475']
Debug: ['X-DSPAM-Probability:', '0.0000']
Debug: []
Traceback (most recent call last):
    File "search9.py", line 6, in <module>
    if words[0] != 'From' : continue
IndexError: list index out of range
```

Cada línea de depuración está imprimiendo la lista de palabras que obtenemos cuando "dividimos" la línea en palabras. Cuando el programa falla, la lista de palabras está vacía `[]`. Si abrimos el archivo en un editor de texto y miramos el archivo, en ese punto se ve como sigue:

```
X-DSPAM-Result: Innocent
X-DSPAM-Processed: Sat Jan  5 09:14:16 2008
X-DSPAM-Confidence: 0.8475
X-DSPAM-Probability: 0.0000

Details: http://source.sakaiproject.org/viewsvn/?view=rev&rev=39772
```

¡El error ocurre cuando nuestro programa encuentra una línea en blanco! Por supuesto, hay "cero palabras" en una línea en blanco. ¿Por qué no pensamos en eso cuando estábamos escribiendo el código? Cuando el código busca la primera palabra (`words[0]`) para verificar si coincide con "From", obtenemos un error de "índice fuera de rango".

Por supuesto, este es el lugar perfecto para agregar un código **guardian** para evitar marcar la primera palabra si la primera palabra no está allí. Hay muchas maneras de proteger este código. Elegiremos verificar el número de palabras que tenemos antes de mirar la primera palabra:

```
fhand = open('mbox-short.txt')
count = 0
for line in fhand:
    words = line.split()
    # print 'Debug:', words
    if len(words) == 0 : continue
    if words[0] != 'From' : continue
    print(words[2])
```

Primero comentamos la declaración de impresión de depuración en lugar de eliminarla, en caso de que nuestra modificación falle y tengamos que volver a depurar. Luego agregamos una declaración que verifique si tenemos cero palabras y, si es así, usamos `continue` para saltar a la siguiente línea del archivo.

Podemos pensar que las dos afirmaciones "continue" nos ayudan a refinar el conjunto de líneas que son "interesantes" para nosotros y que queremos procesar un poco más. Una línea que no tiene palabras es "poco interesante" para nosotros, así que saltamos a la siguiente línea. Una línea que no tiene "From" como su primera palabra no nos interesa, así que la omitimos.

El programa modificado se ejecuta con éxito, así que quizás sea correcto. Nuestra declaración guardiana asegura que `words[0]` nunca fallarán, pero tal vez no sea suficiente. Cuando estamos programando, siempre debemos pensar: "¿Qué podría salir mal?"

# Ejercicios

**Ejercicio 2**: Averigua qué línea del programa anterior aún no está correctamente protegida. Intenta construir un archivo de texto que haga que el programa falle y luego modifícalo para que la línea esté bien protegida y pruébalo para asegurarte de que manejas tu nuevo archivo de texto.

**Ejercicio 3**: reescribe el código guardián en el ejemplo anterior sin dos declaraciones 'if'. En su lugar, usa una expresión lógica compuesta utilizando el operador lógico `and` con una sola instrucción `if`.

**Ejercicio 4**: descarga una copia del archivo desde http://www.py4e.com/code3/romeo.txt

Escribe un programa para abrir el archivo `romeo.txt` y léelo línea por línea. Para cada línea, divide la línea en una lista de palabras usando la función `split`.

Para cada palabra, verifica si la palabra ya está en una lista. Si la palabra no está en la lista, agrégala a la lista.

Cuando se complete el programa, ordena e imprime las palabras resultantes en orden alfabético.

```Enter file: romeo.txt ['Arise', 'But', 'It', 'Juliet', 'Who', 'already', 'and', 'breaks', 'east', 'envious', 'fair', 'grief', 'is', 'kill', 'light', 'moon', 'pale', 'sick', 'soft', 'sun', 'the', 'through', 'what', 'window', 'with', 'yonder']
```

```
**Ejercicio 5:** Escribe un programa para leer los datos del buzón de correo y cuando
```

encuentre la línea que comienza con "From", dividirá la línea en palabras usando la función `split`. Estamos interesados en quién envió el mensaje, que es la segunda palabra en la línea From.

`De stephen.marquard@uct.ac.za sábado 5 de enero 09:14:16 2008`

Analizará la línea Desde e imprimirá la segunda palabra para cada línea From, también contará el número de líneas From (no From:) e imprimirá un recuento al final.

Esta es una buena salida de muestra con algunas líneas eliminadas:

```python
python fromcount.py
Enter a file name: mbox-short.txt
stephen.marquard@uct.ac.za
louis@media.berkeley.edu
zqian@umich.edu

[...some output removed...]

ray@media.berkeley.edu
cwen@iupui.edu
cwen@iupui.edu
cwen@iupui.edu
There were 27 lines in the file with From as the first word
```

**Ejercicio 6:** Reescribe el programa que solicita al usuario una lista de números e imprime el máximo y el mínimo de los números al final cuando el usuario ingresa "listo". Escribe el programa para almacenar los números que el usuario ingresa en una lista y use las funciones `max()` y `min()` para calcular los números máximo y mínimo después de que se complete el ciclo.

```
Enter a number: 6
Enter a number: 2
Enter a number: 9
Enter a number: 3
Enter a number: 5
Enter a number: done
```

```
Maximum: 9.0
Minimum: 2.0
```

Copyright 2026

# 9 Diccionarios

Un **diccionario** es como una lista, pero más general. En una lista, las posiciones del índice deben ser enteros; En un diccionario, los índices pueden ser (casi) de cualquier tipo.

Puedes pensar en un diccionario como una asignación entre un conjunto de índices (que se denominan **claves**) y un conjunto de valores. Cada valor se asigna a una clave. La asociación de una clave y un valor se denomina **par clave-valor** o, a veces, un **elemento**.

Como ejemplo, construiremos un diccionario que asigne palabras del inglés al español, por lo que las claves y los valores son todas cadenas.

La función `dict` crea un nuevo diccionario sin elementos. Como `dict` es el nombre de una función incorporada, debes evitar usarla como nombre de variable.

```
named_variable = dict()
# También puedes crear un diccionario de la siguiente manera
named_variable_2 = {}
```

Las llaves, `{}`, representan un diccionario vacío. Para agregar elementos al diccionario, puede usar corchetes:

```
>>> eng2sp['one'] = 'uno'
```

Esta línea crea un elemento que se asigna desde la clave `'one'` al valor "uno". Si imprimimos el diccionario nuevamente, vemos un par clave-valor con dos puntos entre la clave y el valor:

```
>>> print(eng2sp)
{'one': 'uno'}
```

Este formato de salida es también un formato de entrada. Por ejemplo, puedes crear un diccionario nuevo con tres elementos. Pero si imprimes `eng2sp`, podría sorprenderse:

```
>>> eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> print(eng2sp)
{'one': 'uno', 'three': 'tres', 'two': 'dos'}
```

Copyright 2026

El orden de los pares clave-valor no es el mismo. De hecho, si escribes el mismo ejemplo en tu ordenador, puedes obtener un resultado diferente. En general, el orden de los elementos en un diccionario es impredecible.

Pero eso no es un problema porque los elementos de un diccionario nunca se indexan con índices enteros. En su lugar, utiliza las claves para buscar los valores correspondientes:

```
>>> print(eng2sp['two'])
'dos'
```

La clave `'two'` siempre contiene el valor "dos", por lo que el orden de los elementos no importa.

Si la clave no está en el diccionario, obtendrás una excepción:

```
>>> print(eng2sp['four'])
KeyError: 'four'
```

La función `len` funciona en los diccionarios; devuelve el número de pares clave-valor:

```
>>> len(eng2sp)
3
```

El operador `in` funciona también en los diccionarios; le indica si algo aparece como una **clave** en el diccionario.

```
>>> 'one' in eng2sp
True
>>> 'uno' in eng2sp
False
```

Para ver si algo aparece como un valor en un diccionario, puede usar el método `values()`, que devuelve los valores como una lista, y luego usar el operador `in`:

```
>>> 'uno' in eng2sp.values()
True
```

El operador `in` usa diferentes algoritmos para listas y diccionarios. Para listas, utiliza un algoritmo de búsqueda lineal. A medida que la lista se alarga, el tiempo de búsqueda se alarga en proporción directa a la longitud de la lista. Para los diccionarios, Python usa un algoritmo llamado **tabla hash**

que tiene una propiedad notable: el operador `in` toma aproximadamente la misma cantidad de tiempo, sin importar cuántos elementos haya en un diccionario. No explicaré por qué las funciones hash son tan mágicas, pero puedes leer más sobre esto en wikipedia.org/wiki/Hash_table.

**Ejercicio 1** [lista de palabras 2]

Escribe un programa que lea las palabras en `words.txt` y las almacene como claves en un diccionario. No importa cuáles sean los valores. Luego, puedes usar el operador `in` como una forma rápida de verificar si una cadena está en el diccionario.

# Diccionario como conjunto de contadores

Supongamos que se le asigna una cadena y desea contar cuántas veces aparece cada letra. Hay varias formas de hacerlo:

1. Podrías crear 27 variables, una para cada letra del alfabeto. Luego, podrías atravesar la cadena y, para cada caracter, incrementar el contador correspondiente, probablemente utilizando un condicional encadenado.
2. Podrías crear una lista con 27 elementos. Luego, puedes convertir cada carácter en un número (utilizando la función incorporada `ord`), usar el número como un índice en la lista e incrementar el contador apropiado.
3. Podrías crear un diccionario con caracteres como claves y contadores como los valores correspondientes. La primera vez que veas un caracter, agregarás un elemento al diccionario. Después de eso, incrementarías el valor de un artículo existente.

Cada una de estas opciones realiza el mismo cálculo, pero cada una de ellas implementa ese cálculo de una manera diferente.

Una **implementación** es una forma de realizar un cálculo. Algunas implementaciones son mejores que otras. Por ejemplo, una ventaja de la implementación del diccionario es que no tenemos que saber con antelación qué letras aparecen en la cadena y solo tenemos que crear espacio para las letras que van apareciendo.

Así es cómo se vería el código:

Estamos efectivamente calculando un **histograma** , que es un término estadístico para un conjunto de contadores (o frecuencias).

```
word = 'brontosaurus'
d = {}
for c in word:
  if c not in d:
    d[c] = 1
  else:
    d[c] = d[c] + 1
print(d)
```

El bucle `for` atraviesa la cadena. Cada vez que recorre el bucle, si el caracter `c` no está en el diccionario, creamos un nuevo elemento con la clave `c` y el valor inicial 1 (ya que hemos visto esta letra una vez). Si `c` ya está en el diccionario, incrementamos `d[c]`.

Aquí está la salida del programa:

```
{'a': 1, 'b': 1, 'o': 2, 'n': 1, 's': 2, 'r': 2, 'u': 2, 't': 1}
```

El histograma indica que las letras `'a'` y `'b'` aparecen una vez; "O" aparece dos veces, y así sucesivamente.

Los diccionarios tienen un método llamado `get` que toma una clave y un valor predeterminado. Si la clave aparece en el diccionario, `get` devuelve el valor correspondiente; de lo contrario, devuelve el valor predeterminado. Por ejemplo:

```
>>> counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
>>> print(counts.get('jan', 0))
100
>>> print(counts.get('tim', 0))
0
```

Podemos usar `get` para escribir nuestro bucle de histograma de forma más concisa. Debido a que el método `get` maneja automáticamente el caso en el que una clave no esté en un diccionario, podemos reducir cuatro líneas a una y eliminar la instrucción `if`.

```
word = 'brontosaurus'
d = dict()
for c in word:
    d[c] = d.get(c,0) + 1
```

```
print(d)
```

El uso del método `get` para simplificar este bucle de conteo termina siendo un "idiom" muy usado en Python y lo usaremos muchas veces en el resto del libro. Así que debes tomarte un momento y comparar el bucle usando la instrucción `if` y el operador `in` con el bucle usando el método `get`. Hacen exactamente lo mismo, pero uno es más sucinto.

# Diccionarios y archivos

Uno de los usos comunes de un diccionario es contar la aparición de palabras en un archivo con algún texto escrito. Comencemos con un archivo muy simple de palabras tomadas del texto de **Romeo y Julieta**.

Para el primer conjunto de ejemplos, usaremos una versión abreviada y simplificada del texto sin puntuación. Posteriormente trabajaremos con el texto de la escena con puntuación incluida.

```
But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

Escribiremos un programa en Python para leer las líneas del archivo, dividir cada línea en una lista de palabras y luego recorrer cada una de las palabras en la línea y contar cada palabra con un diccionario.

Verás que tenemos dos bucles `for`. El bucle externo está leyendo las líneas del archivo y el bucle interno está iterando a través de cada una de las palabras en esa línea en particular. Este es un ejemplo de un patrón llamado **bucles anidados** porque uno de los bucles es el bucle **externo** y el otro bucle es el bucle **interno**.

Debido a que el bucle interno ejecuta todas sus iteraciones cada vez que el bucle externo realiza una única iteración, pensamos que el bucle interno se repite "más rápidamente" y que el bucle externo se repite más lentamente.

La combinación de los dos bucles anidados asegura que contaremos cada palabra en cada línea del archivo de entrada.

https://trinket.io/embed/python3/17a4334148

Cuando ejecutamos el programa, vemos un volcado sin formato de todos los recuentos en orden hash. (El archivo `romeo.txt` está disponible en www.py4e.com/code3/romeo.txt)

```
python count1.py
Enter the file name: romeo.txt
{'and': 3, 'envious': 1, 'already': 1, 'fair': 1,
'is': 3, 'through': 1, 'pale': 1, 'yonder': 1,
'what': 1, 'sun': 2, 'Who': 1, 'But': 1, 'moon': 1,
'window': 1, 'sick': 1, 'east': 1, 'breaks': 1,
'grief': 1, 'with': 1, 'light': 1, 'It': 1, 'Arise': 1,
'kill': 1, 'the': 3, 'soft': 1, 'Juliet': 1}
```

Es un poco incómodo mirar en el diccionario para encontrar las palabras más comunes y sus conteos, por lo que necesitamos agregar un poco más de código de Python para obtener el resultado que será más útil.

# Bucles y diccionarios

Si utilizas un diccionario como la secuencia en una instrucción `for`, atraviesa las claves del diccionario. Este bucle imprime cada clave y el valor correspondiente:

```
counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
for key in counts:
    print(key, counts[key])
```

Así es como se ve la salida:

```
jan 100
chuck 1
annie 42
```

Una vez más, las claves no están en ningún orden en particular.

Podemos usar este patrón para implementar los diversos modismos de bucle que hemos descrito anteriormente. Por ejemplo, si quisiéramos encontrar todas las entradas en un diccionario con un

valor superior a diez, podríamos escribir el siguiente código:

```
counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
for key in counts:
    if counts[key] > 10 :
        print(key, counts[key])
```

El bucle `for` recorre las **claves** del diccionario, por lo que debemos utilizar el operador de índice para recuperar el valor correspondiente para cada clave. Así es como se ve la salida:

```
jan 100
annie 42
```

Solo vemos las entradas con un valor superior a 10.

Si deseas imprimir las claves en orden alfabético, primero haz una lista de las claves en el diccionario usando el método `keys` disponible en los objetos de tipo diccionario, y luego ordena esa lista y recorre la lista ordenada, mirando cada clave e imprimiendo pares clave-valor ordenados de la siguiente manera:

```
counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
lst = list(counts.keys())
print(lst)
lst.sort()
for key in lst:
    print(key, counts[key])
```

Así es como se ve la salida:

```
['jan', 'chuck', 'annie']
annie 42
chuck 1
jan 100
```

Primero ve la lista desordenada de claves que obtenemos del método `keys`. Luego vemos los pares clave-valor en orden desde el bucle `for`.

Copyright 2026

# Análisis de texto avanzado

En el ejemplo anterior usando el archivo `romeo.txt`, hicimos el archivo lo más simple posible eliminando toda puntuación a mano. El texto real tiene mucha puntuación, como se muestra a continuación.

```
But, soft! what light through yonder window breaks?
It is the east, and Juliet is the sun.
Arise, fair sun, and kill the envious moon,
Who is already sick and pale with grief,
```

Ya que la función `split` de Python busca espacios y trata las palabras como tokens separados por espacios, trataríamos las palabras "soft!" y "soft" como **diferentes** palabras y crearíamos una entrada de diccionario separada para cada palabra.

Además, dado que el archivo tiene mayúsculas, trataríamos "who" y "Who" como palabras diferentes con diferentes valores.

Podemos resolver ambos problemas utilizando los métodos de cadena `lower`, `punctuation` y `translate`. `translate` es el más sutil de los métodos. Aquí está la documentación para `translate`:

`line.translate(str.maketrans(fromstr, tostr, deletestr))`

*Reemplaza los caracteres en `fromstr` con el carácter en la misma posición en `tostr` y borra todos los caracteres que estén en `deletestr`. `Fromstr` y `tostr` pueden ser cadenas vacías y el parámetro `deletestr` puede omitirse.*

No especificaremos la 'tabla', pero usaremos el parámetro `deletechars` para eliminar toda la puntuación. Incluso dejaremos que Python nos diga la lista de caracteres que considera "puntuación":

```
>>> import string
>>> string.punctuation
'!"#$%&amp;\'()*+,-./:;<=>?@[\\]^_`{|}~'
```

Los parámetros utilizados por `translate` eran diferentes en Python 2.0.

Realizamos las siguientes modificaciones a nuestro programa:

https://trinket.io/embed/python3/25874ef3fd

Parte de aprender el "Arte de Python" o "Thinking Pythonically" es darse cuenta de que Python a menudo tiene capacidades incorporadas para muchos problemas comunes de análisis de datos. Con el tiempo, verás suficiente código de ejemplo y leerás suficiente documentación para saber dónde buscar para ver si alguien ya ha escrito algo que facilita mucho tu trabajo.

La siguiente es una versión abreviada de la salida:

```
Enter the file name: romeo-full.txt
{'swearst': 1, 'all': 6, 'afeard': 1, 'leave': 2, 'these': 2,
'kinsmen': 2, 'what': 11, 'thinkst': 1, 'love': 24, 'cloak': 1,
a': 24, 'orchard': 2, 'light': 5, 'lovers': 2, 'romeo': 40,
'maiden': 1, 'whiteupturned': 1, 'juliet': 32, 'gentleman': 1,
'it': 22, 'leans': 1, 'canst': 1, 'having': 1, ...}
```

La salida aún es difícil de manejar y podemos usar Python para darnos exactamente lo que estamos buscando, pero para hacerlo, necesitamos aprender sobre las tuplas de Python. Retomaremos este ejemplo una vez que aprendamos sobre las tuplas.

# Depurando

A medida que trabajas con conjuntos de datos más grandes, puede volverse difícil manejar la depuración imprimiendo y verificando los datos a mano. Aquí hay algunas sugerencias para depurar grandes conjuntos de datos:

**Reduce la entrada**

Si es posible, reduzce el tamaño del conjunto de datos. Por ejemplo, si el programa lee un archivo de texto, comienza solo con las primeras 10 líneas o con el ejemplo más pequeño que puedas encontrar. Puedes editar los archivos por sí mismos o (mejor) modificar el programa para que solo lea las primeras líneas `n`.

Si hay un error, puede reducir `n` al valor más pequeño que manifiesta el error, y luego aumentarlo gradualmente a medida que encuentre y corrija los errores.

**Comprueba sumarios y tipos**

En lugar de imprimir y verificar todo el conjunto de datos, considera imprimir resúmenes de los datos: por ejemplo, la cantidad de elementos en un diccionario o el total de una lista de números.

Una causa común de errores en tiempo de ejecución es un valor que no es el tipo correcto. Para depurar este tipo de error, a menudo es suficiente imprimir el tipo de un valor.

Otro tipo de verificación compara los resultados de dos cálculos diferentes para ver si son consistentes. Esto se llama "verificación de consistencia".

**Imprime la salida con formato**

Dar formato a la salida de depuración puede facilitar la detección un error.

Nuevamente, el tiempo que dedicas a construir andamios puede reducir el tiempo que dedicas a la depuración.

# Ejercicios

**Ejercicio 2**: Escribe un programa que categorice cada mensaje de correo según el día de la semana en que se realizó la confirmación. Para hacer esto, busca líneas que comiencen con "From", luego busca la tercera palabra y manten un conteo de cada uno de los días de la semana. Al final del programa, imprime el contenido de tu diccionario (el orden no importa).

Línea de muestra:

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
```

Ejecución de la muestra:

```
python dow.py
Enter a file name: mbox-short.txt
{'Fri': 20, 'Thu': 6, 'Sat': 1}
```

**Ejercicio 3**: Escribe un programa que lea un registro de correo, cree un histograma usando un diccionario para contar cuántos mensajes has recibido de cada dirección de correo electrónico y luego imprima el diccionario.

```
Enter file name: mbox-short.txt
{'gopal.ramasammycook@gmail.com': 1, 'louis@media.berkeley.edu': 3,
```

```
'cwen@iupui.edu': 5, 'antranig@caret.cam.ac.uk': 1,
'rjlowe@iupui.edu': 2, 'gsilver@umich.edu': 3,
'david.horwitz@uct.ac.za': 4, 'wagnermr@iupui.edu': 1,
'zqian@umich.edu': 4, 'stephen.marquard@uct.ac.za': 2,
'ray@media.berkeley.edu': 1}
```

**Ejercicio4**: Agrega código al programa anterior para averiguar quién tiene más mensajes en el archivo.

Una vez que hayas leído todos los datos y se haya creado el diccionario, examina el diccionario utilizando un bucle máximo para encontrar quién tiene más mensajes e imprime cuántos mensajes tiene la persona.

```
Enter a file name: mbox-short.txt
cwen@iupui.edu 5

Enter a file name: mbox.txt
zqian@umich.edu 195
```

**Ejercicio 5**: Este programa registra el nombre de dominio (en lugar de la dirección) desde donde se envió el mensaje en lugar de a quién le llegó el correo (es decir, la dirección de correo electrónico completa). Al final del programa, imprime el contenido de tu diccionario.

```
python schoolcount.py
Enter a file name: mbox-short.txt
{'media.berkeley.edu': 4, 'uct.ac.za': 6, 'umich.edu': 7,
'gmail.com': 1, 'caret.cam.ac.uk': 1, 'iupui.edu': 8}
```

Copyright 2026

# 10 Tuplas

## Las tuplas son inmutables

Una tupla[1] es una secuencia de valores muy parecida a una lista. Los valores almacenados en una tupla pueden ser de cualquier tipo, y están indexados por enteros. La diferencia importante es que las tuplas son **inmutables**. Las tuplas también son **comparables** y **hashables**, por lo que podemos ordenar las listas y usar tuplas como valores en los diccionarios de Python.

Sintácticamente, una tupla es una lista de valores separados por comas:

```
>>> t = 'a', 'b', 'c', 'd', 'e'
```

Aunque no es necesario, es común incluir tuplas entre paréntesis para ayudarnos a identificar tuplas rápidamente cuando observamos el código de Python:

```
>>> t = ('a', 'b', 'c', 'd', 'e')
```

Para crear una tupla con un solo elemento, debe incluir la coma final:

```
>>> var_tupla = 'a',
>>> type(var_tupla)
<class 'tuple'>
```

Sin la coma, Python trata `('a')` como una expresión con una cadena entre paréntesis que se evalúa como una cadena:

```
>>> t2 = ('a')
>>> type(t2)
<type 'str'>
```

Otra forma de construir una tupla es la función incorporada `tupla`. Sin argumento, crea una tupla vacía; o usando paréntesis `()`:

```
>>> t2 = ()
>>> type(t2)
<class 'tuple'>
```

Si el argumento es una secuencia (cadena, lista o tupla), el resultado de la llamada a `tuple` es una tupla con los elementos de la secuencia:

```
>>> t = tuple('lupins')
>>> print(t)
('l', 'u', 'p', 'i', 'n', 's')
```

Como `tuple` es el nombre de un constructor, debes evitar usarlo como nombre de variable.

La mayoría de los operadores de listas también funcionan con las tuplas. El operador de corchete permite acceder a un elemento:

```
>>> t = ('a', 'b', 'c', 'd', 'e')
>>> print(t[0])
a
```

Y el operador de corte selecciona un rango de elementos.

```
>>> print(t[1:3])
('b', 'c')
```

Pero si intentas modificar uno de los elementos de la tupla, obtendrás un error:

```
>>> t[0] = 'A'
TypeError: object doesn't support item assignment
```

No puedes modificar los elementos de una tupla, pero puedes reemplazar una tupla por otra:

```
>>> t = ('a', 'b', 'c', 'd', 'e')
>>> t = ('A',) + t
>>> t
('A', 'a', 'b', 'c', 'd', 'e')
```

Copyright 2026

# Comparando tuplas

Los operadores de comparación funcionan en tuplas y otras secuencias. Python comienza comparando el primer elemento de cada secuencia. Si son iguales, continúa con el siguiente elemento, y así sucesivamente, hasta que encuentre elementos que difieran. Los elementos subsiguientes no se consideran (incluso si son realmente grandes).

```
>>> (0, 1, 2) < (0, 3, 4)
True
>>> (0, 1, 2000000) < (0, 3, 4)
True
```

La función `sort` funciona de la misma manera. Se ordena principalmente por el primer elemento, pero en el caso de un empate, se ordena por el segundo elemento, y así sucesivamente.

Esta característica se presta a un patrón llamado **DSU**:

**Decorate**

Una secuencia mediante la creación de una lista de tuplas con una o más claves de clasificación que preceden a los elementos de la secuencia,

**Sort**

la lista de tuplas usando el `sort` incorporado de Python, y

**Undecorate**

Extrayendo los elementos ordenados de la secuencia.

[DSU]

Por ejemplo, supón que tienes una lista de palabras y deseas clasificarlas de la más larga a la más corta:

https://trinket.io/embed/python3/84b1d6d4a1

El primer bucle crea una lista de tuplas, donde cada tupla es una palabra precedida por su longitud.

`sort` compara el primer elemento, la longitud, y solo considera el segundo elemento para romper empates. El argumento `reverse=True` le dice a `sort` que vaya en orden decreciente.

El segundo bucle recorre la lista de tuplas y crea una lista de palabras en orden descendente de longitud. Las palabras de cuatro caracteres están ordenadas en orden **alfabético inverso**, por lo que "what" aparece antes de "soft" en la siguiente lista.

La salida del programa es:

```
['yonder', 'window', 'breaks', 'light', 'what',
'soft', 'but', 'in']
```

Por supuesto, la línea pierde gran parte de su impacto poético cuando se convierte en una lista de Python y se clasifica en orden de longitud de palabra descendente.

# Asignación de tupla

Una de las características sintácticas únicas del lenguaje Python es la capacidad de tener una tupla en el lado izquierdo de una asignación. Esto le permite asignar más de una variable a la vez cuando el lado izquierdo es una secuencia.

En este ejemplo tenemos una lista de dos elementos (que es una secuencia) y asignamos el primer y segundo elementos de la secuencia a las variables `x` y `y` en una sola declaración.

```
>>> m = [ 'have', 'fun' ]
>>> x, y = m
>>> x
'have'
>>> y
'fun'
```

No es magia, Python traduce la sintaxis de asignación de tupla para que sea la siguiente: [2]

```
>>> m = [ 'have', 'fun' ]
>>> x = m[0]
```

```
>>> y = m[1]
>>> x
'have'
>>> y
'fun'
```

Estilísticamente, cuando usamos una tupla en el lado izquierdo de la declaración de asignación, omitimos los paréntesis, pero la siguiente es una sintaxis igualmente válida:

```
>>> m = [ 'have', 'fun' ]
>>> (x, y) = m
>>> x
'have'
>>> y
'fun'
>>>
```

Una aplicación especialmente inteligente de la asignación de tuplas nos permite **intercambiar** los valores de dos variables en una sola declaración:

```
>>> a, b = b, a
```

Ambos lados de esta declaración son tuplas, pero el lado izquierdo es una tupla de variables. El lado derecho es una tupla de expresiones. Cada valor en el lado derecho se asigna a su variable respectiva en el lado izquierdo. Todas las expresiones en el lado derecho son evaluadas antes de cualquiera de las asignaciones.

El número de variables a la izquierda y el número de valores a la derecha deben ser iguales:

```
>>> a, b = 1, 2, 3
ValueError: too many values to unpack
```

Generalmente, el lado derecho puede ser cualquier tipo de secuencia (cadena, lista o tupla). Por ejemplo, para dividir una dirección de correo electrónico en un nombre de usuario y un dominio, podría escribir:

```
>>> addr = 'monty@python.org'
>>> uname, domain = addr.split('@')
```

Copyright 2026

El valor de retorno de `split` es una lista con dos elementos; el primer elemento se asigna a `uname`, el segundo a `domain`.

```
>>> print(uname)
monty
>>> print(domain)
python.org
```

# Diccionarios y tuplas

Los diccionarios tienen un método llamado `items` que devuelve una lista de tuplas, donde cada tupla es un par clave-valor:

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> print(t)
[('a', 10), ('b', 1), ('c', 22)]
```

Como puedes esperar de un diccionario, los elementos no están en ningún orden en particular.

Sin embargo, como la lista de tuplas es una lista y las tuplas son comparables, ahora podemos ordenar la lista de tuplas. Convertir un diccionario en una lista de tuplas es una forma de generar el contenido de un diccionario ordenado por clave:

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> t
[('b', 1), ('a', 10), ('c', 22)]
>>> t.sort()
>>> t
[('a', 10), ('b', 1), ('c', 22)]
```

La nueva lista se clasifica en orden alfabético ascendente por el valor clave.

# Asignación múltiple con diccionarios

Combinando `items`, asignación de tuplas y `for`, puedes ver un bonito patrón de código para recorrer las claves y los valores de un diccionario en un solo bucle:

```
for key, val in list(d.items()):
    print(val, key)
```

Este bucle tiene dos **variables de iteración** porque `items` devuelve una lista de tuplas y `key, val` es una asignación de tuplas que se repite sucesivamente a través de cada uno de los pares clave-valor en el diccionario.

Para cada iteración a través del bucle, tanto `key` como `val` avanzan al siguiente par clave-valor en el diccionario (aún en orden hash).

La salida de este bucle es:

```
10 a
22 c
1 b
```

Nuevamente, está en orden de clave hash (es decir, no hay un orden particular).

Si combinamos estas dos técnicas, podemos imprimir el contenido de un diccionario ordenado por el **valor** almacenado en cada par clave-valor.

Para hacer esto, primero hacemos una lista de tuplas donde cada tupla es `(valor, clave)`. El método `items` nos daría una lista de tuplas `(clave, valor)`, pero esta vez queremos ordenar por valor, no por clave. Una vez que hemos construido la lista con las tuplas de clave de valor, es una cuestión simple ordenar la lista en orden inverso e imprimir la nueva lista ordenada.

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> l = list()
>>> for key, val in d.items() :
...     l.append( (val, key) )
...
>>> l
[(10, 'a'), (22, 'c'), (1, 'b')]
>>> l.sort(reverse=True)
>>> l
[(22, 'c'), (10, 'a'), (1, 'b')]
```

```
>>>
```

Al construir cuidadosamente la lista de tuplas para que tenga el valor como primer elemento de cada tupla, podemos ordenar la lista de tuplas y obtener el contenido de nuestro diccionario ordenado por valor.

# Las palabras más comunes

Volviendo a nuestro ejemplo de ejecución del texto de **Romeo y Julieta** Act 2, Scene 2, podemos aumentar nuestro programa para usar esta técnica para imprimir las diez palabras más comunes en el texto de la siguiente manera:

https://trinket.io/embed/python3/2f13e5b367

La primera parte del programa, que lee el archivo y crea el diccionario que asigna a cada palabra su conteo en el documento, no se modifica. Pero en lugar de simplemente imprimir `count` y finalizar el programa, construimos una lista de tuplas `(val, key)` y luego ordenamos la lista en orden inverso.

Como el valor es primero, se utilizará para las comparaciones. Si hay más de una tupla con el mismo valor, se verá el segundo elemento (la clave), por lo que las tuplas en las que el valor es el mismo se ordenarán según el orden alfabético de la clave.

Al final, escribimos un bonito bucle `for` que realiza una iteración de asignación múltiple e imprime las diez palabras más comunes al recorrer una parte de la lista (`lst[:10]`).

Así que ahora la salida finalmente se parece a lo que queremos para nuestro análisis de frecuencia de palabras.

```
61 i
42 and
40 romeo
34 to
34 the
32 thou
32 juliet
```

```
30 that
29 my
24 thee
```

El hecho de que este análisis de datos complejos se pueda realizar con un programa Python de 19 líneas fácil de entender es una de las razones por las que Python es una buena opción como lenguaje para explorar información.

# Uso de tuplas como claves en los diccionarios

Debido a que las tuplas son **hashables** y las listas no, si queremos crear una clave **compuesta** para usar en un diccionario, debemos usar una tupla como clave.

Nos encontraríamos con una clave compuesta si quisiéramos crear un directorio telefónico que se asigne desde los pares de apellidos y nombre a los números de teléfono. Suponiendo que hayamos definido las variables `last`, `first` y `number`, podríamos escribir una declaración de asignación de diccionario de la siguiente manera:

```
directory[last,first] = number
```

La expresión entre corchetes es una tupla. Podríamos usar la asignación de tuplas en un bucle `for` para recorrer este diccionario.

```
for last, first in directory:
    print(first, last, directory[last,first])
```

Este bucle recorre las claves en `directory`, que son tuplas. Asigna los elementos de cada tupla a `last` y `first`, luego imprime el nombre, apellido y el número de teléfono correspondientes.

# Secuencias: cadenas, listas y tuplas - ¡Oh My!

Me he centrado en las listas de tuplas, pero casi todos los ejemplos de este capítulo también funcionan con listas de listas, tuplas de tuplas y tuplas de listas. Para evitar enumerar las posibles

combinaciones, a veces es más fácil hablar de secuencias de secuencias.

En muchos contextos, los diferentes tipos de secuencias (cadenas, listas y tuplas) se pueden usar indistintamente. Entonces, ¿cómo y por qué eliges uno sobre los otros?

Para comenzar con lo obvio, las cadenas son más limitadas que otras secuencias porque los elementos deben ser caracteres. También son inmutables. Si necesita la capacidad de cambiar los caracteres en una cadena (en lugar de crear una nueva cadena), es posible que desee utilizar una lista de caracteres en su lugar.

Las listas son más comunes que las tuplas, principalmente porque son mutables. Pero hay algunos casos en los que quizás prefieras las tuplas:

1. En algunos contextos, como una declaración `return`, es sintácticamente más simple crear una tupla que una lista. En otros contextos, es posible que prefieras una lista.
2. Si deseas usar una secuencia como clave de diccionario, debes usar un tipo inmutable como una tupla o cadena.
3. Si estás pasando una secuencia como un argumento a una función, el uso de tuplas reduce el potencial de comportamiento inesperado debido al aliasing.

Debido a que las tuplas son inmutables, no proporcionan métodos como `sort` y `reverse`, que modifican las listas existentes. Sin embargo, Python proporciona las funciones integradas `sorted` y `reverseed`, que toman cualquier secuencia como parámetro y devuelven una nueva secuencia con los mismos elementos en un orden diferente.

# Depurando

Las listas, los diccionarios y las tuplas se conocen genéricamente como **estructuras de datos**. En este capítulo estamos empezando a ver estructuras de datos compuestas, como listas de tuplas y diccionarios que contienen tuplas como claves y listas como valores. Las estructuras de datos compuestos son útiles, pero son propensas a lo que yo llamo **errores de forma**, es decir, los errores causados cuando una estructura de datos tiene el tipo, tamaño o composición incorrectos. Puede ocurrir que escribas un código, olvides la forma de tus datos y posteriormente introduzcas un error.

Por ejemplo, si estás esperando una lista con un entero y te doy un entero simple (no en una lista), no funcionará.

Cuando estás depurando un programa, y especialmente si estás trabajando en un error, hay cuatro cosas que puedes intentar:

**léelo**

Examina tu código, léelo de nuevo y verifica que diga lo que querías decir.

**ejecútalo**

Experimenta haciendo cambios y ejecutando diferentes versiones. A menudo, si muestras lo correcto en el lugar correcto en el programa, el problema se vuelve obvio, pero a veces debes dedicar algo de tiempo a construir andamios.

**máscalo**

¡Tómate un tiempo para pensar! ¿Qué tipo de error es: sintáctico, en tiempo de ejecución, semántico? ¿Qué información puedes obtener de los mensajes de error o de la salida del programa? ¿Qué tipo de error podría causar el problema que estás viendo? ¿Qué fue lo último que cambiaste, antes de que apareciera el problema?

**déjalo**

En algún momento, lo mejor que puede hacer es retirarte, deshacer los cambios recientes, hasta que vuelvas a un programa que funcione y que comprendas. Entonces puedes empezar a reconstruir.

Los programadores principiantes a veces se atascan en una de estas tareas y se olvidan de las demás. Cada actividad viene con su propio modo de falla.

Por ejemplo, leer tu código puede ayudar si el problema tiene un error tipográfico, pero no si el problema es un malentendido conceptual. Si no entiendes lo que hace tu programa, puedes leerlo 100 veces y nunca ver el error, porque el error está en tu cabeza.

Realizar experimentos puede ayudar, especialmente si ejecutas pruebas pequeñas y simples. Pero si ejecutas experimentos sin pensar o leer tu código, podrías caer en un patrón que yo llamo "programación aleatoria", que es el proceso de hacer cambios aleatorios hasta que el programa haga lo correcto. No hace falta decir que la programación aleatoria puede llevar mucho tiempo.

Tienes que tomarte tiempo para pensar. La depuración es como una ciencia experimental. Debes tener al menos una hipótesis sobre cuál es el problema. Si hay dos o más posibilidades, trata de pensar en una prueba que elimine una de ellas.

Tomar un descanso ayuda a pensar. También lo hace hablar. Si explicas el problema a otra persona (o incluso a ti mismo), a veces encontrarás la respuesta antes de terminar de formular la pregunta.

Copyright 2026

Pero incluso las mejores técnicas de depuración fallarán si hay demasiados errores, o si el código que intentas corregir es demasiado grande y complicado. A veces, la mejor opción es retirarte, simplificando el programa hasta que llegues a algo que funcione y que comprendas.

Los programadores principiantes a menudo son reacios a retirarse porque no pueden soportar eliminar una línea de código (incluso si está mal). Si te hace sentir mejor, copia tu programa en otro archivo antes de comenzar a borrarlo. Luego puedes pegar las piezas de nuevo poco a poco.

Encontrar un error difícil requiere leer, correr, rumiar y, a veces, retirarse. Si te quedas estancado en una de estas actividades, prueba las otras.

# Ejercicios

**Ejercicio 1:** Revisa un programa anterior de la siguiente manera: Lee y analiza las líneas "From" y saca las direcciones de la línea. Cuenta el número de mensajes de cada persona usando un diccionario.

Después de haber leído todos los datos, imprime la persona con el mayor número de confirmaciones creando una lista de tuplas (conteo, correo electrónico) del diccionario. Luego, ordena la lista en orden inverso e imprime a la persona que tenga más envíos.

```
Sample Line:
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008

Enter a file name: mbox-short.txt
cwen@iupui.edu 5

Enter a file name: mbox.txt
zqian@umich.edu 195
```

**Ejercicio 2:** Este programa cuenta la distribución de las horas del día para cada uno de los mensajes. Puedes sacar la hora de la línea "From" encontrando la cadena de tiempo y luego dividiendo esa cadena en partes usando el carácter de dos puntos. Una vez que hayas acumulado los conteos para cada hora, imprime los conteos, uno por línea, ordenados por hora como se muestra a continuación.

Ejecución de la muestra:

```
python timeofday.py
Enter a file name: mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
```

**Ejercicio 3:** Escribe un programa que lea un archivo e imprima las **letras** en orden decreciente de frecuencia. tu programa debe convertir todas las entradas a minúsculas y solo contar las letras a-z. Tu programa no debe contar espacios, dígitos, puntuación o cualquier otra cosa que no sean las letras a-z. Encuentra muestras de texto de varios idiomas diferentes y observa cómo la frecuencia de las letras varía según el idioma. Compara tus resultados con las tablas en wikipedia.org/wiki/Letter_frequencies.

---

> [1]. Dato curioso: la palabra "tupla" proviene de los nombres que se dan a las secuencias de números de diferentes longitudes: simple, doble, triple, cuádruple, quituple, sextuple, septuple, etc. ↩

> [1]. Python no traduce la sintaxis literalmente. Por ejemplo, si intentas esto con un diccionario, no funcionará como podría esperarse. ↩

# 11 Regex

Hasta ahora hemos estado leyendo archivos, buscando patrones y extrayendo varios trozos de líneas que nos parecen interesantes. Hemos ido utilizando métodos de cadena como `split` y `find`, utilizando listas y troceado de cadenas para extraer partes de las líneas.

Esta tarea de búsqueda y extracción es tan común que Python tiene una biblioteca muy poderosa llamada **expresiones regulares** que maneja muchas de estas tareas de manera muy elegante. La razón por la que no hemos introducido expresiones regulares anteriormente en el libro es que, si bien son muy potentes, son un poco complicadas y su sintaxis requiere un tiempo para acostumbrarse.

Las expresiones regulares son casi su propio lenguaje de programación para buscar y analizar cadenas. De hecho, se han escrito libros completos sobre el tema de las expresiones regulares. En este capítulo, solo cubriremos los conceptos básicos de las expresiones regulares. Aquí tienes más detalles sobre expresiones regulares. También en la documentación de Python:

https://docs.python.org/3.7/library/re.html

La biblioteca de expresiones regulares, `re`, debe importarse en tu programa antes de poder usarla. El uso más simple de la biblioteca de expresiones regulares es la función `search()`. El siguiente programa demuestra un uso trivial de la función de búsqueda.

```
# Search for lines that contain 'From'
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('From:', line):
        print(line)

# Code: http://www.py4e.com/code3/re01.py
```

Abrimos el archivo, recorremos cada línea y usamos la función `search()` para imprimir solo las líneas que contienen la cadena "From:". Este programa no usa el poder real de las expresiones regulares, ya que podríamos haber utilizado `line.find()` con la misma facilidad para lograr el mismo resultado.

Copyright 2026

El poder de las expresiones regulares llega cuando agregamos caracteres especiales a la cadena de búsqueda que nos permiten controlar con mayor precisión qué líneas coinciden con la cadena. Agregar estos caracteres especiales a nuestra expresión regular nos permite hacer una comparación y extracción sofisticadas mientras escribimos muy poco código.

Por ejemplo, el carácter `^` se usa en expresiones regulares para coincidir con "el principio" de una línea. Podríamos cambiar nuestro programa para que solo coincida con las líneas donde "De:" está al principio de la línea de la siguiente manera:

https://trinket.io/embed/python3/77c2350d8b

Ahora solo haremos coincidir las líneas que **comiencen con** la cadena "De:". Este es un ejemplo muy simple que podríamos haber hecho de manera equivalente con el método `startswith()` de la biblioteca de cadenas. Pero sirve para introducir la idea de que las expresiones regulares contienen caracteres de acción especiales que nos dan más control en cuanto a qué coincidirá con la expresión regular.

# Coincidencia de caracteres en expresiones regulares

Hay una serie de caracteres especiales que nos permiten construir expresiones regulares aún más potentes. El carácter especial más utilizado es el punto, que coincide con cualquier carácter.

En el siguiente ejemplo, la expresión regular "F..m:" coincidiría con cualquiera de las cadenas "From:", "Fxxm:", "F12m:" o "F!@M:".

https://trinket.io/embed/python3/d558453717

Esto es particularmente poderoso cuando se combina con la capacidad de indicar que un caracter puede repetirse cualquier cantidad de veces utilizando los caracteres "*" o "+" en su expresión regular. Estos caracteres especiales significan que, en lugar de coincidir con un solo caracter en la cadena de búsqueda, coinciden con cero o más caracteres (en el caso del asterisco), o uno o más caracteres (en el caso del signo más) .

Copyright 2026

Podemos restringir aún más las líneas 'interesantes' utilizando el caracter repetido de **comodín** (wildcard) en el siguiente ejemplo:

https://trinket.io/embed/python3/e82ead95e5

La cadena de búsqueda `^From:.+@` coincidirá con las líneas que comienzan con "From:", seguido de uno o más caracteres (". +"), Seguido de un signo de at. Así que esto coincidirá con la siguiente línea:

```
From: uct.ac.za
```

Puede pensar que el comodín ".+" se expande para que coincida con todos los caracteres entre el carácter de dos puntos y la arroba.

Es bueno saber que los caracteres de más y asterisco son "agresivos". Por ejemplo, la siguiente cadena coincidiría con la última arroba en la cadena cuando el ".+" Empuja hacia afuera, como se muestra a continuación:

```
From: iupui.edu
```

Es posible decirle a un asterisco o signo más que no sea tan "codicioso" al agregar otro personaje.

Consulte la documentación detallada para obtener información sobre cómo desactivar el comportamiento codicioso.

# Extracción de datos usando expresiones regulares

Si queremos extraer datos de una cadena en Python, podemos usar el método `findall()` para extraer todas las subcadenas que coinciden con una expresión regular. Usemos el ejemplo de querer extraer todo lo que parece una dirección de correo electrónico desde cualquier línea, independientemente del formato. Por ejemplo, queremos extraer las direcciones de correo electrónico de cada una de las siguientes líneas:

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
Return-Path: <postmaster@collab.sakaiproject.org>
          for <source@collab.sakaiproject.org>;
```

```
Received: (from apache@localhost)
Author: stephen.marquard@uct.ac.za
```

No queremos escribir código para cada uno de los tipos de líneas, dividir y dividir de manera diferente para cada línea. Este programa siguiente utiliza `findall()` para encontrar las líneas con direcciones de correo electrónico en ellas y extraer una o más direcciones de cada una de esas líneas.

https://trinket.io/embed/python3/e8248e2794

El método `findall()` busca la cadena en el segundo argumento y devuelve una lista de todas las cadenas que parecen direcciones de correo electrónico. Estamos utilizando una secuencia de dos caracteres que coincide con uno o más caracteres que no sean un espacio en blanco (`\S`).

La salida del programa sería:

```
['csev@umich.edu', 'cwen@iupui.edu']
```

Al traducir la expresión regular, estamos buscando subcadenas que tengan al menos un carácter que no sea un espacio en blanco, seguido de un signo `@`, seguido de al menos un carácter más que no sea un espacio en blanco. La "`\S+`" coincide con la mayor cantidad posible de caracteres que no sean espacios en blanco.

La expresión regular coincidiría dos veces (csev@umich.edu y cwen@iupui.edu), pero no coincidiría con la cadena "@2PM" porque no hay caracteres que no estén en blanco **antes** del signo de inicio. Podemos usar esta expresión regular en un programa para leer todas las líneas de un archivo e imprimir cualquier cosa que parezca una dirección de correo electrónico de la siguiente manera:

https://trinket.io/embed/python3/674fe79c92

Leemos cada línea y luego extraemos todas las subcadenas que coinciden con nuestra expresión regular. Como `findall()` devuelve una lista, simplemente verificamos si el número de elementos en nuestra lista devuelta es más que cero para imprimir solo las líneas donde encontramos al menos una subcadena que parece una dirección de correo electrónico.

Si ejecutamos el programa en `mbox.txt` obtenemos el siguiente resultado:

```
['wagnermr@iupui.edu']
['cwen@iupui.edu']
['<postmaster@collab.sakaiproject.org>']
['<200801032122.m03LMFo4005148@nakamura.uits.iupui.edu>']
['<source@collab.sakaiproject.org>;']
['<source@collab.sakaiproject.org>;']
['<source@collab.sakaiproject.org>;']
['apache@localhost)']
['source@collab.sakaiproject.org;']
```

Algunas de nuestras direcciones de correo electrónico tienen caracteres incorrectos como "`<`" o ";" al principio o al final. Declaremos que solo nos interesa la parte de la cadena que comienza y termina con una letra o un número.

Para ello, utilizamos otra característica de las expresiones regulares. Los corchetes se utilizan para indicar un conjunto de múltiples caracteres aceptables que estamos dispuestos a considerar que coincidan. En cierto sentido, el "`\S`" está pidiendo que coincida con el conjunto de "caracteres que no son espacios en blanco". Ahora vamos a ser un poco más explícitos en términos de los caracteres con los que coincidiremos.

Aquí está nuestra nueva expresión regular:

```
[a-zA-Z0-9]\S*@\S*[a-zA-Z]
```

Esto se está complicando un poco y puedes empezar a ver porqué las expresiones regulares son un lenguaje en sí mismas. Al traducir esta expresión regular, vemos que estamos buscando subcadenas que comiencen con una sola minúscula, mayúscula o número "[a-zA-Z0-9]", seguido de cero o más caracteres que no estén en blanco ("`\S*`"), seguido de un signo arroba, seguido de cero o más caracteres que no estén en blanco, seguido de una letra mayúscula o minúscula. Ten en cuenta que cambiamos de "+" a `*` para indicar cero o más caracteres que no están en blanco ya que "[a-zA-Z0-9]" ya es un carácter que no está en blanco. Recuerda que el `*` o "+" se aplica al carácter único inmediatamente a la izquierda del signo más o asterisco.

Si usamos esta expresión en nuestro programa, nuestros datos son mucho más limpios:

https://trinket.io/embed/python3/1b176f4d0e

Copyright 2026

```
...
['wagnermr@iupui.edu']
['cwen@iupui.edu']
['postmaster@collab.sakaiproject.org']
['200801032122.m03LMFo4005148@nakamura.uits.iupui.edu']
['source@collab.sakaiproject.org']
['source@collab.sakaiproject.org']
['source@collab.sakaiproject.org']
['apache@localhost']
```

Observa que en las líneas "source@collab.sakaiproject.org", nuestra expresión regular eliminó dos letras al final de la cadena (">;"). Esto se debe a que cuando agregamos "[a-zA-Z]" al final de nuestra expresión regular, estamos exigiendo que cualquier cadena que el analizador de expresiones regulares encuentre debe terminar con una letra. Entonces, cuando ve ">" después de "sakaiproject.org> simplemente se detiene en la última letra "coincidente" que encontró (es decir, la "g" fue la última buena coincidencia).

También tenga en cuenta que la salida del programa es una lista de Python que tiene una cadena como elemento único en la lista.

# Combinando búsqueda y extracción

Si queremos encontrar números en las líneas que comienzan con la cadena "X-", como por ejemplo:

```
X-DSPAM-Confidence: 0.8475
X-DSPAM-Probability: 0.0000
```

No solo queremos números de punto flotante de cualquier línea. Solo queremos extraer números de líneas que tengan la sintaxis anterior.

Podemos construir la siguiente expresión regular para seleccionar las líneas:

```
^X-.*: [0-9.]+
```

Tras traducir esto, estamos diciendo que queremos líneas que comiencen con "X-", seguidas de cero o más caracteres (".*"), Seguidas de dos puntos (":") y luego un espacio. Después del espacio, estamos buscando uno o más caracteres que sean un dígito (0-9) o un punto "[0-9.]+". Ten en

cuenta que dentro de los corchetes, el período coincide con un período real (es decir, no es un comodín entre los corchetes).

Esta es una expresión muy estricta que coincidirá en gran medida solo con las líneas que nos interesan de la siguiente manera:

https://trinket.io/embed/python3/18870dcf26

Cuando ejecutamos el programa, vemos que los datos se filtran bien para mostrar solo las líneas que estamos buscando.

```
X-DSPAM-Confidence: 0.8475
X-DSPAM-Probability: 0.0000
X-DSPAM-Confidence: 0.6178
X-DSPAM-Probability: 0.0000
```

Pero ahora tenemos que resolver el problema de extraer los números. Si bien sería lo suficientemente simple como para usar `split`, podemos usar otra característica de las expresiones regulares para buscar y analizar la línea al mismo tiempo.

Los paréntesis son otro carácter especial en las expresiones regulares. Cuando agrega paréntesis a una expresión regular, se ignoran cuando coinciden con la cadena. Pero cuando está utilizando `findall()`, los paréntesis indican que aunque desea que la expresión completa coincida, solo le interesa extraer una parte de la subcadena que coincida con la expresión regular.

Entonces hacemos el siguiente cambio a nuestro programa:

https://trinket.io/embed/python3/38a31c1ce9

En lugar de llamar a `search()`, agregamos paréntesis a la parte de la expresión regular que representa el número de punto flotante para indicar que solo queremos que `findall()` nos devuelva la parte del número de punto flotante de la cadena correspondiente .

La salida de este programa es la siguiente:

```
['0.8475']
['0.0000']
['0.6178']
['0.0000']
['0.6961']
['0.0000']
..
```

Los números aún están en una lista y deben convertirse de cadenas a puntos flotantes, pero hemos utilizado el poder de las expresiones regulares para buscar y extraer la información que encontramos interesante.

Como otro ejemplo de esta técnica, si observas el archivo, hay varias líneas del formulario:

```
Details: http://source.sakaiproject.org/viewsvn/?view=rev&amp;rev=39772
```

Si quisiéramos extraer todos los números de revisión (el número entero al final de estas líneas) utilizando la misma técnica anterior, podríamos escribir el siguiente programa:

https://trinket.io/embed/python3/93a830d48e

Al traducir nuestra expresión regular, buscamos líneas que comiencen con "Details:", seguidas de cualquier número de caracteres (".*"), seguidas de "rev=" y luego con uno o más dígitos. Queremos encontrar líneas que coincidan con la expresión completa, pero solo queremos extraer el número entero al final de la línea, por lo que rodeamos "[0-9]+" con paréntesis.

Cuando ejecutamos el programa, obtenemos el siguiente resultado:

```
['39772']
['39771']
['39770']
['39769']
...
```

Recuerde que el "[0-9] +" es "codicioso" y trata de hacer una cadena de dígitos tan grande como sea posible antes de extraer esos dígitos. Este comportamiento "codicioso" es la razón por la que obtenemos los cinco dígitos para cada número. La biblioteca de expresiones regulares se expande

Copyright 2026

en ambas direcciones hasta que encuentra un no dígito, o el principio o el final de una línea.

Ahora podemos usar expresiones regulares para rehacer un ejercicio anterior en el libro en el que estábamos interesados en la hora del día de cada mensaje de correo. Buscamos líneas con la forma siguiente:

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
```

Y queríamos extraer la hora del día para cada línea. Anteriormente hicimos esto con dos llamadas a `split`. Primero, la línea se dividió en palabras y luego sacamos la quinta palabra y la dividimos nuevamente en el caracter de dos puntos para sacar los dos caracteres que nos interesaban.

Si bien esto funcionó, en realidad dio como resultado código bastante frágil que asumía que las líneas estaban bien formateadas. Si tuvieras que agregar suficientes comprobaciones de errores (o un bloque grande de try/except) para asegurarte de que tu programa nunca fallara cuando se le presentaran líneas de formato incorrecto, el código se inflaría a 10-15 líneas de código bastante difícil de leer.

Podemos hacer esto de una manera mucho más simple con la siguiente expresión regular:

```
^From .* [0-9][0-9]:
```

La traducción de esta expresión regular es que estamos buscando líneas que comiencen con "From " (nota el espacio), seguidas de cualquier número de caracteres (".*"), seguidas de un espacio, seguidas de dos dígitos `[0 -9][0-9]`, seguido de un caracter de dos puntos. Esta es la definición de los tipos de líneas que estamos buscando.

Para extraer solo la hora usando `findall()`, agregamos paréntesis alrededor de los dos dígitos de la siguiente manera:

```
^From .* ([0-9][0-9]):
```

Esto resulta en el siguiente programa:

https://trinket.io/embed/python3/6f58c712ee

Cuando el programa se ejecuta, produce el siguiente resultado:

```
['09']
['18']
['16']
['15']
...
```

# carácter de escape

Ya que usamos caracteres especiales en expresiones regulares para hacer coincidir el comienzo o el final de una línea o especificar comodines, necesitamos una forma de indicar que estos caracteres son "normales" y queremos que coincidan con el caracter real, como un signo de dólar o un acento circunflejo.

Podemos indicar que queremos simplemente hacer coincidir un caracter precediendo a ese caracter con una barra invertida. Por ejemplo, podemos encontrar cantidades de dinero con la siguiente expresión regular.

```
import re
x = 'We just received $10.00 for cookies.'
y = re.findall('\$[0-9.]+',x)
```

Como prefijamos el signo de dólar con una barra invertida, en realidad coincide con el signo de dólar en la cadena de entrada en lugar de coincidir con el "fin de línea", y el resto de la expresión regular coincide con uno o más dígitos o el carácter de período. **Nota:** Dentro de los corchetes, los caracteres no son "especiales". Entonces cuando decimos "[0-9.]", Realmente significa dígitos o un punto. Fuera de los corchetes, un punto es el carácter de "comodín" y coincide con cualquier caracter. Dentro de los corchetes, el período es un período.

# Resumen

Si bien esto solo rozó la superficie de las expresiones regulares, hemos aprendido un poco sobre el lenguaje de las expresiones regulares. Son cadenas de búsqueda con caracteres especiales que comunican sus deseos al sistema de expresión regular en cuanto a qué define "coincidencia" y qué se extrae de las cadenas coincidentes. Éstos son algunos de esos caracteres especiales y secuencias de caracteres:

`^` Coincide con el principio de la línea.

Copyright 2026

`$` Coincide con el final de la línea.

`.` Coincide con cualquier caracter (un comodín).

`\s` Coincide con un caracter de espacio en blanco.

`\S` Coincide con un caracter que no es un espacio en blanco (opuesto a `\s`).

`*` Se aplica al caracter inmediatamente anterior e indica que coincida con cero o más de los caracteres anteriores.

`*?` Se aplica al caracter inmediatamente anterior e indica que coincida con cero o más de los caracteres anteriores en "modo no codicioso".

`+` Se aplica al caracter inmediatamente anterior e indica que coincida con uno o más de los caracteres anteriores.

`+?` Se aplica al caracter inmediatamente anterior e indica que coincida con uno o más de los caracteres anteriores en "modo no codicioso".

`[aeiou]` Coincide con un solo caracter siempre que ese caracter esté en el conjunto especificado. En este ejemplo, coincidiría con "a", "e", "i", "o", o "u", pero no con otros caracteres.

`[a-z0-9]` Puedes especificar rangos de caracteres usando el signo menos. Este ejemplo es un solo caracter que debe ser una letra minúscula o un dígito.

`[^A-Za-z]` Cuando dentro de corchetes el primer caracter es un acento circunflejo, invierte la lógica. Este ejemplo coincide con un solo caracter que es cualquier cosa **distinta de** una letra mayúscula o minúscula.

`()` Cuando se agregan paréntesis a una expresión regular, se ignoran con el fin de hacer coincidir permitiendo extraer un subconjunto particular de la cadena coincidente en lugar de toda la cadena cuando se usa `findall()`.

`\b` Coincide con la cadena vacía, pero solo al principio o al final de una palabra.

`\B` Coincide con la cadena vacía, pero no al principio o al final de una palabra.

`\d` Coincide con cualquier dígito decimal; Equivalente al conjunto [0-9].

`\D` Coincide con cualquier caracter que no sea un dígito; equivalente al conjunto [`^` 0-9].

Copyright 2026

# Sección extra para usuarios de Unix / Linux

El soporte para buscar archivos usando expresiones regulares se incorporó al sistema operativo Unix desde la década de 1960 y está disponible en casi todos los lenguajes de programación de una forma u otra.

De hecho, hay un programa de línea de comandos integrado en Unix llamado **grep** (Analizador de Expresión Regular Generalizada) que hace prácticamente lo mismo que los ejemplos de `search()` en este capítulo. Entonces, si tienes un sistema Macintosh o Linux, puedes probar los siguientes comandos en tu terminal.

```
$ grep '^From:' mbox-short.txt
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
```

Esto le dice a `grep` que le muestre las líneas que comienzan con la cadena "From:" en el archivo `mbox-short.txt`. Si experimentas con el comando `grep` un poco y lees la documentación de `grep`, encontrarás algunas diferencias sutiles entre el soporte de expresiones regulares en Python y el soporte de expresiones regulares en `grep`. Como ejemplo, `grep` no admite el caracter que no está en blanco `\S`, por lo que deberá usar la notación de conjuntos un poco más compleja `[^ ]`, que simplemente significa que coincida con un caracter que sea cualquier cosa Aparte de un espacio.

# Depurando

Python tiene una documentación incorporada simple y rudimentaria que puede ser muy útil si necesitas una actualización rápida para activar tu memoria sobre el nombre exacto de un método en particular. Esta documentación se puede ver en el intérprete de Python en modo interactivo.

Puedes abrir un sistema de ayuda interactivo usando `help()`.

```
>>> help()
help> modules
```

Si sabes qué módulo deseas usar, puedes usar el comando `dir()` para encontrar los métodos en el módulo de la siguiente manera:

```
>>> import re
>>> dir(re)
['A', 'ASCII', 'DEBUG', 'DOTALL', 'I', 'IGNORECASE', 'L', 'LOCALE', 'M', 'MULTILINE',
'RegexFlag', 'S', 'Scanner', 'T', 'TEMPLATE', 'U', 'UNICODE', 'VERBOSE', 'X', '_MAXCACHE',
'__all__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__',
'__package__', '__spec__', '__version__', '_alphanum_bytes', '_alphanum_str', '_cache',
'_compile', '_compile_repl', '_expand', '_locale', '_pattern_type', '_pickle', '_subx',
'compile', 'copyreg', 'enum', 'error', 'escape', 'findall', 'finditer', 'fullmatch',
'functools', 'match', 'purge', 'search', 'split', 'sre_compile', 'sre_parse', 'sub', 'subn',
'template']
>>>
```

También puedes obtener una pequeña cantidad de documentación sobre un método en particular usando el comando help.

```
>>> import re
>>> help(re.search)
```

La documentación incorporada no es muy extensa, pero puede ser útil cuando tienes prisa o no tienes acceso a un navegador web o motor de búsqueda.

# Ejercicios

**Ejercicio 1:** Escribe un programa simple para simular el funcionamiento del comando `grep` en Unix. Pídele al usuario que ingrese una expresión regular y cuente el número de líneas que coincidieron con la expresión regular:

```
$ python grep.py
Enter a regular expression: ^Author
mbox.txt had 1798 lines that matched ^Author

$ python grep.py
Enter a regular expression: ^X-
```

```
mbox.txt had 14368 lines that matched ^X-

$ python grep.py
Enter a regular expression: java$
mbox.txt had 4218 lines that matched java$
```

**Ejercicio 2:** Escribe un programa para buscar líneas del formulario

```
`New Revision: 39772`
```

y extrae el número de cada una de las líneas utilizando una expresión regular y el método `findall()`. Calcula el promedio de los números e imprime el promedio.

```
Enter file:mbox.txt
38549.7949721

Enter file:mbox-short.txt
39756.9259259
```

# 12 Programas en red

Si bien muchos de los ejemplos de este libro se han centrado en leer archivos y buscar datos en esos archivos, hay muchas fuentes diferentes de información cuando se considera internet.

En este capítulo simularemos ser un navegador web y recuperar páginas web utilizando el Protocolo de transporte de hipertexto (HTTP). Luego leeremos los datos de la página web y los analizaremos.

# Protocolo de transporte de hipertexto - HTTP

El protocolo de red que alimenta la web es en realidad bastante simple y hay un soporte incorporado en Python llamado `sockets` que hace que sea muy fácil hacer conexiones de red y recuperar datos a través de esos sockets en un programa Python.

Un **socket** es muy parecido a un archivo, excepto que un solo socket proporciona una conexión bidireccional entre dos programas. Puedes leer y escribir en el mismo socket. Si escribes algo en un socket, se envía a la aplicación en el otro extremo del socket. Si lees desde el socket, se te proporcionan los datos que la otra aplicación ha enviado.

Pero si intentas leer un socket cuando el programa en el otro extremo del socket no ha enviado ningún dato, simplemente siéntate y espera. Si los programas en ambos extremos del socket simplemente esperan algunos datos sin enviar nada, esperarán por mucho tiempo.

Una parte importante de los programas que se comunican a través de Internet es tener algún tipo de protocolo. Un protocolo es un conjunto de reglas precisas que determinan quién debe ir primero, qué deben hacer, y luego cuáles son las respuestas a ese mensaje, y quién envía el siguiente, y así sucesivamente. En cierto sentido, las dos aplicaciones en cada extremo del socket están bailando y asegurándose de no pisarse los dedos de los pies.

Hay muchos documentos que describen estos protocolos de red. El protocolo de transporte de hipertexto se describe en el siguiente documento:

http://www.w3.org/Protocols/rfc2616/rfc2616.txt

Copyright 2026

Este es un documento largo y complejo de 176 páginas con muchos detalles. Si lo encuentras interesante, siéntete libre de leerlo todo. Pero si echas un vistazo a la página 36 de RFC2616 encontrarás la sintaxis de la solicitud GET. Para solicitar un documento de un servidor web, hacemos una conexión al servidor `www.pr4e.org` en el puerto 80 y luego enviamos una línea del formulario

```
GET http://data.pr4e.org/romeo.txt HTTP/1.0
```

donde el segundo parámetro es la página web que solicitamos, y luego también enviamos una línea en blanco. El servidor web responderá con cierta información de encabezado sobre el documento y una línea en blanco seguida del contenido del documento.

# El navegador web más simple del mundo

Quizás la forma más fácil de mostrar cómo funciona el protocolo HTTP es escribir un programa Python muy simple que haga una conexión a un servidor web y siga las reglas del protocolo HTTP para solicitar un documento y mostrar lo que el servidor envía.

https://trinket.io/embed/python3/734cda1053

Primero, el programa realiza una conexión al puerto 80 en el servidor [www.py4e.com] ( http://www.py4e.com). Dado que nuestro programa desempeña el papel de "navegador web", el protocolo HTTP dice que debemos enviar el comando GET seguido de una línea en blanco.

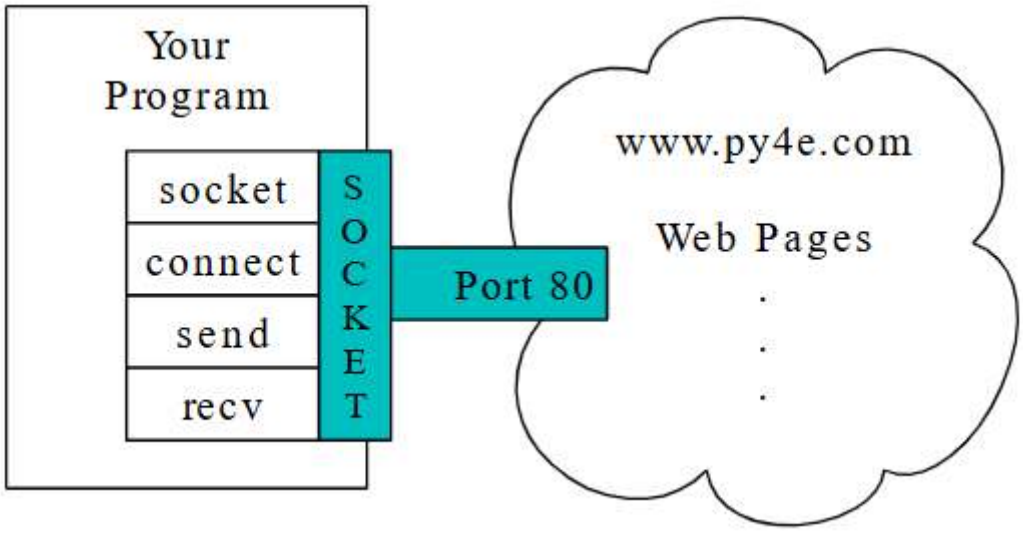


Imagen - A Socket Connection

Una vez que enviamos esa línea en blanco, escribimos un bucle que recibe datos en fragmentos de 512 caracteres del socket e imprime los datos hasta que no haya más datos para leer (es decir, recv() devuelve una cadena vacía).

El programa produce la siguiente salida:

```
HTTP/1.1 200 OK
Date: Sun, 14 Mar 2010 23:52:41 GMT
Server: Apache
Last-Modified: Tue, 29 Dec 2009 01:31:22 GMT
ETag: "143c1b33-a7-4b395bea"
Accept-Ranges: bytes
Content-Length: 167
Connection: close
Content-Type: text/plain

But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

La salida comienza con los encabezados que el servidor web envía para describir el documento. Por ejemplo, el encabezado `Content-Type` indica que el documento es un documento de texto sin formato (`text/plain`).

Una vez que el servidor nos envía los encabezados, agrega una línea en blanco para indicar el final de los encabezados y luego envía los datos reales del archivo `romeo.txt`.

Este ejemplo muestra cómo realizar una conexión de red de bajo nivel con sockets. Los sockets se pueden utilizar para comunicarse con un servidor web o con un servidor de correo o con muchos otros tipos de servidores. Todo lo que se necesita es encontrar el documento que describe el protocolo y escribir el código para enviar y recibir los datos de acuerdo con el protocolo.

Sin embargo, dado que el protocolo que usamos con mayor frecuencia es el protocolo web HTTP, Python tiene una biblioteca especial diseñada específicamente para admitir el protocolo HTTP para la recuperación de documentos y datos a través de la web.

# Recuperar una imagen sobre HTTP

En el ejemplo anterior, recuperamos un archivo de texto sin formato que tenía nuevas líneas en el archivo y simplemente copiamos los datos a la pantalla mientras se ejecutaba el programa. Podemos usar un programa similar para recuperar una imagen usando HTTP. En lugar de copiar los datos en la pantalla a medida que se ejecuta el programa, acumulamos los datos en una cadena, recortamos los encabezados y luego guardamos los datos de la imagen en un archivo de la siguiente manera:

https://trinket.io/embed/python3/ba6a2ee376

Cuando el programa se ejecuta produce el siguiente resultado:

```
$ python urljpeg.py
2920 2920
1460 4380
1460 5840
1460 7300
...
1460 62780
1460 64240
2920 67160
1460 68620
1681 70301
Header length 240
HTTP/1.1 200 OK
Date: Sat, 02 Nov 2013 02:15:07 GMT
Server: Apache
Last-Modified: Sat, 02 Nov 2013 02:01:26 GMT
ETag: "19c141-111a9-4ea280f8354b8"
Accept-Ranges: bytes
Content-Length: 70057
Connection: close
Content-Type: image/jpeg
```

Copyright 2026

Puedes ver que para esta URL, el encabezado `Content-Type` indica que el cuerpo del documento es una imagen (`image/jpeg`). Una vez que el programa se completa, puedes ver los datos de la imagen abriendo el archivo `stuff.jpg` en un visor de imágenes.

A medida que se ejecuta el programa, puedes ver que no obtenemos 5120 caracteres cada vez que llamamos al método `recv()`. Recibimos tantos caracteres como nos ha sido transferido a través de la red por el servidor web en el momento que llamamos `recv()`. En este ejemplo, obtenemos 1460 o 2920 caracteres cada vez que solicitamos hasta 5120 caracteres de datos.

Sus resultados pueden ser diferentes dependiendo de la velocidad de su red. También ten en cuenta que en la última llamada a `recv()` obtenemos 1681 bytes, que es el final de la secuencia, y en la siguiente llamada a `recv()` obtenemos una cadena de longitud cero que nos dice que el servidor ha llamado a `close()` en su extremo del socket y no hay más datos por venir.

Podemos ralentizar nuestras llamadas sucesivas `recv()` sin comentar la llamada a `time.sleep()`. De esta manera, esperamos un cuarto de segundo después de cada llamada para que el servidor pueda "adelantarse" y enviarnos más datos antes de que llamemos a `recv()` nuevamente. Con el retraso, en su lugar el programa se ejecuta de la siguiente manera:

```
$ python urljpeg.py
1460 1460
5120 6580
5120 11700
...
5120 62900
5120 68020
2281 70301
Header length 240
HTTP/1.1 200 OK
Date: Sat, 02 Nov 2013 02:22:04 GMT
Server: Apache
Last-Modified: Sat, 02 Nov 2013 02:01:26 GMT
ETag: "19c141-111a9-4ea280f8354b8"
Accept-Ranges: bytes
Content-Length: 70057
Connection: close
Content-Type: image/jpeg
```

Ahora, aparte de las primeras y últimas llamadas a `recv()`, ahora obtenemos 5120 caracteres cada vez que pedimos nuevos datos.

Hay un búfer entre el servidor que realiza las solicitudes `send()` y nuestra aplicación que realiza las solicitudes `recv()`. Cuando ejecutamos el programa con tiempos de espera, en algún momento el servidor podría llenar el búfer en el socket y ser forzado a pausar hasta que nuestro programa comience a vaciar el búfer. La pausa de la aplicación de envío o la aplicación de recepción se denomina "control de flujo".

# Recuperar páginas web con `urllib`

Si bien podemos enviar y recibir datos manualmente a través de HTTP utilizando la biblioteca de sockets, hay una forma mucho más sencilla de realizar esta tarea común en Python utilizando la biblioteca `urllib`.

Usando `urllib`, puedes tratar una página web como un archivo. Simplemente indique qué página web deseas recuperar y `urllib` se encarga de todos los detalles del protocolo HTTP y del encabezado.

El código equivalente para leer el archivo `romeo.txt` de la web usando`urllib` es el siguiente:

https://trinket.io/embed/python3/8ead88f3fa

Una vez que la página web se ha abierto con `urllib.urlopen`, podemos tratarla como un archivo y leerla utilizando un bucle `for`.

Cuando el programa se ejecuta, solo vemos la salida del contenido del archivo. Los encabezados aún se envían, pero el código `urllib` consume los encabezados y solo nos devuelve los datos.

```
But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

Como ejemplo, podemos escribir un programa para recuperar los datos para `romeo.txt` y calcular la frecuencia de cada palabra en el archivo de la siguiente manera:

https://trinket.io/embed/python3/6669c56892

Nuevamente, una vez que hemos abierto la página web, podemos leerla como un archivo local.

# Analizando HTML y raspando la web

Uno de los usos comunes de la capacidad `urllib` en Python es **raspar** la web. El raspado web (web scraping) tiene lugar cuando escribimos un programa que pretende ser un navegador web y recupera páginas, luego examina los datos en esas páginas en busca de patrones.

Como ejemplo, un motor de búsqueda como Google buscará en el origen de una página web y extraerá los enlaces a otras páginas y recuperará esas páginas, extrayendo enlaces, etc. Usando esta técnica, Google Spider se abre paso a través de casi todas las páginas de la web.

Google también utiliza la frecuencia de los enlaces de las páginas que encuentra a una página en particular como una medida de cuán "importante" es una página y qué tan alta debe aparecer la página en los resultados de búsqueda.

# Análisis HTML usando expresiones regulares

Una forma sencilla de analizar HTML es usar expresiones regulares para buscar y extraer subcadenas que coincidan con un patrón en particular.

Aquí hay una simple página web:

```
<h1>The First Page</h1>
<p>
If you like, you can switch to the
<a href="http://www.dr-chuck.com/page2.htm">
Second Page</a>.
</p>
```

Podemos construir una expresión regular bien formada para que coincida y extraiga los valores de enlace del texto anterior de la siguiente manera:

```
href="http://.+?"
```

Nuestra expresión regular busca cadenas que comiencen con `href="http://`, seguidas de uno o más caracteres (".+?"), seguidas de otra comilla doble. El signo de interrogación ".+? " indica que la coincidencia debe realizarse de manera "no codiciosa" en lugar de "codiciosa". Una coincidencia no codiciosa trata de encontrar la cadena de coincidencia **más pequeña** posible y una coincidencia codiciosa trata de encontrar la **más grande** posible cadena coincidente.

Agregamos paréntesis a nuestra expresión regular para indicar qué parte de nuestra cadena coincidente nos gustaría extraer y producir el siguiente programa:

https://trinket.io/embed/python3/436b82b9ea

El método de expresión regular `findall` nos dará una lista de todas las cadenas que coinciden con nuestra expresión regular, devolviendo solo el texto del enlace entre las comillas dobles.

Cuando ejecutamos el programa, obtenemos el siguiente resultado:

```
python urlregex.py
Enter - http://www.dr-chuck.com/page1.htm
http://www.dr-chuck.com/page2.htm
```

```
python urlregex.py
Enter - http://www.py4e.com/book.htm
http://www.greenteapress.com/thinkpython/thinkpython.html
http://allendowney.com/
http://www.py4e.com/code
http://www.lib.umich.edu/espresso-book-machine
http://www.py4e.com/py4inf-slides.zip
```

Las expresiones regulares funcionan muy bien cuando tu HTML está bien formateado y es predecible. Pero como hay muchas páginas HTML "rotas" por ahí, una solución que solo use expresiones regulares podría perder algunos enlaces válidos o terminar con datos erróneos.

Esto se puede resolver utilizando una biblioteca de análisis HTML robusta.

# Analizando HTML usando BeautifulSoup

Hay una serie de bibliotecas de Python que pueden ayudarte a analizar HTML y extraer datos de las páginas. Cada una de las bibliotecas tiene sus fortalezas y debilidades y puedes elegir una según tus necesidades.

Como ejemplo, simplemente analizaremos algunas entradas de HTML y extraeremos enlaces utilizando la biblioteca **BeautifulSoup**. Puedes descargar e instalar el código de BeautifulSoup abriendo una terminal y escribiendo `pip install bs4`

A pesar de que el HTML se parece a XML [1] y algunas páginas se construyen cuidadosamente para ser XML, la mayoría de HTML generalmente se rompe de manera que un analizador XML rechaza la página completa de HTML como incorrectamente formado. BeautifulSoup tolera HTML muy defectuoso y aún le permite extraer fácilmente los datos que necesita.

Usaremos `urllib` para leer la página y luego usaremos `BeautifulSoup` para extraer los atributos `href` de las etiquetas de anclaje (`a`).

https://trinket.io/embed/python3/d64a07806f

El programa solicita una dirección web, luego abre la página web, lee los datos y pasa los datos al analizador BeautifulSoup, y luego recupera todas las etiquetas de anclaje e imprime el atributo `href` para cada etiqueta.

Cuando el programa se ejecuta se ve como sigue:

```
python urllinks.py
Enter - http://www.dr-chuck.com/page1.htm
http://www.dr-chuck.com/page2.htm
```

```
python urllinks.py
Enter - http://www.py4e.com/book.htm
```

```
http://www.greenteapress.com/thinkpython/thinkpython.html
http://allendowney.com/
http://www.si502.com/
http://www.lib.umich.edu/espresso-book-machine
http://www.py4e.com/code
http://www.py4e.com/
```

Puedes usar BeautifulSoup para extraer varias partes de cada etiqueta de la siguiente manera:

https://trinket.io/embed/python3/0d6e91e4d7

```
python urllink2.py
Enter - http://www.dr-chuck.com/page1.htm
TAG: <a href="http://www.dr-chuck.com/page2.htm">
Second Page</a>
URL: http://www.dr-chuck.com/page2.htm
Content: ['\nSecond Page']
Attrs: [('href', 'http://www.dr-chuck.com/page2.htm')]
```

Estos ejemplos solo comienzan a mostrar el poder de BeautifulSoup cuando se trata de analizar HTML.

# Leyendo archivos binarios usando urllib

A veces deseas recuperar un archivo que no sea de texto (o binario), como un archivo de imagen o video. Los datos en estos archivos generalmente no son útiles para imprimir, pero puedes hacer fácilmente una copia de una URL a un archivo local en tu disco duro usando `urllib`.

El patrón es abrir la URL y usar `read` para descargar todo el contenido del documento en una variable de cadena (`img`) y luego escribir esa información en un archivo local de la siguiente manera:

https://trinket.io/embed/python3/5594c1233b

Este programa lee todos los datos de una sola vez a través de la red y los almacena en la variable `img` en la memoria principal de tu ordenador, luego abre el archivo `cover.jpg` y escribe los datos en tu disco. Esto funcionará si el tamaño del archivo es menor que el tamaño de la memoria de tu ordenador.

Sin embargo, si se trata de un gran archivo de audio o video, este programa puede fallar o, al menos, ejecutarse muy lentamente cuando el ordenador se queda sin memoria. Para evitar quedarse sin memoria, recuperamos los datos en bloques (o búferes) y luego escribimos cada bloque en tu disco antes de recuperar el siguiente bloque. De esta manera, el programa puede leer archivos de cualquier tamaño sin utilizar toda la memoria que tienes en tu ordenador.

https://trinket.io/embed/python3/a78adf3c8a

En este ejemplo, leemos solo 100,000 caracteres a la vez y luego escribimos esos caracteres en el archivo `cover.jpg` antes de recuperar los siguientes 100,000 caracteres de datos de la web.

Este programa se ejecuta de la siguiente manera:

```
python curl2.py
568248 characters copied.
```

Si tienes un ordenador Unix o Macintosh, es probable que tengas un comando integrado en tu sistema operativo que realice esta operación de la siguiente manera:

```
curl -O http://www.py4e.com/cover.jpg
```

El comando `curl` es la abreviatura de "copy URL", por lo que estos dos ejemplos se denominan `curl1.py` y `curl2.py` en www.py4e.com/code3 ya que implementan una funcionalidad similar a la del comando `curl`. También hay un programa de ejemplo `curl3.py` que realiza esta tarea con un poco más de eficacia, en caso de que realmente desees utilizar este patrón en un programa que estés escribiendo.

# Ejercicios

**Ejercicio 1:** Cambia el programa de socket `socket1.py` para solicitar al usuario la URL para que pueda leer cualquier página web. Puede usar `split('/')` para dividir la URL en sus componentes, de modo que puedas extraer el nombre de host para la llamada socket `connect`. Agrega la

comprobación de errores utilizando `try` y `except` para manejar la condición en la que el usuario ingresa una URL con formato incorrecto o inexistente.

**Ejercicio 2:** Cambia tu programa de socket para que cuente la cantidad de caracteres que ha recibido y deje de mostrar cualquier texto después de haber mostrado 3000 caracteres. El programa debe recuperar todo el documento, contar el número total de caracteres y mostrar el recuento del número de caracteres al final del documento.

**Ejercicio 3:** Usa `urllib` para replicar el ejercicio anterior de (1) recuperar el documento de una URL, (2) mostrar hasta 3000 caracteres, y (3) contar el número total de caracteres en el documento. No te preocups por los encabezados de este ejercicio, simplemente muestra los primeros 3000 caracteres del contenido del documento.

**Ejercicio 4:** Cambia el programa `urllinks.py` para extraer y contar las etiquetas de párrafo (p) del documento HTML recuperado y mostrar el recuento de los párrafos como salida de tu programa. No mostrar el texto del párrafo, solo contarlos. Prueba tu programa en varias páginas web pequeñas, así como en algunas páginas web más grandes.

**Ejercicio 5:** (Avanzado) Cambia el programa de socket para que solo muestre datos después de que se hayan recibido los encabezados y una línea en blanco. Recuerda que `recv` está recibiendo caracteres (nuevas líneas y demás), no líneas.

---

> [1]. El formato XML se describe en el siguiente capítulo. ↩

# 13 Python y servicios web

# Uso de servicios web

Una vez que fue fácil recuperar documentos y analizarlos a través de HTTP usando programas, no tomó mucho tiempo desarrollar un enfoque en el que comenzó a producir documentos que fueron diseñados específicamente para ser consumidos por otros programas (es decir, no HTML para mostrar en un navegador).

Hay dos formatos comunes que utilizamos cuando intercambiamos datos a través de la web. El "lenguaje de marcado extensible" o XML ha estado en uso durante mucho tiempo y es el más adecuado para intercambiar datos de tipo documento. Cuando los programas solo desean intercambiar diccionarios, listas u otra información interna entre ellos, utilizan la notación de objetos JavaScript o JSON (consulte [www.json.org] (http://www.json.org)). Veremos ambos formatos.

# Lenguaje de marcado extensible - XML

XML se ve muy similar a HTML, pero XML es más estructurado que HTML. Aquí hay una muestra de un documento XML:

```
<person>
  <name>Chuck</name>
  <phone type="intl">
     +1 734 303 4456
   </phone>
   <email hide="yes"/>
</person>
```

A menudo es útil pensar en un documento XML como una estructura de árbol donde hay una etiqueta superior "persona" y otras etiquetas como "teléfono" se crean como **hijos** de sus nodos padres.

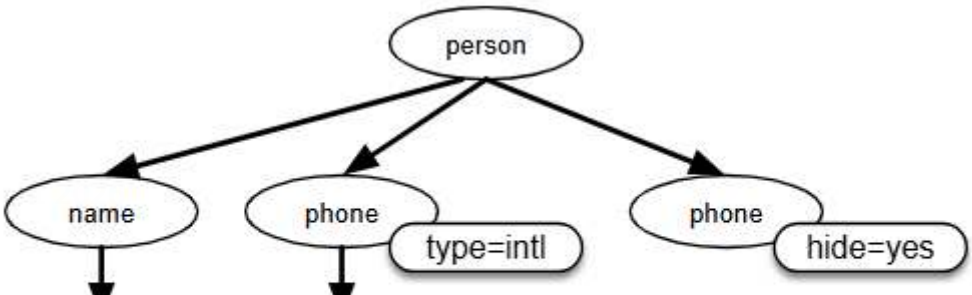


Imagen - Una representación en árbol de XML

Una representación en árbol de XML

# Analizando XML

Aquí hay una aplicación simple que analiza algunos XML y extrae algunos elementos de datos del XML:

https://trinket.io/embed/python3/b4dfccee06

`fromstring` convierte la representación de cadena del XML en un "árbol" de nodos XML. Cuando el XML está en un árbol, tenemos una serie de métodos a los que podemos llamar para extraer partes de datos del XML.

La función `find` busca en el árbol XML y recupera un **nodo** que coincide con la etiqueta especificada. Cada nodo puede tener algún texto, algunos atributos (como ocultar) y algunos nodos "secundarios". Cada nodo puede ser la parte superior de un árbol de nodos.

```
Name: Chuck
Attr: yes
```

El uso de un analizador XML como `ElementTree` tiene la ventaja de que si bien el XML en este ejemplo es bastante simple, resulta que hay muchas reglas con respecto a XML válido y el uso de `ElementTree` nos permite extraer datos de XML sin preocuparnos por las reglas de la sintaxis XML.

# Bucle a través de nodos

A menudo, el XML tiene varios nodos y necesitamos escribir un bucle para procesar todos los nodos. En el siguiente programa, recorremos todos los nodos `user`:

https://trinket.io/embed/python3/80ba384fbd

El método `findall` recupera una lista Python de subárboles que representan las estructuras `user` en el árbol XML. Luego podemos escribir un bucle `for` que mira cada uno de los nodos de usuario e imprime los elementos de texto `name` e `id`, así como el atributo`x` del nodo `user`.

```
User count: 2
Name Chuck
Id 001
Attribute 2
Name Brent
Id 009
Attribute 7
```

# Notación de objetos de JavaScript - JSON

El formato JSON se inspiró en el formato de objeto y matriz utilizado en el lenguaje JavaScript. Pero como Python se inventó antes de JavaScript, la sintaxis de Python para diccionarios y listas influyó en la sintaxis de JSON. Por lo tanto, el formato de JSON es casi idéntico a una combinación de listas y diccionarios de Python.

Aquí hay una codificación JSON que es aproximadamente equivalente al XML simple de arriba:

```
{
  "name" : "Chuck",
  "phone" : {
    "type" : "intl",
    "number" : "+1 734 303 4456"
   },
   "email" : {
     "hide" : "yes"
   }
}
```

Notarás algunas diferencias. Primero, en XML, podemos agregar atributos como "intl" a la etiqueta "phone". En JSON, simplemente tenemos pares clave-valor. Además, la etiqueta XML "persona" ha sido reemplazada por un conjunto de llaves exteriores.

En general, las estructuras JSON son más simples que las de XML porque JSON tiene menos prestaciones que le XML. Pero JSON tiene la ventaja de que mapea **directamente** a una combinación de diccionarios y listas. Y dado que casi todos los lenguajes de programación tienen algo equivalente a los diccionarios y listas de Python, JSON es un formato muy natural para que dos programas intercambien datos.

JSON se está convirtiendo rápidamente en el formato de elección para casi todo el intercambio de datos entre aplicaciones debido a su relativa simplicidad en comparación con XML.

# Parsing JSON

Construimos nuestro JSON al anidar diccionarios (objetos) y listas según sea necesario. En este ejemplo, representamos una lista de usuarios donde cada usuario es un conjunto de pares clave-valor (es decir, un diccionario). Así que tenemos una lista de diccionarios.

En el siguiente programa, usamos la biblioteca incorporada **json** para analizar el JSON y leer los datos. Compare esto de cerca con el código y los datos XML equivalentes de arriba. El JSON tiene menos detalles, por lo que debemos saber de antemano que estamos obteniendo una lista y que la lista es de usuarios y cada usuario es un conjunto de pares clave-valor. El JSON es más sucinto (una ventaja) pero también es menos autodescriptivo (una desventaja).

https://trinket.io/embed/python3/b540d86a7f

Si comparas el código para extraer datos de JSON y XML analizados, verás que lo que obtenemos de **json.loads()** es una lista de Python que recorremos con un bucle `for`, y cada elemento dentro de esa lista es un diccionario de Python. Una vez que se hayas analizado el JSON, podemos utilizar el operador de índice de Python para extraer los distintos datos de cada usuario. No tenemos que usar la biblioteca JSON para explorar el JSON analizado, ya que los datos devueltos son simplemente estructuras nativas de Python.

La salida de este programa es exactamente la misma que la versión XML anterior.

```
User count: 2
Name Chuck
Id 001
Attribute 2
Name Brent
```

```
Id 009
Attribute 7
```

En general, hay una tendencia de la industria a alejarse del XML y apostar por JSON para servicios web. Debido a que el JSON es más simple y se asigna más directamente a las estructuras de datos nativos que ya tenemos en los lenguajes de programación, el código de análisis y extracción de datos suele ser más simple y más directo cuando se utiliza JSON. Pero XML es más autodescriptivo que JSON y, por lo tanto, hay algunas aplicaciones en las que XML conserva una ventaja. Por ejemplo, la mayoría de los procesadores de texto almacenan documentos internamente utilizando XML en lugar de JSON.

# Interfaces de programación de aplicaciones

Ahora tenemos la capacidad de intercambiar datos entre aplicaciones mediante el Protocolo de transporte de hipertexto (HTTP) y una forma de representar datos complejos que enviamos de un lado a otro entre estas aplicaciones mediante el lenguaje de marcado extensible (XML) o la notación de objetos de JavaScript (JSON).

El siguiente paso es comenzar a definir y documentar los "contratos" entre las aplicaciones que utilizan estas técnicas. El nombre general de estos contratos de aplicación a aplicación es API (Aplication Programming Interfaces o *Interfaces de Programación de Aplicaciones*). Cuando usamos una API, generalmente un programa hace que un conjunto de **servicios** esté disponible para su uso por parte de otras aplicaciones, publicando las API (es decir, las "reglas") que deben seguirse para acceder a los servicios proporcionados por el programa.

Cuando comenzamos a construir nuestros programas donde la funcionalidad de nuestro programa incluye el acceso a los servicios proporcionados por otros programas, llamamos a este enfoque **Arquitectura Orientada a ServiciosAPI** o SOA. Un enfoque SOA es uno en el que nuestra aplicación general hace uso de los servicios de otras aplicaciones. Un enfoque que no es SOA es cuando la aplicación es una única aplicación independiente que contiene todo el código necesario para implementar la aplicación.

Vemos muchos ejemplos de SOA cuando usamos la web. Podemos ir a un solo sitio web y reservar viajes aéreos, hoteles y automóviles desde un solo sitio. Los datos de los hoteles no se almacenan en los ordenadores de la aerolínea. En su lugar, los ordenadores de la línea aérea se ponen en contacto con los servicios de los ordenadores del hotel, recuperan los datos del hotel y los presentan al usuario. Cuando el usuario acepta hacer una reserva de hotel utilizando el sitio de la

aerolínea, el sitio de la línea aérea utiliza otro servicio web en los sistemas del hotel para realizar la reserva. Y cuando llega el momento de cargar su tarjeta de crédito por toda la transacción, aún otros ordenadores se involucran en el proceso.

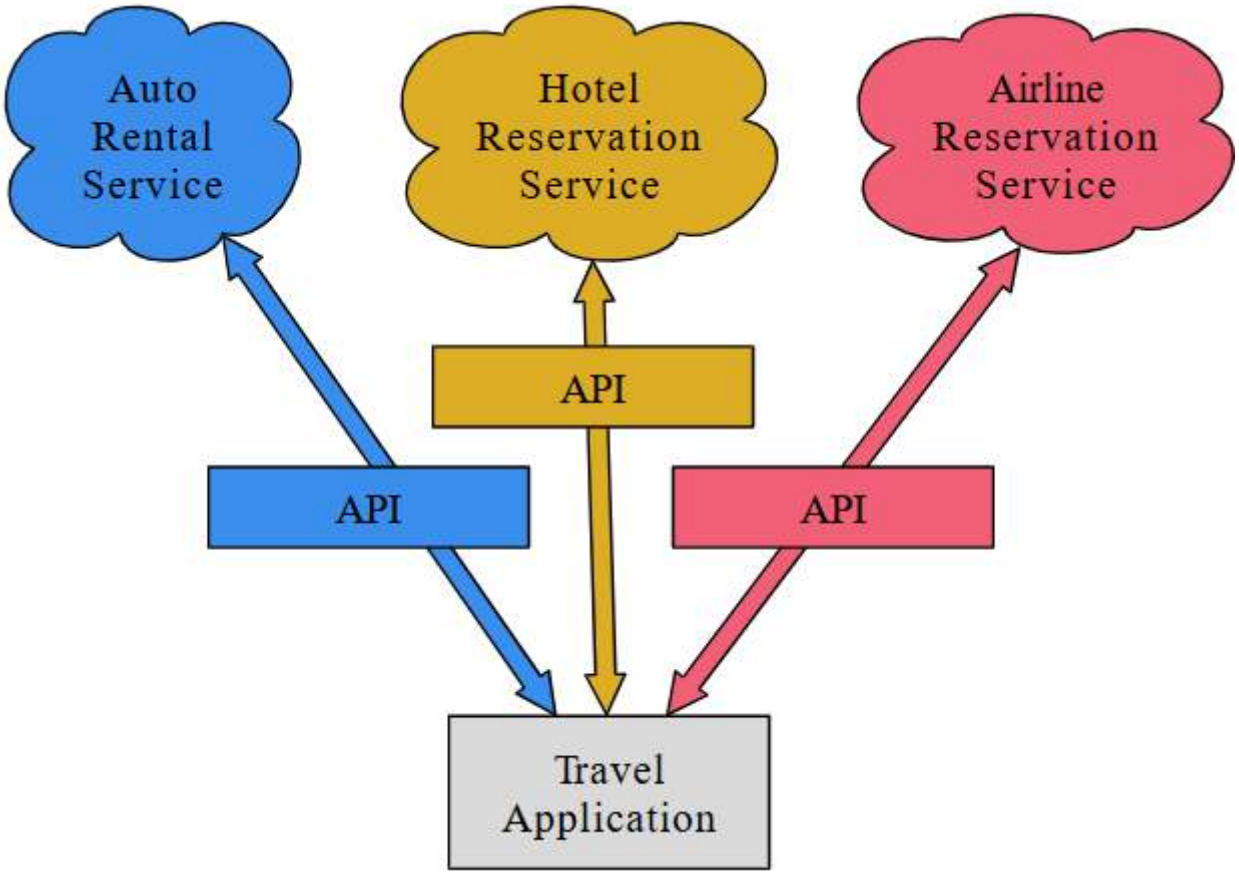



Imagen - Arquitectura orientada a Servicios

Una arquitectura orientada a servicios tiene muchas ventajas, entre ellas: (1) siempre mantenemos una sola copia de datos (esto es particularmente importante para cosas como reservas de hoteles donde no queremos comprometernos en exceso) y (2) los propietarios de los datos pueden establecer las reglas sobre el uso de sus datos. Con estas ventajas, un sistema SOA debe diseñarse cuidadosamente para tener un buen rendimiento y satisfacer las necesidades del usuario.

Cuando una aplicación hace que un conjunto de servicios en su API esté disponible en la web, llamamos a éstos **servicios web**.

# Seguridad y uso de API

Es bastante común que necesiteS algún tipo de "clave API" para hacer uso de la API de un proveedor. La idea general es que quieren saber quién está usando sus servicios y cuánto está usando cada usuario. Tal vez tengan niveles de servicios gratuitos y de pago, o tengan una política

que limite la cantidad de solicitudes que una sola persona puede realizar durante un período de tiempo determinado.

A veces, una vez que obtienes tu clave de API, simplemente incluyes la clave como parte de los datos POST o tal vez como un parámetro en la URL al llamar a la API.

Otras veces, el proveedor desea una mayor seguridad de la fuente de las solicitudes y, por lo tanto, agregan que esperan que usted envíe mensajes firmados criptográficamente con claves y secretos compartidos. Una tecnología muy común que se utiliza para firmar solicitudes a través de Internet se llama **OAuth**. Puede leer más sobre el protocolo OAuth en [www.oauth.net] ( http://www.oauth.net).

A medida que la API de Twitter se hizo cada vez más valiosa, Twitter pasó de una API abierta y pública a una API que requería el uso de firmas OAuth en cada solicitud de API. Afortunadamente, todavía hay una serie de bibliotecas OAuth convenientes y gratuitas para que pueda evitar escribir una implementación de OAuth desde cero leyendo la especificación. Estas bibliotecas son de complejidad variable y tienen diversos grados de riqueza. El sitio web de OAuth tiene información sobre varias bibliotecas de OAuth.

Para este próximo programa de muestra, descargaremos los archivos **twurl.py**, **hidden.py**, **oauth.py** y **twitter1.py** de www.py4e.com/code3 y póngalos en una carpeta en su ordenador.

Para utilizar estos programas, necesitarás tener una cuenta de Twitter y autorizar su código Python como una aplicación, configurar una clave, un secreto, un token y un token secreto. Editarás el archivo **hidden.py** y colocarás estas cuatro cadenas en las variables apropiadas en el archivo:

https://trinket.io/embed/python3/ab6352e0a3

Se accede al servicio web de Twitter usando una URL como esta:

https://api.twitter.com/1.1/statuses/user_timeline.json

Pero una vez que se haya agregado toda la información de seguridad, la URL se verá más como:

```
https://api.twitter.com/1.1/statuses/user_timeline.json?count=2
&oauth_version=1.0&oauth_token=101...SGI&screen_name=drchuck
&oauth_nonce=09239679&oauth_timestamp=1380395644
&oauth_signature=rLK...BoD&oauth_consumer_key=h7Lu...GNg
```

```
&oauth_signature_method=HMAC-SHA1
```

Puedes leer la especificación de OAuth si deseas saber más sobre el significado de los diversos parámetros que se agregan para cumplir con los requisitos de seguridad de OAuth.

Para los programas que ejecutamos con Twitter, ocultamos toda la complejidad de los archivos **oauth.py** y **twurl.py**. Simplemente configuramos los secretos en **hidden.py** y luego enviamos la URL deseada a la función **twurl.augment()** y el código de la biblioteca agrega todos los parámetros necesarios a la URL para nosotros.

Este programa recupera la línea de tiempo para un usuario de Twitter en particular y nos la devuelve en formato JSON en una cadena. Simplemente imprimimos los primeros 250 caracteres de la cadena:

```
import urllib.request, urllib.parse, urllib.error
import twurl
import ssl

# https://apps.twitter.com/
# Create App and get the four strings, put them in hidden.py

TWITTER_URL = 'https://api.twitter.com/1.1/statuses/user_timeline.json'

# Ignore SSL certificate errors
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE

while True:
    print('')
    acct = input('Enter Twitter Account:')
    if (len(acct) < 1): break
    url = twurl.augment(TWITTER_URL,
                        {'screen_name': acct, 'count': '2'})
    print('Retrieving', url)
    connection = urllib.request.urlopen(url, context=ctx)
    data = connection.read().decode()
    print(data[:250])
```

```
    headers = dict(connection.getheaders())
    # print headers
    print('Remaining', headers['x-rate-limit-remaining'])

# Code: http://www.py4e.com/code3/twitter1.py
# Or select Download from this trinket's left-hand menu
```

Cuando el programa se ejecuta produce el siguiente resultado:

```
Enter Twitter Account:drchuck
Retrieving https://api.twitter.com/1.1/ ...
[{"created_at":"Sat Sep 28 17:30:25 +0000 2013","
id":384007200990982144,"id_str":"384007200990982144",
"text":"RT @fixpert: See how the Dutch handle traffic
intersections: http:\/\/t.co\/tIiVWtEhj4\n#brilliant",
"source":"web","truncated":false,"in_rep
Remaining 178

Enter Twitter Account:fixpert
Retrieving https://api.twitter.com/1.1/ ...
[{"created_at":"Sat Sep 28 18:03:56 +0000 2013",
"id":384015634108919808,"id_str":"384015634108919808",
"text":"3 months after my freak bocce ball accident,
my wedding ring fits again! :)\n\nhttps:\/\/t.co\/2XmHPx7kgX",
"source":"web","truncated":false,
Remaining 177

Enter Twitter Account:
```

Junto con los datos de línea de tiempo devueltos, Twitter también devuelve metadatos sobre la solicitud en los encabezados de respuesta HTTP. Un encabezado en particular, **x-rate-limit-restantes**, nos informa cuántas solicitudes más podemos hacer antes de que podamos cerrarnos por un corto período de tiempo. Puedes ver que nuestras recuperaciones restantes se reducen en una cada vez que realizamos una solicitud a la API.

En el siguiente ejemplo, recuperamos los amigos de Twitter de un usuario, analizamos el JSON devuelto y extraemos parte de la información sobre los amigos. También volcamos el JSON después de analizarlo y lo "imprimimos" con una sangría de cuatro caracteres para permitirnos

analizar los datos cuando queremos extraer más campos.

https://trinket.io/embed/python3/54ce78fb02

Dado que JSON se convierte en un conjunto de listas y diccionarios Python anidados, podemos usar una combinación de la operación de índice y los bucles `for` para recorrer las estructuras de datos devueltos con muy poco código Python.

El resultado del programa es el siguiente (algunos de los elementos de datos se acortan para ajustarse a la página):

```
Enter Twitter Account:drchuck
Retrieving https://api.twitter.com/1.1/friends ...
Remaining 14
```

```
{
    "next_cursor": 1444171224491980205,
    "users": [
        {
            "id": 662433,
            "followers_count": 28725,
            "status": {
                "text": "@jazzychad I just bought one .__.",
                "created_at": "Fri Sep 20 08:36:34 +0000 2013",
                "retweeted": false,
            },
            "location": "San Francisco, California",
            "screen_name": "leahculver",
            "name": "Leah Culver",
        },
        {
            "id": 40426722,
            "followers_count": 2635,
            "status": {
                "text": "RT @WSJ: Big employers like Google ...",
```

```
                "created_at": "Sat Sep 28 19:36:37 +0000 2013",
            },
            "location": "Victoria Canada",
            "screen_name": "_valeriei",
            "name": "Valerie Irvine",
    ],
    "next_cursor_str": "1444171224491980205"
}
```

```
leahculver
   @jazzychad I just bought one .__.
_valeriei
   RT @WSJ: Big employers like Google, AT&amp;amp;T are h
ericbollens
   RT @lukew: sneak peek: my LONG take on the good &amp;a
halherzog
   Learning Objects is 10. We had a cake with the LO,
scweeker
   @DeviceLabDC love it! Now where so I get that "etc

Enter Twitter Account:
```

El último bit de la salida es donde vemos el bucle for leyendo los cinco "amigos" más recientes de la cuenta de Twitter **drchuck** e imprimiendo el estado más reciente de cada amigo. Hay muchos más datos disponibles en el JSON devuelto. Si observas la salida del programa, también puedes ver que el "buscar amigos" de una cuenta en particular tiene una limitación de tasa diferente a la cantidad de consultas en la línea de tiempo que podemos ejecutar por período de tiempo.

Estas claves de API seguras permiten que Twitter tenga una sólida confianza de que saben quién está usando su API y sus datos y en qué nivel. El enfoque de limitación de velocidad nos permite realizar recuperaciones de datos personales simples, pero no nos permite crear un producto que extraiga datos de su API millones de veces al día.

# Ejercicios

**Ejercicio 1:** Cambie el www.py4e.com/code3/geojson.py o www.py4e.com/code3/geoxml.py para imprimir el código de país de dos caracteres a partir de los datos recuperados. Agregue la verificación de errores para que su programa no se registre si el código de país no está allí. Una vez que lo tenga funcionando, busque "Océano Atlántico" y asegúrese de que pueda manejar ubicaciones que no estén en ningún país.

# 14 Objetos

## Administrar programas más grandes

Al comienzo de este libro, presentamos cuatro patrones de programación básicos que utilizamos para construir programas:

- Código secuencial
- Código condicional (si las declaraciones)
- Código iterativo (bucles)
- Almacenar y reutilizar (funciones)

Después exploramos variables simples y estructuras de datos de recopilación como listas, tuplas y diccionarios.

A medida que construimos programas, diseñamos estructuras de datos y escribimos código para manipular esas estructuras de datos. Hay muchas formas de escribir programas y, a estas alturas, es probable que hayas escrito algunos programas que "no son tan elegantes" y otros que son "más elegantes". A pesar de que tus programas pueden ser pequeños, estás empezando a ver cómo escribir código tiene un poco de "arte" y "estética".

A medida que los programas alcanzan millones de líneas, cada vez es más importante escribir código que sea fácil de entender. Si estás trabajando en un programa de un millón de líneas, nunca podrás tener todo el programa en tu mente al mismo tiempo. Por lo tanto, necesitamos formas de dividir el programa en múltiples partes más pequeñas para resolver un problema, corregir un error o agregar una nueva función que tenemos menos que ver.

En cierto modo, la programación orientada a objetos es una forma de organizar tu código para que puedas hacer zoom en 500 líneas del código y entenderlo mientras ignoras las otras 999,500 líneas de código por el momento.

## Comenzando

Al igual que muchos aspectos de la programación, es necesario aprender los conceptos de programación orientada a objetos antes de poder usarlos de manera efectiva. Así que enfoca este capítulo como una manera de aprender algunos términos y conceptos y trabaja con algunos

ejemplos simples para sentar las bases de un aprendizaje futuro. En el resto del libro usaremos objetos en muchos de los programas, pero no construiremos nuestros propios objetos nuevos en los programas.

El resultado clave de este capítulo es tener una comprensión básica de cómo se construyen los objetos y cómo funcionan y, lo que es más importante, cómo hacemos uso de las capacidades de los objetos que nos proporcionan las libreraís de Python y el propio Python.

# Usando objetos

Resulta que hemos estado utilizando objetos a lo largo de este curso. Python nos proporciona muchos objetos incorporados. Aquí hay un código simple donde las primeras líneas deben ser muy simples y naturales para ti.

https://trinket.io/embed/python3/27adad9e85

Pero en lugar de centrarse en lo que logran estas líneas, echemos un vistazo a lo que realmente está sucediendo desde el punto de vista de la programación orientada a objetos. No te preocupes si los siguientes párrafos no tienen ningún sentido la primera vez que los lees porque todavía no hemos definido todos estos términos.

La primera línea es **construyendo** un objeto del tipo **lista**, la segunda y la tercera línea están llamando al método `append()`, la cuarta línea está llamando al método `sort ()`, y la quinta línea está recuperando el artículo en la posición 0.

La sexta línea está llamando al método `__getitem__()` en la lista `stuff` con un parámetro de cero.

```
print (stuff.__getitem__(0))
```

La séptima línea es una forma aún más detallada de recuperar el artículo 0 en la lista.

```
print (list.__getitem__(stuff,0))
```

En este código, lamamos al método `__getitem__` en la clase `list` y pasamos la lista (`stuff`) y el elemento que queremos recuperar de la lista como parámetros.

Las últimas tres líneas del programa son completamente equivalentes, pero es más conveniente simplemente usar la sintaxis del corchete para buscar un elemento en una posición particular en

una lista.

Podemos observar las capacidades de un objeto mirando la salida de la función `dir()`:

```
>>> stuff = list()
>>> dir(stuff)
['__add__', '__class__', '__contains__', '__delattr__',
'__delitem__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattribute__', '__getitem__',
'__gt__', '__hash__', '__iadd__', '__imul__', '__init__',
'__iter__', '__le__', '__len__', '__lt__', '__mul__',
'__ne__', '__new__', '__reduce__', '__reduce_ex__',
'__repr__', '__reversed__', '__rmul__', '__setattr__',
'__setitem__', '__sizeof__', '__str__', '__subclasshook__',
'append', 'clear', 'copy', 'count', 'extend', 'index',
'insert', 'pop', 'remove', 'reverse', 'sort']
>>>
```

La definición precisa de `dir()` es que enumera los **métodos** y **atributos** de un objeto Python.

El resto de este capítulo definirá todos los términos anteriores, así que asegúrate de regresar después de terminar el capítulo y volver a leer los párrafos anteriores para verificar su comprensión.

# Comenzando con programas

Un programa en su forma más básica toma algo de entrada, realiza algún procesamiento y produce algo de salida. Nuestro programa de conversión entre distintos sistemas de numeración de plantas, muestra un programa muy corto pero completo en tres pasos.

https://trinket.io/embed/python3/d3c8ba77d5

Si pensamos un poco más en este programa, existe el "mundo exterior" y el programa. Los puntos de entrada y salida son donde el programa interactúa con el mundo exterior. Dentro del programa, tenemos código y datos para realizar la tarea que el programa está diseñado para resolver.

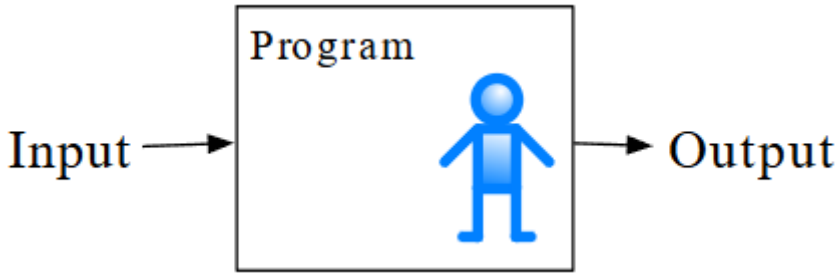



Imagen - Un programa

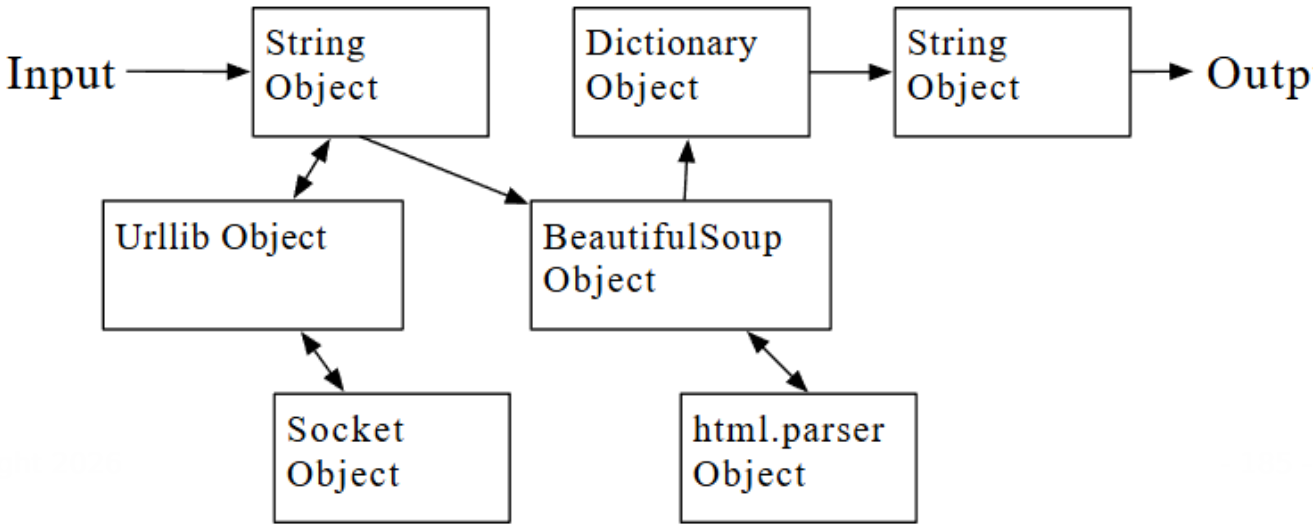
Cuando estamos "en" el programa, tenemos algunas interacciones definidas con el mundo "externo", pero esas interacciones están bien definidas y generalmente no son algo en lo que nos enfocamos. Mientras estamos programando, nos preocupamos solo por los detalles "dentro del programa".

Una forma de pensar acerca de la programación orientada a objetos es que estamos separando nuestro programa en múltiples "zonas". Cada "zona" contiene algunos códigos y datos (como un programa) y tiene interacciones bien definidas con el mundo exterior y las otras zonas dentro del programa.

Si miramos hacia atrás en la aplicación de extracción de enlaces donde usamos la biblioteca BeautifulSoup, podemos ver un programa que se construye conectando diferentes objetos para realizar una tarea:

https://trinket.io/embed/python3/9bb573e676

Leemos la URL en una cadena y luego la pasamos a `urllib` para recuperar los datos de la web. La biblioteca `urllib` usa la biblioteca `socket` para hacer la conexión de red real para recuperar los datos. Tomamos la cadena que recibimos de `urllib` y la entregamos a BeautifulSoup para que la analice. BeautifulSoup utiliza otro objeto llamado `html.parser`[1] y devuelve un objeto. Llamamos al método `tags()` en el objeto devuelto y luego obtenemos un diccionario de objetos `etiquetas`, hacemos un bucle a través de las etiquetas y llamamos al método `get()` para que cada etiqueta imprima el atributo 'href'.




Copyr

Imagen - Un programa como red de objetos

Podemos dibujar una imagen de este programa y cómo los objetos trabajan juntos.

La clave aquí no es entender completamente cómo funciona este programa, sino ver cómo construimos una red de objetos interactivos y organizamos el movimiento de información entre los objetos para crear un programa. También es importante tener en cuenta que cuando miraste ese programa varios capítulos atrás, podías entender completamente lo que estaba sucediendo en el programa sin siquiera darte cuenta de que el programa estaba "orquestando el movimiento de datos entre objetos". Entonces solo eran líneas de código que hacían el trabajo.

# Subdivisión de un problema: encapsulación

Una de las ventajas del enfoque orientado a objetos es que puede ocultar la complejidad. Por ejemplo, aunque necesitamos saber cómo usar el código `urllib` y BeautifulSoup, no necesitamos saber cómo funcionan internamente esas bibliotecas. Nos permite enfocarnos en la parte del problema que necesitamos para resolver e ignorar las otras partes del programa.

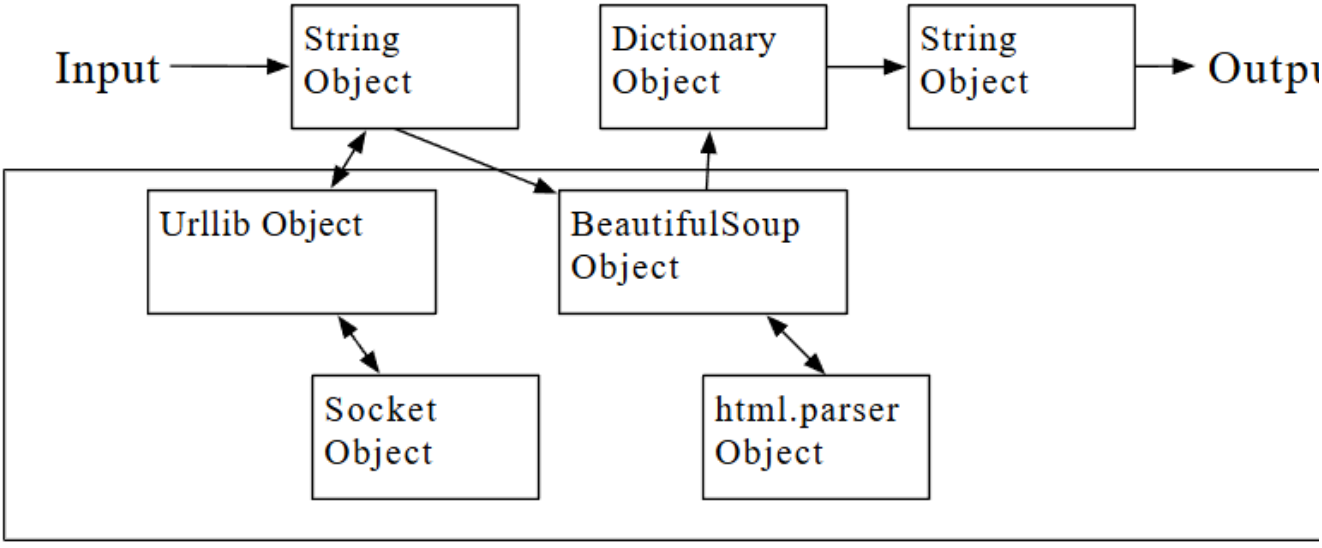



Imagen - Ignorar los detalles al usar un objeto

Esta capacidad de centrarse en una parte de un programa que nos importa e ignorar el resto del programa también es útil para los desarrolladores de los objetos. Por ejemplo, los programadores que desarrollan BeautifulSoup no necesitan saber ni preocuparse sobre cómo recuperamos nuestra página HTML, qué partes queremos leer o qué planeamos hacer con los datos que extraemos de la página web.

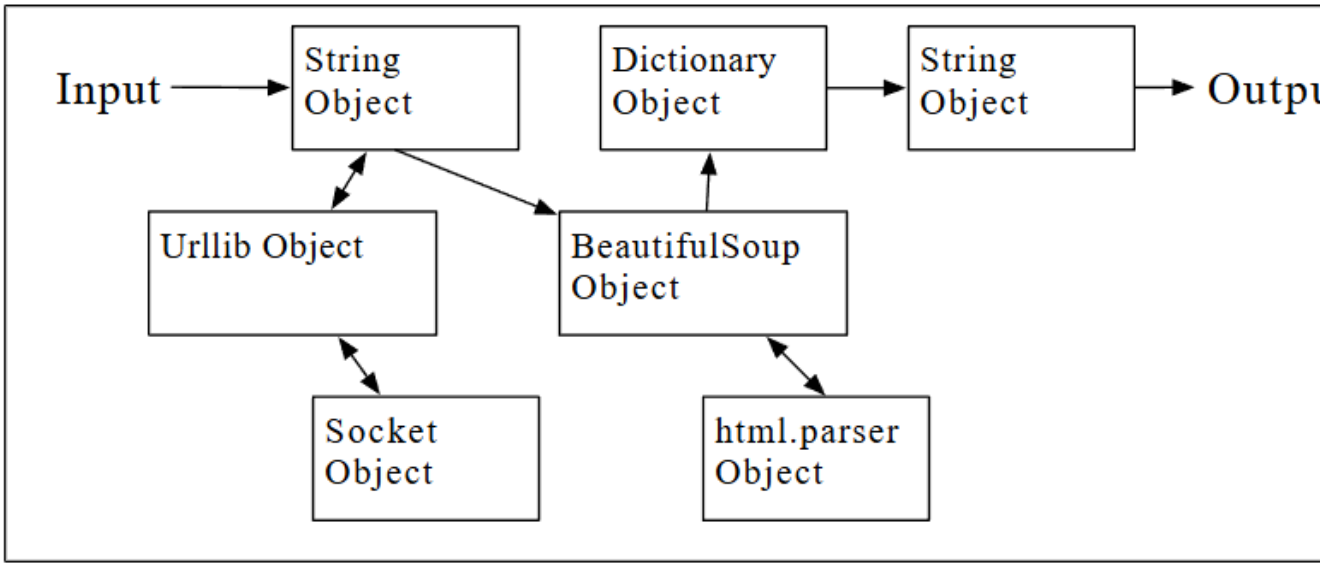



Imagen - Ignorar los detalles al construir un objeto

A esta idea de ignorar los detalles internos de los objetos que usamos también la llamamos "encapsulación". Esto significa que podemos saber cómo usar un objeto sin saber cómo cumple internamente lo que debemos hacer.

# Nuestro primer objeto Python

En su forma más simple, un objeto es un código, con estructuras de datos, que es más pequeño que un programa completo. Definir una función nos permite almacenar un poco de código y darle un nombre y luego invocar ese código usando el nombre de la función.

Un objeto puede contener una serie de funciones (que denominamos "métodos"), así como los datos que utilizan esas funciones. A los elementos de datos que forman parte del objeto los llamamos "atributos".

Usamos la palabra clave `class` para definir los datos y el código que conformará cada uno de los objetos. La palabra clave class incluye el nombre de la clase y comienza un bloque de código con sangría donde incluimos los atributos (datos) y los métodos (funciones).

https://trinket.io/embed/python3/b88be451e5

Cada método se parece a una función, comenzando con la palabra clave `def` y consiste en un bloque de código con sangría. Este ejemplo tiene un atributo (x) y un método (party). Los métodos tienen un primer parámetro especial que nombramos por convención `self`.

Al igual que la palabra clave `def` no hace que se ejecute el código de función, la palabra clave `class` no crea un objeto. En cambio, la palabra clave `class` define una plantilla que indica qué datos y código estarán contenidos en cada objeto de tipo `PartyAnimal`. La clase es como un cortador de galletas y los objetos creados usando la clase son las propias galletas[2]. No eches azucar glass en el cortador de galletas, espolvoreando el glaseado en las galletas y puedes realizar un glaseado diferente en cada galleta.



Imagen - Una clase y dos objetos

Si continúa con el código de ejemplo, vemos la primera línea de código ejecutable:

```
an = PartyAnimal()
```

Aquí es donde le pedimos a Python que construya un **objeto** o "instancia de la clase llamada PartyAnimal". Parece una llamada de función a la clase en sí misma y Python construye el objeto con los métodos y datos correctos, devolviendo el objeto que luego se asigna a la variable `an`. En cierto modo, esto es bastante similar a la siguiente línea que hemos estado usando todo este tiempo:

```
counts = dict()
```

Aquí le estamos indicando a Python que construya un objeto usando la plantilla `dict` (ya presente en Python), devuelva la instancia del diccionario y la asigne a la variable `count`.

Cuando la clase PartyAnimal se usa para construir un objeto, la variable `an` se usa para apuntar a ese objeto. Usamos `an` para acceder al código y los datos de esa instancia particular de tipo PartyAnimal.

Cada objeto/instancia de Partyanimal contiene en su interior una variable `x` y un método/función llamado `party`. Llamamos a ese método `party` en esta línea:

```
an.party()
```

Cuando se llama al método `party`, el primer parámetro (al que llamamos por convención `self`) apunta a la instancia particular del objeto PartyAnimal que llama a `party` desde dentro. Dentro del método `party`, vemos la línea:

```
self.x = self.x + 1
```

Esta sintaxis que usa el operador 'punto' está diciendo 'la x dentro de uno mismo'. Entonces, cada vez que se llamas a `party()`, el valor interno de `x` se incrementa en 1 y el valor se imprime.

Para ayudar a entender la diferencia entre una función global y un método dentro de una clase / objeto, la siguiente línea es otra forma de llamar al método `party` dentro del objeto `an`:

```
PartyAnimal.party(an)
```

En esta variación, accedemos al código desde dentro de la **clase** y pasamos explícitamente el puntero del objeto `an` como primer parámetro (es decir, `self` dentro del método). Puedes pensar en `an.party()` como una abreviatura de la línea anterior.

Cuando el programa se ejecuta, produce el siguiente resultado:

```
So far 1
So far 2
So far 3
So far 4
```

El objeto se construye y el método `party` se llama cuatro veces, incrementando e imprimiendo el valor de `x` dentro del objeto `an`.

# Clases como tipos

Como hemos visto, en Python, todas las variables tienen un tipo. Y podemos usar la función `dir` incorporada para examinar las capacidades de una variable. Podemos usar `type` y `dir` con las clases que creamos.

https://trinket.io/embed/python3/a502e6571d

Cuando este programa se ejecuta, produce la siguiente salida:

```
Type <class '__main__.PartyAnimal'>
Dir  ['__class__', '__delattr__', ...
'__sizeof__', '__str__', '__subclasshook__',
'__weakref__', 'party', 'x']
Type <class 'int'>
Type <class 'method'>
```

Puedes ver que usando la palabra clave `class`, hemos creado un nuevo tipo. Desde la salida de `dir`, puede ver que tanto el atributo de entero `x` como el método `party` están disponibles en el objeto.

# Object Lifecycle

En los ejemplos anteriores, estamos definiendo una clase (plantilla) y usando esa clase para crear una instancia de esa clase (objeto) y luego usando la instancia. Cuando el programa termina, todas las variables se descartan. Por lo general, no pensamos mucho en la creación y destrucción de variables, pero a menudo, a medida que nuestros objetos se vuelven más complejos, debemos tomar alguna acción dentro del objeto para configurar las cosas a medida que se está construyendo el objeto y posiblemente limpiar las cosas a medida que objeto está siendo descartado.

Copyright 2026

Si queremos que nuestro objeto sea consciente de estos momentos de construcción y destrucción, agregamos métodos especialmente nombrados a nuestro objeto:

https://trinket.io/embed/python3/75323d8f81

Cuando este programa se ejecuta, produce la siguiente salida:

```
I am constructed
So far 1
So far 2
I am destructed 2
an contains 42
```

Como Python está construyendo nuestro objeto, llama a nuestro método `__init__` para darnos la oportunidad de configurar algunos valores predeterminados o iniciales para el objeto. Cuando Python encuentra la línea:

```
an = 42
```

... en realidad, "elimina nuestro objeto", reutilizando la variable `an` para almacenar el valor `42`. Justo en el momento en que nuestro objeto `an` se está 'destruyendo', se llama a nuestro código destructor (`__del__`). No podemos evitar que nuestra variable se destruya, pero podemos hacer cualquier limpieza necesaria justo antes de que nuestro objeto ya no exista.

Cuando se desarrollan objetos, es bastante común agregar un constructor a un objeto para que se establezca en los valores iniciales del objeto, pero es relativamente raro que se necesite un destructor para un objeto.

# Muchas instancias

Hasta ahora, hemos estado definiendo una clase, creando un solo objeto, utilizando ese objeto y luego desechando el objeto. Pero el poder real en la orientación a objetos ocurre cuando hacemos muchos ejemplos de nuestra clase.

Cuando creamos varios objetos de nuestra clase, es posible que deseamos configurar diferentes valores iniciales para cada uno de los objetos. Podemos pasar datos a los constructores para dar a cada objeto un valor inicial diferente:

https://trinket.io/embed/python3/a1464da1b6

El constructor tiene un parámetro 'self' que apunta a la instancia del objeto y luego los parámetros adicionales que se pasan al constructor a medida que se construye el objeto:

```
s = PartyAnimal('Sally')
```

Dentro del constructor, la línea:

```
self.name = nam
```

Copia el parámetro que se pasa (`nam`) en el atributo `name` dentro de la instancia del objeto.

La salida del programa muestra que cada uno de los objetos (`s` y `j`) contienen sus propias copias independientes de `x` y `nam`:

```
Sally constructed
Sally party count 1
Jim constructed
Jim party count 1
Sally party count 2
```

# Herencia

Otra característica poderosa de la programación orientada a objetos es la capacidad de crear una nueva clase al extender una clase existente. Al extender una clase, llamamos a la clase original "clase principal" y a la nueva clase como "clase secundaria".

Para este ejemplo, moveremos nuestra clase `PartyAnimal` a su propio archivo:

https://trinket.io/embed/python3/859f1006a3

Luego, podemos 'importar' la clase `PartyAnimal` en un nuevo archivo y extenderlo de la siguiente manera:

https://trinket.io/embed/python3/980f08c259

Cuando estamos definiendo el objeto `CricketFan`, indicamos que estamos extendiendo la clase `PartyAnimal`. Esto significa que todas las variables (`x`) y métodos (`party`) de la clase `PartyAnimal` son heredadas por la clase `CricketFan`.

Puedes ver que dentro del método `six` en la clase `CricketFan`, podemos llamar al método `party` desde la clase `PartyAnimal`. Las variables y los métodos de la clase principal se **fusionan** en la clase secundaria.

A medida que se ejecuta el programa, podemos ver que `s` y `j` son instancias independientes de `PartyAnimal` y `CricketFan`. El objeto `j` tiene capacidades adicionales más allá del objeto `s`.

```
Sally constructed
Sally party count 1
Jim constructed
Jim party count 1
Jim party count 2
Jim points 6
['__class__', '__delattr__', ... '__weakref__',
'name', 'party', 'points', 'six', 'x']
```

En la salida de `dir` para el objeto `j` (instancia de la clase `CricketFan`) puedes ver que tiene los atributos y métodos de la clase principal, así como los atributos y métodos que se agregaron cuando la clase era ampliada para crear la clase `CricketFan`.

# Resumen

Esta es una introducción muy rápida a la programación orientada a objetos que se centra principalmente en la terminología y la sintaxis de definir y utilizar objetos. Revisemos rápidamente el código que vimos al principio del capítulo. En este punto, debes comprender completamente lo que está sucediendo.

https://trinket.io/embed/python3/ee59ee4759

La primera línea construye un objeto `list`. Cuando Python crea el objeto `list`, llama al método **constructor** (denominado `__init__`) para configurar los atributos de datos internos que se utilizarán para almacenar los datos de la lista. Debido a la **encapsulación** no necesitamos saber, ni tenemos que preocuparnos por esto, ya que los atributos de datos internos están ordenados.

No estamos pasando ningún parámetro al **constructor** y cuando el constructor regresa, usamos la variable `stuff` para apuntar a la instancia devuelta de la clase `list`.

Las líneas segunda y tercera están llamando al método `append` con un parámetro para agregar un nuevo elemento al final de la lista actualizando los atributos dentro de `stuff`. Luego, en la cuarta línea, llamamos al método `sort` sin parámetros para ordenar los datos dentro del objeto `stuff`.

Luego imprimimos el primer elemento de la lista usando los corchetes, que son un atajo para llamar al método `__getitem__` dentro del objeto `stuff`. Y esto es equivalente a llamar al método `__getitem__` en la clase `list` pasando el objeto `stuff` como primer parámetro y la posición que estamos buscando como segundo parámetro.

Al final del programa, el objeto `stuff` se descarta, pero no antes de llamar al **destructor** (llamado `__del__`) para que el objeto pueda limpiar los cabos sueltos según sea necesario.

Esos son los fundamentos y la terminología de la programación orientada a objetos. Hay muchos detalles adicionales sobre cómo usar mejor los enfoques orientados a objetos al desarrollar aplicaciones y bibliotecas grandes que están fuera del alcance de este capítulo.[3]

---

> [1]. https://docs.python.org/3/library/html.parser.html ↩

> [2]. Cookie image copyright CC-BY https://www.flickr.com/photos/dinnerseries/23570475099 ↩

> [3]. Si tienes curiosidad por saber dónde está definida la clase de la lista, echa un vistazo (esperemos que la URL no cambie) https://github.com/python/cpython/blob/master/Objects/listobject.c - La lista de clases está escrita en un lenguaje llamado "C". Si echas un vistazo a ese código fuente y te resulta curioso, quizás quieras explorar algunos cursos de informática. ↩

Copyright 2026

# 15 Python y Bases de datos

# Uso de bases de datos y SQL

## ¿Qué es una base de datos?

Una **base de datos** es un archivo que está organizado para almacenar datos. La mayoría de las bases de datos están organizadas como un diccionario en el sentido de que se asignan de claves a valores. La mayor diferencia es que la base de datos está en el disco (u otro almacenamiento permanente), por lo que persiste después de que finalice el programa. Debido a que una base de datos se almacena en un almacenamiento permanente, puede almacenar muchos más datos que un diccionario, que está limitado al tamaño de la memoria en el ordenador.

Al igual que un diccionario, el software de base de datos está diseñado para mantener la inserción y el acceso de datos muy rápidos, incluso para grandes cantidades de datos. El software de la base de datos mantiene su rendimiento mediante la creación de índices a medida que se agregan datos para permitir que el ordenador salte rápidamente a una entrada en particular.

Hay muchos sistemas de bases de datos diferentes que se utilizan para una amplia variedad de propósitos, incluyendo: Oracle, MySQL, Microsoft SQL Server, PostgreSQL y SQLite. Nos centramos en SQLite en este libro porque es una base de datos muy común y ya está integrada en Python. SQLite está diseñado para ser **integrada** en otras aplicaciones para proporcionar soporte de base de datos dentro de la aplicación. Por ejemplo, el navegador Firefox también usa la base de datos SQLite internamente al igual que muchos otros productos.

http://sqlite.org/

SQLite se adapta bien a algunos de los problemas de manipulación de datos que vemos en Informática, como la aplicación de rastreo de Twitter que describimos en este capítulo.

## Conceptos de base de datos

Cuando miras por primera vez una base de datos, parece una hoja de cálculo con varias hojas. Las estructuras de datos principales en una base de datos son: **tablas**, **filas** y **columnas**.

Copyright 2026

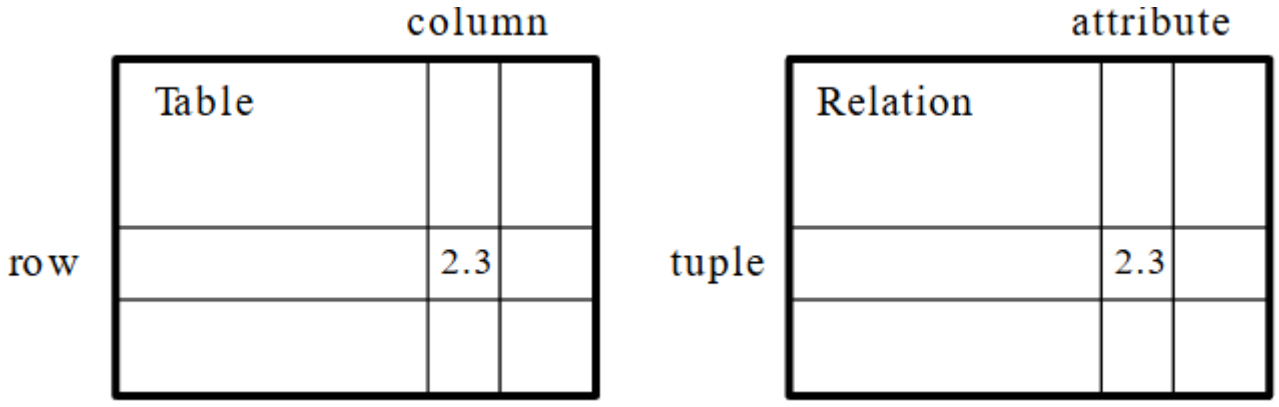



Imagen - Bases de datos relacionales

En las descripciones técnicas de las bases de datos relacionales, los conceptos de tabla, fila y columna se denominan más formalmente **relación**, **tupla** y **atributo**, respectivamente. Usaremos los términos menos formales en este capítulo.

# Database Browser for SQLite

Si bien este capítulo se centrará en el uso de Python para trabajar con datos en archivos de base de datos SQLite, muchas operaciones se pueden realizar de manera más conveniente utilizando el software denominado **Database Browser for SQLite**, que está disponible gratuitamente en:

http://sqlitebrowser.org/

Con el navegador puedes crear fácilmente tablas, insertar datos, editar datos o ejecutar consultas SQL simples.

En cierto sentido, el navegador de la base de datos es similar a un editor de texto cuando se trabaja con archivos de texto. Cuando desees realizar una o muy pocas operaciones en un archivo de texto, puedes abrirlo en un editor de texto y realizar los cambios que desees. Cuando tienes muchos cambios que debes hacer en un archivo de texto, a menudo escribirás un programa Python simple. Encontrarás el mismo patrón cuando trabajes con bases de datos. Harás operaciones simples en el administrador de bases de datos y las operaciones más complejas se realizarán de manera más conveniente en Python.

# Creando una tabla de base de datos

Las bases de datos requieren una estructura más definida que las listas o los diccionarios de Python [1].

Cuando creamos una tabla en la base de datos debemos decir a la base de datos por adelantado los nombres de cada una de las **columnas** en la tabla y el tipo de datos que planeamos almacenar en cada cada una de ellas. Cuando el software de la base de datos conoce el tipo de datos en cada columna, puedes elegir la forma más eficiente de almacenar y buscar los datos según el tipo de datos.

Puedes ver los diversos tipos de datos admitidos por SQLite en la siguiente url:

http://www.sqlite.org/datatypes.html

Definir la estructura de tus datos por adelantado puede parecer inconveniente al principio, pero la recompensa es un acceso rápido a tus datos, incluso cuando la base de datos contenga una gran cantidad de datos.

El código para crear un archivo de base de datos y una tabla llamada `Tracks` con dos columnas en la base de datos es la siguiente:

https://trinket.io/embed/python3/09d1152020

La operación `connect` hace una "conexión" a la base de datos almacenada en el archivo `music.sqlite3` en el directorio actual. Si el archivo no existe, será creado. La razón por la que esto se denomina "conexión" es que a veces la base de datos se almacena en un "servidor de base de datos" independiente del servidor en el que ejecutamos nuestra aplicación. En nuestros ejemplos simples, la base de datos solo será un archivo local en el mismo directorio que el código Python que estamos ejecutando.

Un **cursor** es como un identificador de archivo que podemos usar para realizar operaciones en los datos almacenados en la base de datos. Llamar a `cursor()` es muy similar conceptualmente a llamar a `open()` cuando se trata de archivos de texto.

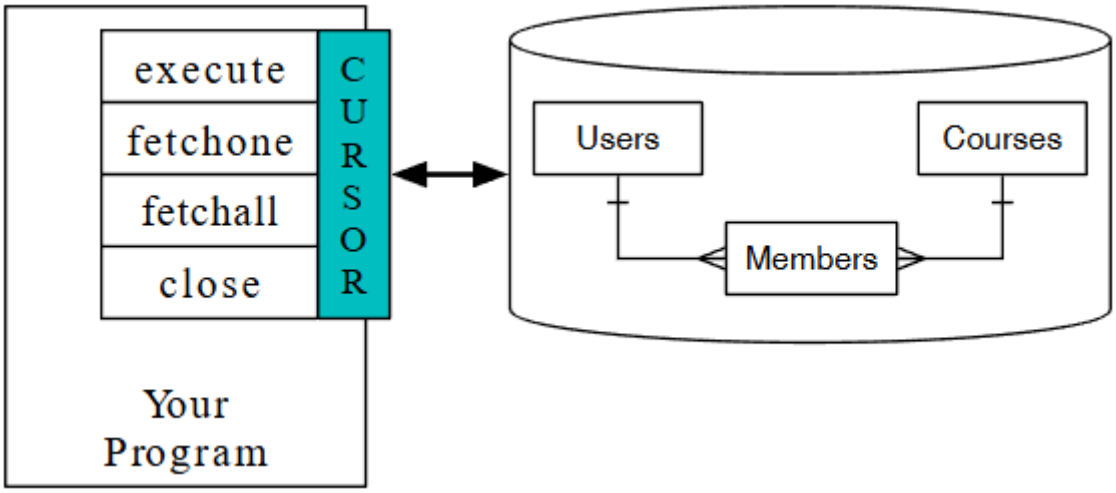


Imagen - A Database Cursor

Una vez que tengamos el cursor, podemos comenzar a ejecutar comandos en el contenido de la base de datos usando el método `execute()`.

Los comandos de la base de datos se expresan en un lenguaje especial que se ha estandarizado en muchos proveedores de bases de datos diferentes para permitirnos aprender un solo idioma de base de datos. El lenguaje de la base de datos se llama **Lenguaje de consulta estructurada** o **SQL** (Structured Query Language) para abreviar.

http://en.wikipedia.org/wiki/SQL

En nuestro ejemplo, estamos ejecutando dos comandos SQL en nuestra base de datos. Como convención, mostraremos las palabras clave de SQL en mayúsculas y las partes del comando que estamos agregando (como los nombres de tablas y columnas) se mostrarán en minúsculas.

El primer comando SQL elimina la tabla `Tracks` de la base de datos si existe. Este patrón es simplemente para permitirnos ejecutar el mismo programa para crear la tabla `Tracks` una y otra vez sin causar un error. Tenga en cuenta que el comando `DROP TABLE` elimina la tabla y todo su contenido de la base de datos (es decir, no hay "deshacer").

```
cur.execute('DROP TABLE IF EXISTS Tracks ')
```

El segundo comando crea una tabla llamada `Tracks` con una columna de texto llamada `title` y una columna entera llamada `plays`.

```
cur.execute('CREATE TABLE Tracks (title TEXT, plays INTEGER)')
```

Ahora que hemos creado una tabla llamada `Tracks`, podemos poner algunos datos en esa tabla usando la operación SQL `INSERT`. Nuevamente, comenzamos haciendo una conexión a la base de datos y obteniendo el `cursor`. Luego podemos ejecutar comandos SQL usando el cursor.

El comando SQL `INSERT` indica qué tabla estamos usando y luego define una nueva fila al enumerar los campos que queremos incluir `(título, jugadas)` seguido de los `VALORES` que queremos colocar en la nueva fila. Especificamos los valores como signos de interrogación `(?,?)` Para indicar que los valores reales se pasan como una tupla `('My Way', 15)` como el segundo parámetro a la llamada `execute()`.

https://trinket.io/embed/python3/e665c26f67

Primero, insertamos dos filas en nuestra tabla y usamos `commit()` para forzar que los datos se escriban en el archivo de la base de datos.

Tracks

| title | plays |
|---|---|
| Thunderstruck | 20 |
| My Way | 15 |

Imagen - Filas en una tabla

Luego usamos el comando `SELECT` para recuperar las filas que acabamos de insertar de la tabla. En el comando `SELECT`, indicamos en qué columnas nos gustaría extraer, `(título, jugamos)`, e indicamos de qué tabla queremos recuperar los datos. Después de ejecutar la instrucción `SELECT`, el cursor es algo que podemos recorrer en una instrucción `for`. Por eficiencia, el cursor no lee todos los datos de la base de datos cuando ejecutamos la instrucción `SELECT`. En su lugar, los datos se leen a medida que recorramos las filas en la declaración `for`.

La salida del programa es la siguiente:

```
Tracks:
('Thunderstruck', 20)
('My Way', 15)
```

Nuestro bucle `for` encuentra dos filas, y cada fila es una tupla de Python con el primer valor como `title` y el segundo valor como el número de `plays`.

**Nota: Puede ver cadenas que comienzan con `u'` en otros libros o en Internet. Esto era una indicación en Python 2 de que las cadenas son** Unicode *cadenas que son capaces de almacenar conjuntos de caracteres no latinos. En Python 3, todas las cadenas son cadenas Unicode por defecto.*

Al final del programa, ejecutamos un comando SQL para 'BORRAR' las filas que acabamos de crear para que podamos ejecutar el programa una y otra vez. El comando `DELETE` muestra el uso de una cláusula `WHERE` que nos permite expresar un criterio de selección para que podamos pedir a la base de datos que aplique el comando solo a las filas que coincidan con el criterio. En este ejemplo, el criterio pasa a aplicarse a todas las filas, de modo que vaciamos la tabla para poder ejecutar el programa repetidamente. Después de que se realiza el `BORRAR`, también llamamos a `commit ()` para forzar que los datos se eliminen de la base de datos.

# Resumen de lenguaje de consulta estructurado

Hasta ahora, hemos estado utilizando el lenguaje de consulta estructurado en nuestros ejemplos de Python y hemos cubierto muchos de los conceptos básicos de los comandos SQL. En esta sección, analizamos el lenguaje SQL en particular y ofrecemos una descripción general de la sintaxis de SQL.

Dado que hay tantos proveedores de bases de datos diferentes, el lenguaje de consulta estructurado (SQL) se estandarizó para que pudiéramos comunicarnos facilmente con los sistemas de bases de datos de varios proveedores.

Una base de datos relacional está formada por tablas, filas y columnas. Las columnas generalmente tienen un tipo como datos de texto, numéricos o de fecha. Cuando creamos una tabla, indicamos los nombres y tipos de las columnas:

```
CREATE TABLE Tracks (title TEXT, plays INTEGER)
```

Para insertar una fila en una tabla, usamos el comando SQL `INSERT`:

```
INSERT INTO Tracks (title, plays) VALUES ('My Way', 15)
```

La declaración `INSERT` especifica el nombre de la tabla, luego una lista de los campos/columnas que te gustaría establecer en la nueva fila, y luego la palabra clave `VALUES` y una lista de valores correspondientes para cada uno de los campos.

El comando SQL `SELECT` se usa para recuperar filas y columnas de una base de datos. La declaración `SELECT` te permite especificar qué columnas deseas recuperar, así como una cláusula `WHERE` para seleccionar qué filas deseas ver. También permite una cláusula opcional `ORDER BY` para controlar la clasificación de las filas devueltas.

```
SELECT * FROM Tracks WHERE title = 'My Way'
```

Usar `*` indica que deseas que la base de datos devuelva todas las columnas para cada fila que coincida con la cláusula `WHERE`.

Ten en cuenta que, a diferencia de Python, en una cláusula SQL `WHERE` usamos un único signo igual para indicar una prueba de igualdad en lugar de un doble signo igual. Otras operaciones lógicas permitidas en una cláusula `WHERE` incluyen `<`, `>`, `<=`, `>=`, `!=`, así como `AND` y `OR` y paréntesis

Copyright 2026

para construir tus expresiones lógicas.

Puedes solicitar que las filas devueltas se ordenen por uno de los campos de la siguiente manera:

```
SELECT title,plays FROM Tracks ORDER BY title
```

Para eliminar una fila, necesitas una cláusula `WHERE` en una sentencia SQL `DELETE`. La cláusula `WHERE` determina qué filas se van a eliminar:

```
DELETE FROM Tracks WHERE title = 'My Way'
```

Es posible `UPDATE` una columna o columnas dentro de una o más filas en una tabla usando la declaración SQL `UPDATE` de la siguiente manera:

```
UPDATE Tracks SET plays = 16 WHERE title = 'My Way'
```

La declaración `UPDATE` especifica una tabla y luego una lista de campos y valores para cambiar después de la palabra clave `SET` y luego una cláusula opcional `WHERE` para seleccionar las filas que se actualizarán. Una sola instrucción `UPDATE` cambiará todas las filas que coincidan con la cláusula `WHERE`. Si no se especificas una cláusula `WHERE`, realiza el `UPDATE` en todas las filas de la tabla.

Estos cuatro comandos SQL básicos (INSERT, SELECT, UPDATE, y DELETE) permiten las cuatro operaciones básicas necesarias para crear y mantener datos.

# Spidering Twitter usando una base de datos

En esta sección, crearemos un programa de spidering simple que pasará por las cuentas de Twitter y creará una base de datos de ellas. **Nota: Ten mucho cuidado al ejecutar este programa. No deseas extraer demasiados datos o ejecutar el programa durante demasiado tiempo y terminar con el cierre de tu acceso a Twitter.**

Uno de los problemas de cualquier tipo de programa spidering es que debe poder detenerse y reiniciarse muchas veces y no quieres perder los datos que has recuperado hasta ahora. No siempre deseas reiniciar tu recuperación de datos desde el principio, por lo que queremos almacenar datos a medida que los recuperamos para que nuestro programa pueda iniciarse nuevamente y continuar donde lo dejó.

Comenzaremos por recuperar los amigos de Twitter de una persona y sus estados, recorreremos la lista de amigos y agregaremos a cada uno de los amigos a una base de datos para recuperarlos en el futuro. Después de procesar los amigos de Twitter de una persona, revisamos nuestra base de datos y recuperamos a uno de los amigos del amigo. Hacemos esto una y otra vez, seleccionando a una persona "no visitada", recuperando su lista de amigos y agregando amigos que no hemos visto en nuestra lista para una futura visita.

También hacemos un seguimiento de cuántas veces hemos visto a un amigo en particular en la base de datos para tener una idea de su "popularidad".

Al almacenar nuestra lista de cuentas conocidas y si hemos recuperado la cuenta o no, y cuán popular es la cuenta en una base de datos en el disco de el ordenador, podemos detener y reiniciar nuestro programa tantas veces como lo deseemos.

Este programa es un poco complejo. Se basa en el código del ejercicio anterior en el libro que utiliza la API de Twitter.

Aquí está el código fuente de nuestra aplicación de spidering de Twitter:

https://trinket.io/embed/python3/f9ee36fa1f

Nuestra base de datos se almacena en el archivo `spider.sqlite3` y tiene una tabla llamada `Twitter`. Cada fila en la tabla `Twitter` tiene una columna para el nombre de la cuenta, si hemos recuperado los amigos de esta cuenta y cuántas veces esta cuenta ha sido "seguida".

En el ciclo principal del programa, le pedimos al usuario un nombre de cuenta de Twitter o "salir" para salir del programa. Si el usuario ingresa una cuenta de Twitter, recuperamos la lista de amigos y estados de ese usuario y agregamos a cada amigo a la base de datos si aún no está en la base de datos. Si el amigo ya está en la lista, agregamos 1 al campo `friends` en la fila de la base de datos.

Si el usuario presiona Intro, buscamos en la base de datos la próxima cuenta de Twitter que aún no hemos recuperado, recuperamos los amigos y los estados de esa cuenta, los agregamos a la base de datos o los actualizamos, y aumentamos el recuento de sus "amigos".

Una vez que recuperamos la lista de amigos y estados, recorremos todos los elementos `user` en el JSON devuelto y recuperamos el `screen_name` para cada usuario. Luego usamos la declaración `SELECT` para ver si ya hemos almacenado este `screen_name` en particular en la base de datos y recuperar el recuento de amigos (`friends`) si el registro existe.

```
countnew = 0
countold = 0
for u in js['users'] :
    friend = u['screen_name']
    print friend
    cur.execute('SELECT friends FROM Twitter WHERE name = ? LIMIT 1',
        (friend, ) )
    try:
        count = cur.fetchone()[0]
        cur.execute('UPDATE Twitter SET friends = ? WHERE name = ?',
            (count+1, friend) )
        countold = countold + 1
    except:
        cur.execute('''INSERT INTO Twitter (name, retrieved, friends)
            VALUES ( ?, 0, 1 )''', ( friend, ) )
        countnew = countnew + 1
print 'New accounts=',countnew,' revisited=',countold
conn.commit()
```

Una vez que el cursor ejecuta la instrucción `SELECT`, debemos recuperar las filas. Podríamos hacer esto con una declaración `for`, pero como solo estamos recuperando una fila (`LIMIT 1`), podemos usar el método `fetchone()` para recuperar la primera (y única) fila que es el resultado de la operación `SELECT`. Como `fetchone()` devuelve la fila como **tupla** (aunque solo haya un campo), tomamos el primer valor de la tupla para obtener el recuento actual de amigos y guardarlo en la variable `count`.

Si esta recuperación es exitosa, usamos la declaración SQL `UPDATE` con una cláusula `WHERE` para agregar 1 a la columna `friends` para la fila que coincide con la cuenta del amigo. Observa que hay dos marcadores de posición (es decir, signos de interrogación) en el SQL, y el segundo parámetro de `execute()` es una tupla de dos elementos que contiene los valores que se sustituirán en el SQL en lugar de los signos de interrogación.

Si el código en el bloque `try` falla, es probable que no haya ningún registro que coincida con la cláusula WHERE name =? en la instrucción SELECT. Entonces, en el bloque `except`, usamos la declaración SQL `INSERT` para agregar el `screen_name` del amigo a la tabla con una indicación de que aún no hemos recuperado el `screen_name` y establecer el recuento de amigos en cero.

Entonces, la primera vez que se ejecuta el programa y entramos en una cuenta de Twitter, el programa se ejecuta de la siguiente manera:

```
Enter a Twitter account, or quit: drchuck
Retrieving http://api.twitter.com/1.1/friends ...
New accounts= 20  revisited= 0
Enter a Twitter account, or quit: quit
```

Como esta es la primera vez que ejecutamos el programa, la base de datos está vacía y creamos la base de datos en el archivo `spider.sqlite3` y agregamos una tabla llamada `Twitter` a la base de datos. Luego recuperamos algunos amigos y los agregamos a la base de datos, que está vacía.

En este punto, podríamos querer escribir un dumper de base de datos simple para echar un vistazo a lo que está en nuestro archivo `spider.sqlite3`:

https://trinket.io/embed/python3/e986702fd6

Este programa simplemente abre la base de datos y selecciona todas las columnas de todas las filas en la tabla `Twitter`, luego recorre las filas e imprime cada fila.

Si ejecutamos este programa después de la primera ejecución de nuestra araña de Twitter anterior, su salida será la siguiente:

```
('opencontent', 0, 1)
('lhawthorn', 0, 1)
('steve_coppin', 0, 1)
('davidkocher', 0, 1)
('hrheingold', 0, 1)
...
20 rows.
```

Vemos una fila para cada `screen_name`, que no hemos recuperado los datos para ese `screen_name`, y todos en la base de datos tienen un amigo.

Ahora nuestra base de datos refleja la recuperación de los amigos de nuestra primera cuenta de Twitter (**drchuck**). Podemos ejecutar el programa nuevamente y decirle que recupere a los amigos de la próxima cuenta "sin procesar" simplemente presionando Intro en lugar de una cuenta de Twitter de la siguiente manera:

```
Enter a Twitter account, or quit:
Retrieving http://api.twitter.com/1.1/friends ...
New accounts= 18  revisited= 2
Enter a Twitter account, or quit:
Retrieving http://api.twitter.com/1.1/friends ...
New accounts= 17  revisited= 3
Enter a Twitter account, or quit: quit
```

Desde que presionamos enter (es decir, no especificamos una cuenta de Twitter), se ejecuta el siguiente código:

```
if ( len(acct) < 1 ) :
    cur.execute('SELECT name FROM Twitter WHERE retrieved = 0 LIMIT 1')
    try:
        acct = cur.fetchone()[0]
    except:
        print 'No unretrieved twitter accounts found'
        continue
```

Usamos la instrucción SQL `SELECT` para recuperar el nombre del primer usuario (`LIMIT 1`) que todavía tiene su valor establecido en cero. También utilizamos el patrón `fetchone() [0]` dentro de un bloque try / except para extraer un `screen_name` de los datos recuperados o para mostrar un mensaje de error y realizar un bucle de copia de seguridad.

Si recuperamos con éxito un `screen_name` sin procesar, recuperamos sus datos de la siguiente manera:

```
url = twurl.augment(TWITTER_URL, {'screen_name': acct, 'count': '20'})
print('Retrieving')
url connection = urllib.urlopen(url)
data = connection.read()
js = json.loads(data)
cur.execute('UPDATE Twitter SET retrieved = 1 WHERE nombre =?', (acct,))
```

Una vez que recuperamos los datos con éxito, usamos la instrucción `UPDATE` para establecer la columna `retrieved` en 1 para indicar que hemos completado la recuperación de los amigos de esta cuenta. Esto nos impide recuperar los mismos datos una y otra vez y nos mantiene progresando a través de la red de amigos de Twitter.

Si ejecutamos el programa de amigos y presionamos Intro dos veces para recuperar a los amigos de los siguientes amigos no visitados, luego ejecutamos el programa de descarga, nos dará el siguiente resultado:

```
('opencontent', 1, 1)
('lhawthorn', 1, 1)
('steve_coppin', 0, 1)
('davidkocher', 0, 1)
('hrheingold', 0, 1)
...
('cnxorg', 0, 2)
('knoop', 0, 1)
('kthanos', 0, 2)
('LectureTools', 0, 1)
...
55 rows.
```

Podemos ver que hemos registrado correctamente que hemos visitado `lhawthorn` y `opencontent`. También las cuentas `cnxorg` y `kthanos` ya tienen dos seguidores. Como ahora hemos recuperado los amigos de tres personas (`drchuck`, `opencontent` y `lhawthorn`), nuestra tabla tiene 55 filas de amigos para recuperar.

Cada vez que ejecutamos el programa y presionamos enter, elegimos la siguiente cuenta no visitada (por ejemplo, la siguiente cuenta será `steve_coppin`), recuperará a sus amigos, los marcará como recuperados y, para cada uno de los amigos de `steve_coppin`, agregará hasta el final de la base de datos o actualizará el recuento de sus amigos si ya están en la base de datos.

Dado que todos los datos del programa se almacenan en el disco en una base de datos, la actividad de rastreo se puede suspender y reanudar tantas veces como desee sin perder datos.

# Modelado básico de datos

El poder real de una base de datos relacional es cuando creamos varias tablas y hacemos enlaces entre esas tablas. El acto de decidir cómo dividir los datos de su aplicación en varias tablas y establecer las relaciones entre las tablas se denomina **modelado de datos**. El documento de diseño que muestra las tablas y sus relaciones se denomina **modelo de datos**.

Copyright 2026

El modelado de datos es una habilidad relativamente sofisticada y solo presentaremos los conceptos más básicos del modelado de datos relacionales en esta sección. Para más detalles sobre el modelado de datos, puedes comenzar con:

http://en.wikipedia.org/wiki/Relational_model

Digamos que para nuestra aplicación de araña de Twitter, en lugar de simplemente contar los amigos de una persona, queríamos mantener una lista de todas las relaciones entrantes para poder encontrar una lista de todas las personas que siguen una cuenta en particular.

Dado que potencialmente todos tendrán muchas cuentas que los siguen, no podemos simplemente agregar una sola columna a nuestra tabla `Twitter`. Así que creamos una nueva tabla que hace un seguimiento de parejas de amigos. La siguiente es una forma simple de hacer una tabla de este tipo:

```
CREATE TABLE Pals (from_friend TEXT, to_friend TEXT)
```

Cada vez que nos encontramos con una persona que está siguiendo `drchuck`, insertamos una fila del formulario:

```
INSERT INTO Pals (from_friend,to_friend) VALUES ('drchuck', 'lhawthorn')
```

Como estamos procesando a los 20 amigos del feed de Twitter `drchuck`, insertaremos 20 registros con "drchuck" como primer parámetro, por lo que terminaremos duplicando la cadena muchas veces en la base de datos.

Esta duplicación de datos de cadena viola una de las mejores prácticas para **la normalización de la base de datos** que básicamente dice que nunca debemos colocar los mismos datos de cadena en la base de datos más de una vez. Si necesitamos los datos más de una vez, creamos una clave numérica para los datos y hacemos referencia a los datos reales utilizando esta clave.

En términos prácticos, una cadena ocupa mucho más espacio que un entero en el disco y en la memoria de nuestra ordenador, y toma más tiempo de procesador para comparar y ordenar. Si solo tenemos unos pocos cientos de entradas, el tiempo de almacenamiento y procesador apenas importa. Pero si tenemos un millón de personas en nuestra base de datos y una posibilidad de 100 millones de enlaces de amigos, es importante poder escanear los datos lo más rápido posible.

Almacenaremos nuestras cuentas de Twitter en una tabla llamada `People` en lugar de la tabla `Twitter` utilizada en el ejemplo anterior. La tabla `People` tiene una columna adicional para almacenar la clave numérica asociada a la fila para este usuario de Twitter. SQLite tiene una función que agrega automáticamente el valor clave para cualquier fila que insertamos en una tabla

usando un tipo especial de columna de datos (`INTEGER PRIMARY KEY`).

Podemos crear la tabla `People` con esta columna adicional `id` de la siguiente manera:

```
CREATE TABLE People (id INTEGER PRIMARY KEY, name TEXT UNIQUE, retrieved INTEGER)
```

Ten en cuenta que ya no estamos manteniendo un recuento de amigos en cada fila de la tabla "People". Cuando seleccionamos `INTEGER PRIMARY KEY` como el tipo de nuestra columna `id`, estamos indicando que nos gustaría que SQLite administre esta columna y asigne una clave numérica única a cada fila que insertamos automáticamente. También agregamos la palabra clave `UNIQUE` para indicar que no permitiremos que SQLite inserte dos filas con el mismo valor para `name` .

Ahora, en lugar de crear la tabla `Pals` anterior, creamos una tabla llamada `Follows` con dos columnas enteras `from_id` y `to_id` y una restricción en la tabla que la combinación de `from_id` y `to_id` debe ser única en esta tabla (es decir, no podemos insertar filas duplicadas) en nuestra base de datos.

```
CREATE TABLE Follows
    (from_id INTEGER, to_id INTEGER, UNIQUE(from_id, to_id) )
```

Cuando agregamos cláusulas `UNIQUE` a nuestras tablas, estamos comunicando un conjunto de reglas que le estamos pidiendo a la base de datos que aplique cuando intentemos insertar registros. Las reglas nos impiden cometer errores y simplifican la escritura de algunos de nuestros códigos.

En esencia, al crear esta tabla "Follows", estamos modelando una "relación" donde una persona "sigue" a otra persona y la representamos con un par de números que indican que (a) las personas están conectadas y (b) la dirección de la relación.

Relaciones entre tablas
Imagen - Relaciones entre tablas

# Programación con varias tablas

Ahora reharemos el programa de arañas de Twitter utilizando dos tablas, las claves principales y las referencias clave, como se describe anteriormente. Aquí está el código para la nueva versión del programa:

https://trinket.io/embed/python3/88efa27a18

Este programa está empezando a complicarse un poco, pero ilustra los patrones que debemos usar cuando estamos usando claves de enteros para vincular tablas. Los patrones básicos son:

1. Crear tablas con claves primarias y restricciones.
2. Cuando tenemos una clave lógica para una persona (es decir, el nombre de la cuenta) y necesitamos el valor `id` para la persona, dependiendo de si la persona ya está en la tabla `People` o bien debemos: (1) buscar la persona en la tabla `People` y recuperar el valor `id` para la persona o (2) agrear la persona a la tabla `People` y obtener el valor `id` para la fila recién agregada.
3. Inserta la fila que captura la relación "follows".

Desarrollaremos brevemente estos puntos.

# Restricciones en tablas de bases de datos

A medida que diseñamos nuestras estructuras de tablas, podemos decirle al sistema de base de datos que nos gustaría aplicar algunas reglas. Estas reglas nos ayudan a cometer errores e introducir datos incorrectos en las tablas. Cuando creamos nuestras tablas:

```
cur.execute('''CREATE TABLE IF NOT EXISTS People
    (id INTEGER PRIMARY KEY, name TEXT UNIQUE, retrieved INTEGER)''')
cur.execute('''CREATE TABLE IF NOT EXISTS Follows
    (from_id INTEGER, to_id INTEGER, UNIQUE(from_id, to_id))''')
```

Indicamos que la columna `name` en la tabla `People` debe ser `UNIQUE`. También indicamos que la combinación de los dos números en cada fila de la tabla `Follows` debe ser única. Estas restricciones nos impiden cometer errores, como agregar la misma relación más de una vez.

Podemos aprovechar estas restricciones en el siguiente código:

```
cur.execute('''INSERT OR IGNORE INTO People (name, retrieved)
    VALUES ( ?, 0)''', ( friend, ) )
```

Añadimos la cláusula `OR IGNORE` a nuestra declaración `INSERT` para indicar que si este `INSERT` en particular causaría una violación de la regla `name` *debe ser único*, el sistema de la base de datos puede ignorar el `INSERT` . Estamos utilizando la restricción de la base de datos como una red de seguridad para asegurarnos de no hacer algo incorrecto sin querer.

De manera similar, el siguiente código asegura que no agregamos la misma relación exacta de `Follows` dos veces.

```
cur.execute('''INSERT OR IGNORE INTO Follows
    (from_id, to_id) VALUES (?, ?)''', (id, friend_id) )
```

Una vez más, simplemente le pedimos a la base de datos que ignore nuestro intento de `INSERT` si viola la restricción de unicidad que especificamos para las filas de `Follows`.

# Recuperar y/o insertar un registro

Cuando solicitamos al usuario una cuenta de Twitter, si la cuenta existe, debemos buscar su valor 'id'. Si la cuenta aún no existe en la tabla `People`, debemos insertar el registro y obtener el valor `id` de la fila insertada.

Este es un patrón muy común y se realiza dos veces en el programa anterior. Este código muestra cómo buscamos el `id` para la cuenta de un amigo cuando hemos extraído un `screen_name` de un nodo `usuario` en el JSON de Twitter recuperado.

Dado que con el tiempo será cada vez más probable que la cuenta ya esté en la base de datos, primero verificamos si existe el registro 'Personas' usando una declaración `SELECT`.

Si todo va bien [2] dentro de la sección `try`, recuperamos el registro usando `fetchone()` y luego recuperamos el primer (y único) elemento de la tupla devuelta y lo guardamos en `friend_id`.

Si el `SELECT` falla, el código `fetchone()[0]` fallará y el control se transferirá a la sección `except`.

```
    friend = u['screen_name']
    cur.execute('SELECT id FROM People WHERE name = ? LIMIT 1',
        (friend, ) )
    try:
        friend_id = cur.fetchone()[0]
        countold = countold + 1
    except:
        cur.execute('''INSERT OR IGNORE INTO People (name, retrieved)
            VALUES ( ?, 0)''', ( friend, ) )
        conn.commit()
        if cur.rowcount != 1 :
            print 'Error inserting account:',friend
```

```
            continue
        friend_id = cur.lastrowid
        countnew = countnew + 1
```

Si terminamos en el código `except`, simplemente significa que no se encontró la fila, por lo que debemos insertar la fila. Usamos `INSERT OR IGNORE` solo para evitar errores y luego llamamos a `commit ()` para forzar que la base de datos se actualice realmente. Una vez que se realiza la escritura, podemos verificar el `cur.rowcount` para ver cuántas filas se vieron afectadas. Ya que estamos intentando insertar una sola fila, si el número de filas afectadas es diferente a 1, es un error.

Si el `INSERT` tiene éxito, podemos ver `cur.lastrowid` para averiguar qué valor asignó la base de datos a la columna `id` en nuestra fila recién creada.

# Guardando la relación de amistad

Una vez que sepamos el valor clave tanto para el usuario de Twitter como para el amigo en el JSON, es muy sencillo insertar los dos números en la tabla `Siguiendo 'con el siguiente código:

```
cur.execute('INSERT OR IGNORE INTO Follows (from_id, to_id) VALUES (?, ?)',
    (id, friend_id) )
```

Ten en cuenta que dejamos que la base de datos se encargue de evitar que "insertemos dos veces" una relación creando la tabla con una restricción de unicidad y luego agregando `OR IGNORE` a nuestra declaración `INSERT`.

Aquí hay una ejecución de muestra de este programa:

```
Enter a Twitter account, or quit:
No unretrieved Twitter accounts found
Enter a Twitter account, or quit: drchuck
Retrieving http://api.twitter.com/1.1/friends ...
New accounts= 20  revisited= 0
Enter a Twitter account, or quit:
Retrieving http://api.twitter.com/1.1/friends ...
New accounts= 17  revisited= 3
Enter a Twitter account, or quit:
Retrieving http://api.twitter.com/1.1/friends ...
```

```
New accounts= 17  revisited= 3
Enter a Twitter account, or quit: quit
```

Comenzamos con la cuenta `drchuck` y luego dejamos que el programa seleccione automáticamente las siguientes dos cuentas para recuperar y agregar a nuestra base de datos.

A continuación se muestran las primeras filas de las tablas `People` y `Follows` después de completar esta ejecución:

```
People:
(1, 'drchuck', 1)
(2, 'opencontent', 1)
(3, 'lhawthorn', 1)
(4, 'steve_coppin', 0)
(5, 'davidkocher', 0)
55 rows.
Follows:
(1, 2)
(1, 3)
(1, 4)
(1, 5)
(1, 6)
60 rows.
```

Puedes ver los campos `id`, `name` y `visited` en la tabla `People` y puedes ver los números de ambos extremos de la relación en la tabla `Follows`. En la tabla `People`, podemos ver que las tres primeras personas han sido visitadas y sus datos han sido recuperados. Los datos en la tabla `Follows` indican que `drchuck` (usuario 1) es un amigo para todas las personas que aparecen en las primeras cinco filas. Esto tiene sentido porque los primeros datos que recuperamos y almacenamos fueron los amigos de Twitter de `drchuck`. Si tuvieras que imprimir más filas de la tabla `Follows`, también verías a los amigos de los usuarios 2 y 3.

# Tres tipos de teclas

TODO sigue por aquí Ahora que hemos empezado a construir un modelo que coloca nuestros datos en varias tablas vinculadas y vincula las filas en esas tablas usando **claves**, debemos analizar la terminología relacionada con las claves. En general, hay tres tipos de claves utilizadas en un

modelo de base de datos.

- Una **clave lógica** es una clave que el "mundo real" puede usar para buscar una fila. En nuestro modelo de datos de ejemplo, el campo `nombre` es una clave lógica. Es el nombre de pantalla para el usuario y, de hecho, buscamos la fila de un usuario varias veces en el programa usando el campo `nombre`. A menudo verás que tiene sentido agregar una restricción `UNIQUE` a una clave lógica. Dado que la clave lógica es la forma en que buscamos una fila del mundo exterior, no tiene mucho sentido permitir varias filas con el mismo valor en la tabla.
- Una **clave principal** suele ser un número que la base de datos asigna automáticamente. Por lo general, no tiene ningún significado fuera del programa y solo se utiliza para vincular filas de diferentes tablas. Cuando queremos buscar una fila en una tabla, generalmente buscar la fila con la clave principal es la forma más rápida de encontrar la fila. Dado que las claves primarias son números enteros, ocupan muy poco espacio de almacenamiento y se pueden comparar o clasificar muy rápidamente. En nuestro modelo de datos, el campo `id` es un ejemplo de una clave primaria.
- Una **clave externa** es generalmente un número que apunta a la clave principal de una fila asociada en una tabla diferente. Un ejemplo de una clave externa en nuestro modelo de datos es el `from_id`.

Estamos utilizando una convención de nomenclatura para llamar siempre el nombre de campo de la clave principal `id` y añadir el sufijo `_id` a cualquier nombre de campo que sea una clave externa.

# Uso de JOIN para recuperar datos

Ahora que hemos seguido las reglas de normalización de la base de datos y hemos separado los datos en dos tablas, vinculadas entre sí mediante claves primarias y externas, debemos poder construir un `SELECT` que vuelva a ensamblar los datos en todas las tablas.

SQL usa la cláusula `JOIN` para volver a conectar estas tablas. En la cláusula `JOIN` usted especifica los campos que se utilizan para volver a conectar las filas entre las tablas.

El siguiente es un ejemplo de un `SELECT` con una cláusula `JOIN`:

```
SELECT * FROM Follows JOIN People
    ON Follows.from_id = People.id WHERE People.id = 1
```

La cláusula `JOIN` indica que los campos que estamos seleccionando cruzan las tablas `Follows` y `People`. La cláusula `ON` indica cómo se deben unir las dos tablas: toma las filas de `Follows` y

agrega la fila de `People` donde el campo `from_id` en `Follows` es el mismo que el valor de `id` en el Tabla de personas.

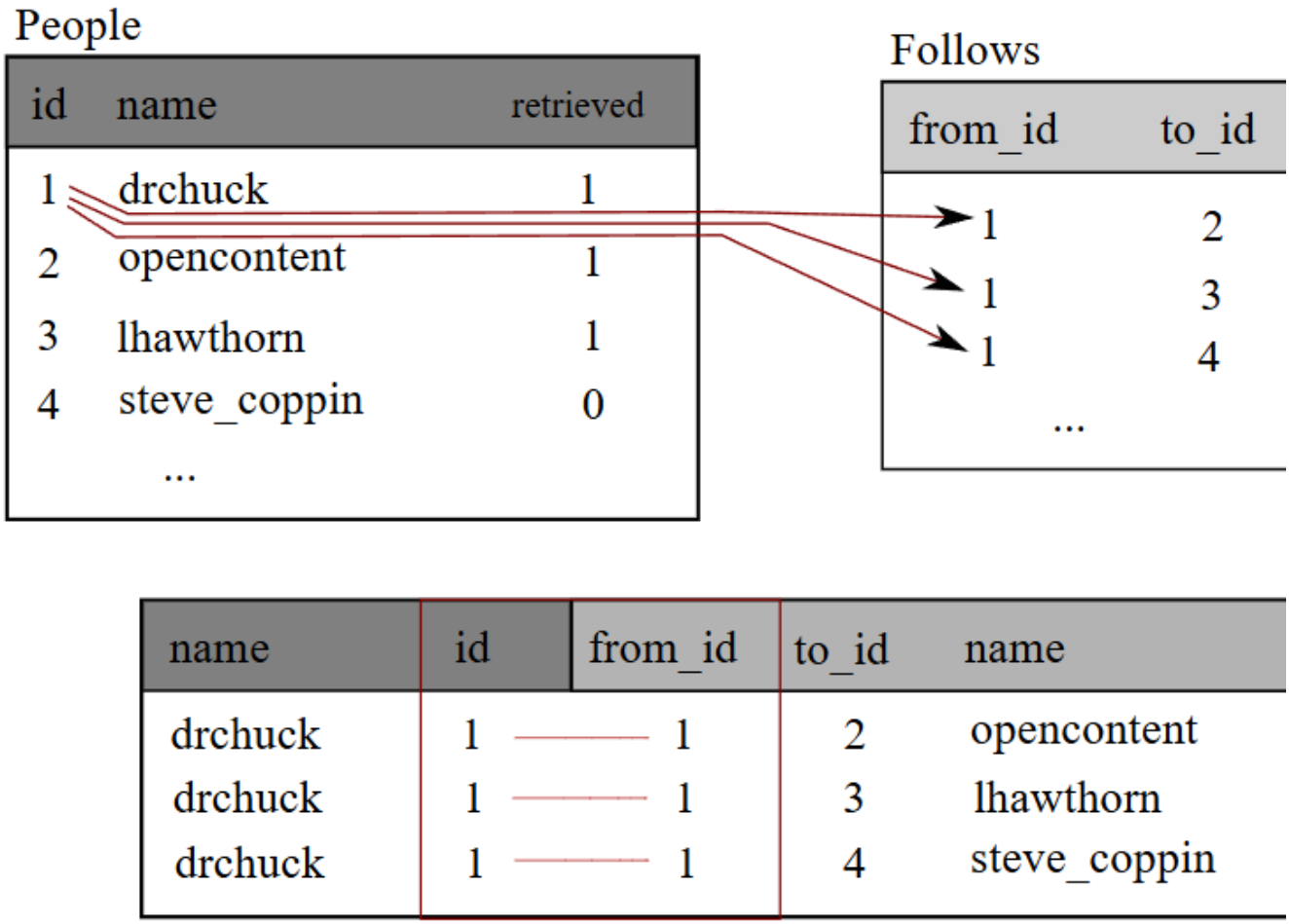



Imagen - Conectando tablas utilizando JOIN

El resultado de JOIN es crear "metarows" extra largos que tienen los campos de "Personas" y los campos correspondientes de los "Follows". Donde hay más de una coincidencia entre el campo `id` de `People` y el `from_id` de `People`, entonces JOIN crea un metarow para cada uno de los pares de filas correspondientes, duplicando los datos según sea necesario.

El siguiente código muestra los datos que tendremos en la base de datos después de que se haya ejecutado varias veces el programa de arañas de Twitter de múltiples tablas (arriba).

https://trinket.io/embed/python3/8f92ad783f

En este programa, primero desechamos `People` y `Follows` y luego volcamos un subconjunto de los datos en las tablas unidas.

Aquí está la salida del programa:

```
python twjoin.py
People:
(1, 'drchuck', 1)
(2, 'opencontent', 1)
(3, 'lhawthorn', 1)
(4, 'steve_coppin', 0)
(5, 'davidkocher', 0)
55 rows.
Follows:
(1, 2)
(1, 3)
(1, 4)
(1, 5)
(1, 6)
60 rows.
Connections for id=2:
(2, 1, 1, 'drchuck', 1)
(2, 28, 28, 'cnxorg', 0)
(2, 30, 30, 'kthanos', 0)
(2, 102, 102, 'SomethingGirl', 0)
(2, 103, 103, 'ja_Pac', 0)
20 rows.
```

Verás las columnas de las tablas `People` y `Follows` y el último conjunto de filas es el resultado de `SELECT` con la cláusula `JOIN`.

En la última selección, estamos buscando cuentas que sean amigos de "opencontent" (es decir, `People.id = 2`).

En cada uno de los "metarows" en la última selección, las dos primeras columnas son de la tabla "Follows" seguidas por las columnas tres a cinco de la tabla "People". También puedes ver que la segunda columna (`Follows.to_id`) coincide con la tercera columna (`People.id`) en cada uno de los "metarows" unidos.

# Resumen

Este capítulo ha cubierto mucho terreno para ofrecerte una descripción general de los conceptos básicos del uso de una base de datos en Python. Es más complicado escribir el código para usar una base de datos para almacenar datos que los diccionarios de Python o los archivos planos, por lo que hay pocas razones para usar una base de datos a menos que su aplicación realmente necesite las capacidades de una base de datos. Las situaciones en las que una base de datos puede ser bastante útil son: (1) cuando tu aplicación necesita realizar muchas actualizaciones aleatorias dentro de un conjunto de datos grande, (2) cuando sus datos son tan grandes que no caben en un diccionario y necesitas buscar y actualizar la información repetidamente, o (3) cuando tengas un proceso de larga ejecución que desees poder detener, reiniciar y conservar los datos de una ejecución a la siguiente.

Puedes crear una base de datos simple con una sola tabla para satisfacer las necesidades de muchas aplicaciones, pero la mayoría de los problemas requerirán varias tablas y vínculos/relaciones entre filas en diferentes tablas. Cuando comienzas a crear enlaces entre tablas, es importante hacer un diseño cuidadoso y seguir las reglas de normalización de la base de datos para aprovechar al máximo sus capacidades. Dado que la motivación principal para usar una base de datos es que tienes una gran cantidad de datos con los que tratar, es importante modelar tus datos de manera eficiente para que tus programas se ejecuten lo más rápido posible.

# depuración

Un patrón común cuando desarrolles un programa en Python para conectarte a una base de datos SQLite será ejecutar un programa y verificar los resultados utilizando el Explorador de bases de datos para SQLite. El navegador te permite verificar rápidamente si tu programa está funcionando correctamente.

Debes tener cuidado porque SQLite se encarga de evitar que dos programas cambien los mismos datos al mismo tiempo. Por ejemplo, si abres una base de datos en el navegador y realizas un cambio en la base de datos y aún no has presionado el botón "guardar" en el navegador, el navegador "bloquea" el archivo de la base de datos y evita que cualquier otro programa acceda al mismo. En particular, tu programa Python no podrá acceder al archivo si está bloqueado.

Por lo tanto, una solución es asegurarte de cerrar el navegador de la base de datos o usar el menú **Archivo** para cerrar la base de datos en el navegador antes de intentar acceder a la base de datos desde Python para evitar el problema.

---

> “ [1]. SQLite realmente permite cierta flexibilidad en el tipo de datos almacenados en una columna, pero mantendremos nuestros tipos de datos estrictos en este

> capítulo para que los conceptos se apliquen por igual a otros sistemas de bases de datos como MySQL. ↩

> “ [1]. En general, cuando una oración comienza con "si todo va bien", encontrarás que el código debe usar try / except. ↩

# Créditos



*Autor Charles R. Severance*

*La web del autor https://www.py4e.com/*

https://www.youtube.com/embed/UjeNA_JtXME?t=3s

Licencia de Creative Commons

Python for person in everybody, escrito a partir del libro 'Python for Everybody' por Charles R. Severance bajo licencia Creative Commons Reconocimiento-NoComercial-CompartirIgual 4.0 Internacional License.